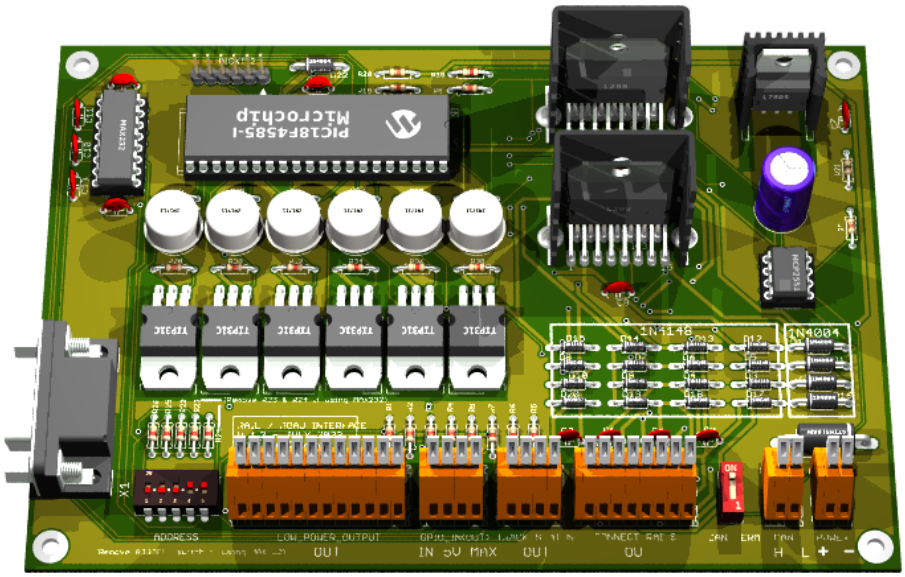
Reference Guide

RAIL / ROAD INTERFACE

V 1.3 (August 2025)



Written by: Philippe Cavenel
17 July 2023
Last update August, 29 2025
Documentation Version 1.11

Table of contents

Illustrations

1 Preamble

1.1 History

History	Exchange	Date
Version 1.0	Creation	18/May/2021
Version 1.1	New PCB design	18/May/2023
Version 1.2	Replace H-bridge transistors with L298s	22/July/2023
Version 1.3	Add more information on last development with PIC18F4680	30/07/2025

1.2 Abbreviations

DCC: Digital Command Control (defined by a NMRA standard)
NMRA: National Model Railroad Association
AMS: Auto Motor Sport (Faller)
HO: Half-O (model railway scale corresponding to 1:87)
DIY: Do It Yourself
PC: Personal Computer
USB: Universal Serial Bus
FTDI: Future Technology Devices International
CAN (bus): Controller Area Network
TTL: Transistor-Transistor logic
GPI : Global Purpose Input-Output
LED: Light-Emitting Diode
PWM: Pulse Width Modulation
EMC: ElectroMagnetic Compatibility
STL: STereo-Lithography
STEP: STandard for the Exchange of Product model data
DRC: Design Rule Check
ERC: Electrical Rule Check

1.3 Typographic convention

The file names are in `Courier New` format, for example `RailDriverCamJob.cam` file

The online order examples or file contents are in bold **`Courier New`** on a grey background, for example

```
stl2pov.exe myModel.stl > myModel.inc.
```

Warning is in bold red inside a frame

Binary names are in bolt like for example **stl2pov.exe**

2 Disclaimer

The information contained in this document has been obtained from sources believed to be reliable. However, it may contain technical inaccuracies or typographical errors.

The author reserves the right to correct such errors as soon as they are brought to his attention. We strongly recommend that you check the accuracy and relevance of the information provided in this document.

The information contained in this document is subject to change at any time, and may have been updated. In particular, it may have been updated between the time it was downloaded and the time the user takes cognizance of it.

Use of the information contained in this document is at the user's sole risk, and the user assumes full responsibility for any consequences arising therefrom, without any liability or recourse against the author.

In no event shall the author be liable for any damages whatsoever resulting from the interpretation or use of the information contained herein.

3 Introduction

3.1 Document purpose

Why this document?

Faller AMS car networks do not have a DCC-type control mode (NMRA standard). This makes it difficult to have several vehicles on the same network. Using diodes, it is possible to operate a maximum of 2 vehicles per track, which is not very many. By creating track sections and vehicle detections per track section, you can have many more vehicles on the track. On the other hand, the realism of these networks is not respected with operating speeds that are too high, and by using PWM-type pulsed current mechanism, a much more realistic effect can be achieved.

As soon as this type of board is developed, it seems interesting to have a universal programmable interface that can also be used to drive a railway network, and thus support DCC mode (NMRA).

Document content

This document describes the development of a DIY Faller AMS network board for HO rail and road models and the various associated software (Firmware for PIC18F4585 and PIC18F4680 and PC software) developed since 2021.

The PIC18F4680 allows for storing more program data in memory; the only difference in firmware programming is the use of the correct 18f4680_g.lkr file during compilation. The use of this microcontroller occurred during the course of the project development, so it is possible that other models may still be referenced in this document. However, the code available on GitHub has been properly developed for the 4680 version of the PIC18F.

The board is designed to communicate with a PC via an RS232 serial link (or via a USB connection using an FTDI cable connected to the RS232 connector, available at Amazon: DSD TECH SH-RS232G USB to DB9 Female serial cable Integrated FTDI FT232RL chip) and with each other via a CAN bus.

Each board can drive up to 4 sections in analog or digital DCC mode (NMRA standard) with electric vehicle presence detection, and 6 low-power controls for lighting or small mechanisms (such as turnout controls). Finally, 8 TTL inputs/outputs (GPIOs) 4 are dedicated to vehicle presence detection, and 4 enable the addition of detectors, control LEDs, or the linking of boards to generate automatic control mechanisms (blocking).

Boards cannot individually mix DCC and analog modes, but it is possible to run one board in DCC mode and another in analog mode.

Each board has a 5-bit address (so 31 can be used, i.e. 128 slots for the whole network). The PC interface board called the "master" is set to address 31 by default. Each other boards should have at least the bit 0 or 1 or 2 set to 0.

Warning

The current 2023 version at the time of writing is V1.2, and this document describes this version.

This board was developed for personal usage and on an old but readily available and quickly mastered environment that can simply be replaced by a tool like KiCad EDA or Altium on more recent computers.

All technical indications (such as layer numbers in the Eagle tool) therefore refer to these tools (see Technical References below). To simplify use of this documentation, a zip file containing all developments is available for download.

Finally, the development of a professional version would require the use of SMD components, a minimum EMC and countries compliance study, integration into a great casing, cost analysis, user manual, and targeting of customers for a possible market launch. This is not the purpose of this documentation. This development should be considered as a simple prototype for personal usage.

3.2 Technical References

In order to reduce development costs and simplify board assembly, the entire board is mounted using radial components, and the design was conceived on an available professional version 6.6.0 of Eagle, MPLAB 8.92 and POV-Ray 3.7 running Mac OS High Sierra on a MacBook Pro in early 2011, 2.7 GHz Intel Core I7 with 16 GB DDR3 1333 MHz memory. This computer runs Mac OS and Windows on Parallel desktop V16.5.1. Development environment needs tools on both operating system, Mac OS and Windows. Pending the development of a bootloader for the master and slave boards, the firmware is updated using a PICkit™ 2 programmer/debugger (PG164120).

Eagle 6.6.0

EAGLE contains a schematic editor, for designing circuit diagrams. Schematics are stored in files with `.SCH` extension, parts are defined in device libraries with `.LBR` extension. Parts can be placed on many sheets and connected together through ports.

The PCB layout editor stores board files with the extension `.BRD`. It allows back-annotation to the schematic and auto-routing to automatically connect traces based on the connections defined in the schematic.

EAGLE saves Gerber and PostScript layout files as well as Excellon and Sieb & Meyer drill files. These are standard file formats accepted by PCB fabrication companies, but given EAGLE's typical user base of small design firms and hobbyists, many PCB fabricators and assembly shops also accept EAGLE board files (with extension `.BRD`) directly to export optimized production files and pick-and-place data themselves.

EAGLE provides a multi-window graphical user interface and menu system for editing, project management and to customize the interface and design parameters. The system can be controlled via mouse, keyboard hotkeys or by entering specific commands at an embedded command line. Keyboard hotkeys can be user defined. Multiple repeating commands can be combined into script files (with file extension `.SCR`). It is also possible to explore design files utilizing an EAGLE-specific object-oriented programming language (with extension `.ULP`).
(source Wikipedia)

POV-Ray 3.7

POV-Ray (Persistence of Vision Raytracer), or POV, is a free raytracing software available on a wide variety of platforms (Windows, Mac OS, GNU/Linux, etc.). It was originally based on DKBTrace sources, and to a lesser extent on Polyray.

POV-Ray does not have an integrated graphical interface (3D modeler) like most current synthesis software, but uses scene description scripts, in which all objects, lights, etc. must be described.

Modelers dedicated solely to POV-Ray exist (KPovModeler, Moray, Yet another POV-Ray modeller...), while many others export to the POV-Ray file format. Its file format is ASCII, and the default file extension is "`.pov`".

This provides basic shapes (spheres, boxes, toroids, etc.) on which Boolean operations can be performed using CSG. It also makes it possible to create volumes or surfaces based on mathematical functions, such as isosurfaces.

```
Example: function {x*x - F/y*y + z*z} draws a kind of gravity well, with F representing
its force.
```

It is also possible to import objects from other software (such as 3D Studio Max, Poser, etc.), which will be rendered in POV-Ray as an assembly of triangles, as many software programs are compatible, but it is very difficult to export POV-Ray objects to other formats.

POV-Ray can also be used to create animations.
(source Wikipedia)

MPLAB 8.92

MPLAB is a proprietary freeware integrated development environment for the development of embedded applications on PIC microcontrollers, and is developed by Microchip Technology.

MPLAB X is the latest edition of MPLAB, and is developed on the NetBeans platform. MPLAB and MPLAB X support project management, code editing, debugging and programming of Microchip 8-bit PIC and AVR (including ATMEGA) microcontrollers, 16-bit PIC24 microcontrollers, as well as 32-bit SAM (ARM) and PIC32 (MIPS) microcontrollers.

MPLAB is designed to work with MPLAB-certified devices such as the MPLAB ICD 3 and MPLAB REAL ICE, for programming and debugging PIC microcontrollers using a personal computer. PICKit programmers are also supported by MPLAB.

MPLAB X supports automatic code generation with the MPLAB Code Configurator and the MPLAB Harmony Configurator plugins.
(source Wikipedia)

PICKIT 2

The PICkit™ 2 programmer/debugger (PG164120) is a low-cost development tool with a comprehensive interface for programming and debugging microchips or microcontrollers.

Its comprehensive Windows® programming interface supports basic families (PIC10F, PIC12F5xx, PIC16F5xx), mid-range families (PIC12F6xx, PIC16F), controller families **PIC18F**, PIC24, dsPIC30, dsPIC33, and PIC32, 8-bit, 16-bit and 32-bit, and a large number of EEPROM-type serial microchips.

With the powerful Integrated Development Environment (IDE), PICkit™ 2 lets you debug on-circuit directly, on most PIC® microcontrollers. On-circuit debugging runs, pauses, and executes the program

step by step, while the microcontroller is integrated into the application. When paused on a breakpoint, file registers can be examined and modified.
(source https://pickit2.software.informer.com)

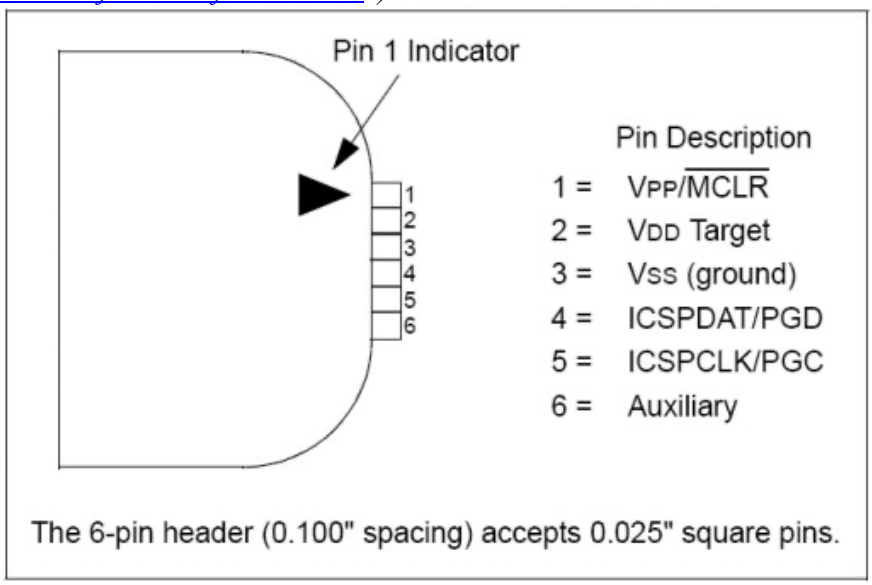


Figure 1 PICkit 2 Programmer Connector pinout

4 Electronic design

4.1 Global architecture

The complete system consists of a PC running, a rail or road network control application. This may be a fully developed application, or an existing one with a programmable interface driver. The interface between this PC and the control boards is via a 115200 Baud serial bus connected to a board with an RS232 serial interface. Only one of these boards, known as the "Master" board, has this interface.

The master board communicates with the other boards, known as slaves, via a 500K Baud CAN bus. The master board contains an RS232 transmitter (MAX232). By default, this board has address 31 (the address selector on this board must be removed, as well as resistors R23 and R24, giving it the value 31 by hardware setting). Address 31 is therefore forbidden on other boards, as it indicates to the PIC18F the presence of an RS232 transmitter.

If two cards have the same address, the overall behavior of the system may be inconsistent and render the entire system non-operational.

Each board is supplied with a power supply between 12 and 24V DC. The optimum supply voltage depends on the type of equipment operating on the network. The maximum theoretical voltage supported is 32V, but such a voltage could destroy the equipment connected to the board. In practice, we advise you not to exceed a supply voltage of 20V, especially if you use DCC environment.

Each board can drive 4 PWM or DCC blocks and 6 low-power modules. The boards also feature 8 TTL-level inputs/outputs (not to be used for direct control of a power module).

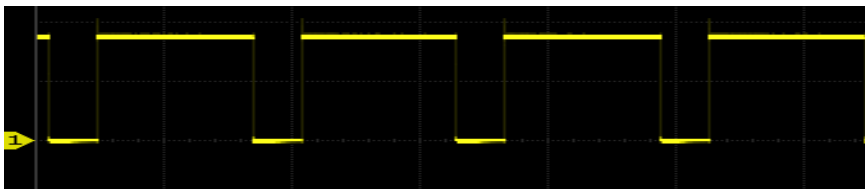


Figure 2 PWM Mode

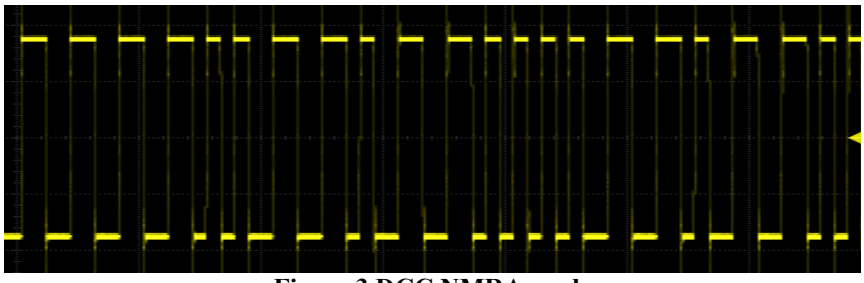


Figure 3 DCC NMRA mode

The system is controlled using a command language sent to the master board via the RS232 serial link. This language has two modes: a programming mode and a control mode.

- In programming mode, it is possible to specify the direction of each board's GPIOS, and to trigger automatic actions on events, e.g. "on detection of a vehicle on block 3 of board 8, feed block 6 of board 5 in the forward direction at speed 10" (speeds are between 0 and 15).

- In control mode, commands can be given, e.g. "feed block 3 of board 2 in the reverse direction at speed 2".

Each event (detection or loss of presence of a vehicle or change of status on a GPIO or TIMER triggered) generates an event which is sent to the other boards on CAN bus.

The commands and language are described in the following chapters.

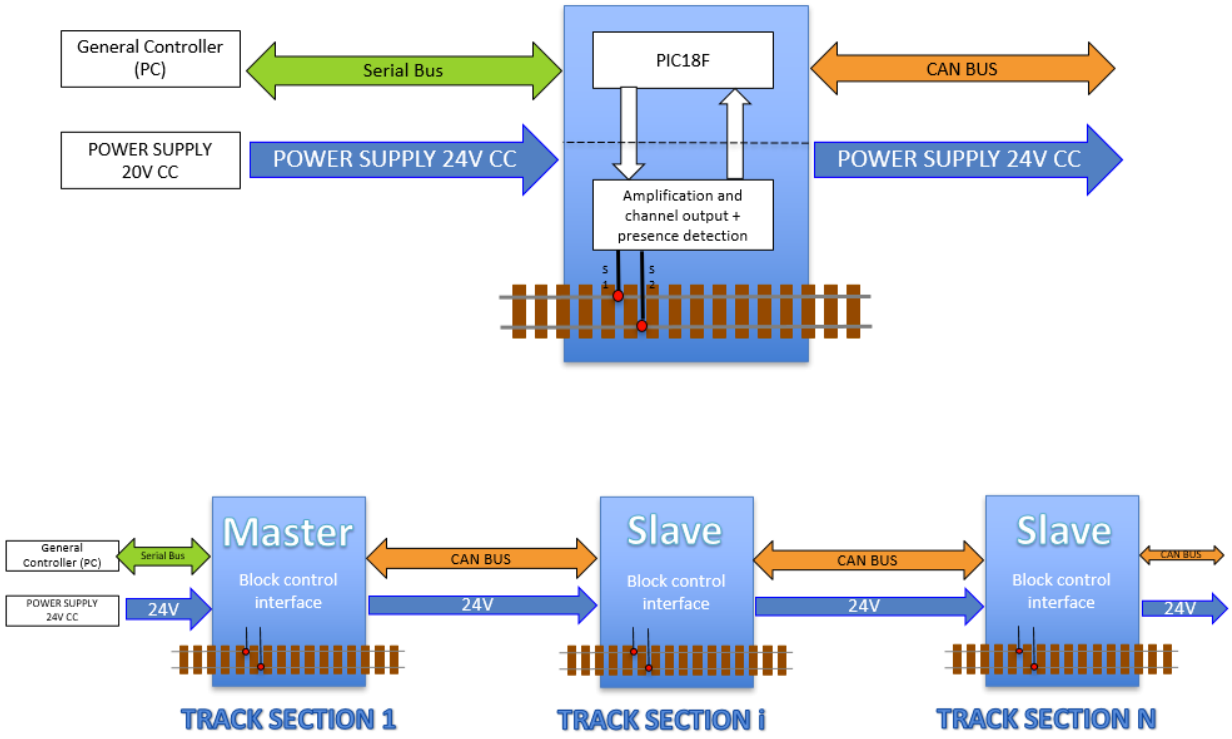


Figure 4 Master/slave board operating diagram

4.2 Data transfer between boards

Data transfer between boards is via a 500 KBaud CAN bus. The boards are connected to the CAN bus.

The first and last boards in this network have a switch for connecting a terminating resistor.

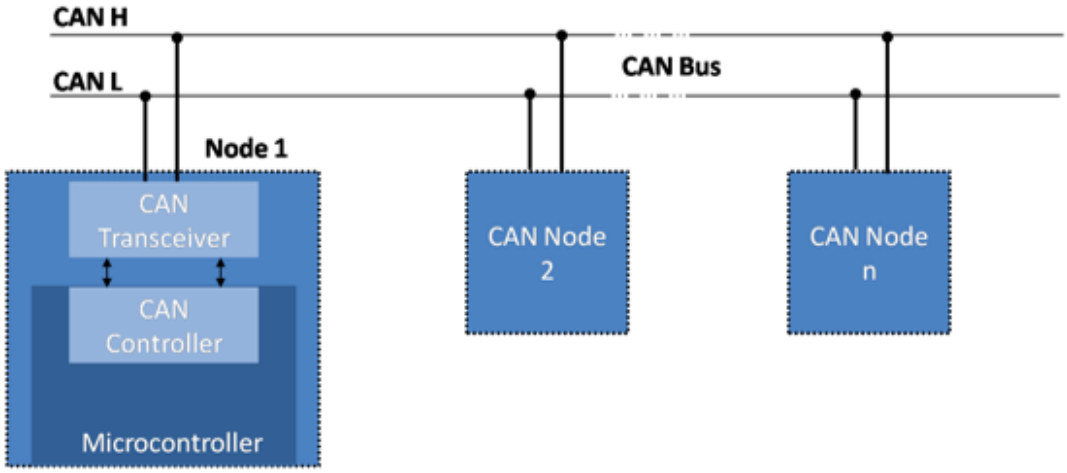


Figure 5 CAN bus wiring diagram

A transceiver MCP2551 is used to link the PIC18F to the CAN bus.

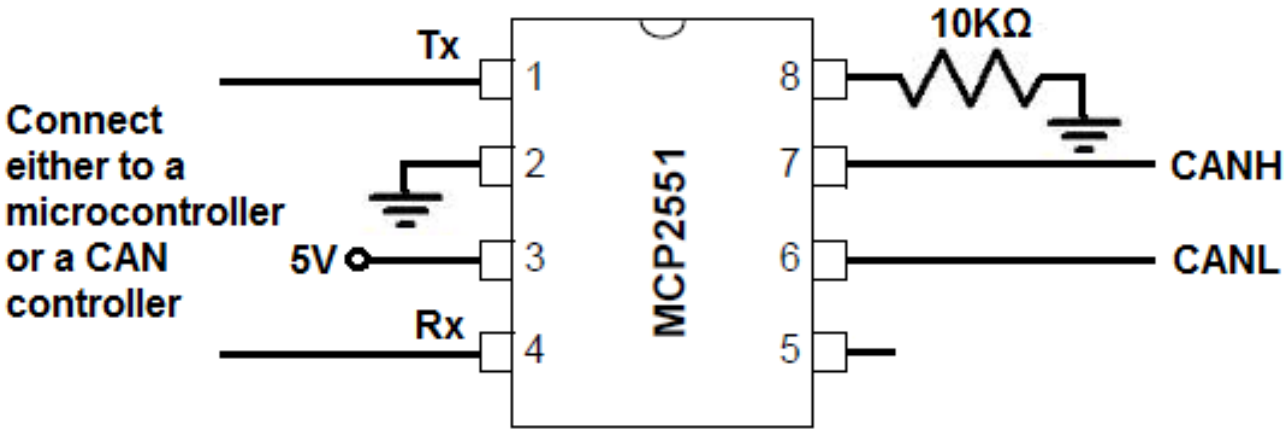


Figure 6 Transceiver MCP2551 wiring diagram

4.3 Data transfer between master board and PC

Data transfer between the master board and the PC is via an RS232 serial bus at 115200 baud. The link to the PC and the PIC18F is done via a MAX232 transmitter, which can be connected to an FTDI RS232/USB cable if the user so wishes.

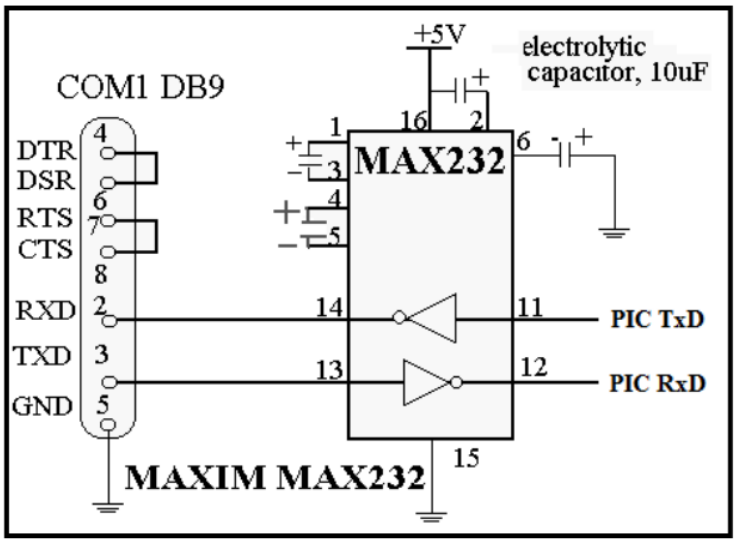


Figure 7 (Proposed by Kyle Thomson Revised 8/18/2009)

4.4 Schematic

The schematic consists of two parts:

- The first contains the 5V power supply (Note the STTH5L06RL inversion protection diode on this power supply). The L7805 could be replaced by a DollaTek 5V 1A regulated board, more robust. (https://www.amazon.fr/DollaTek-r%C3%A9gulateur-bornes-lentr%C3%A9e-LM7805/dp/B081JMJZG6)

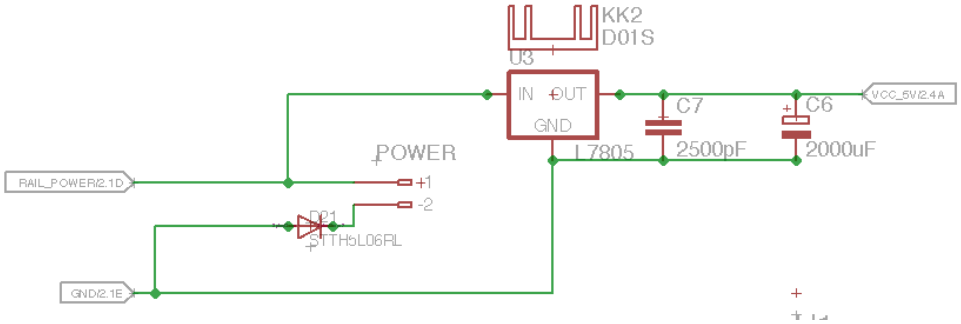


Figure 8 5V power supply with standard 7805

- The RS232 and CAN bus transmitters and transceivers, the programming connector for the PIC18F and the board address selector.

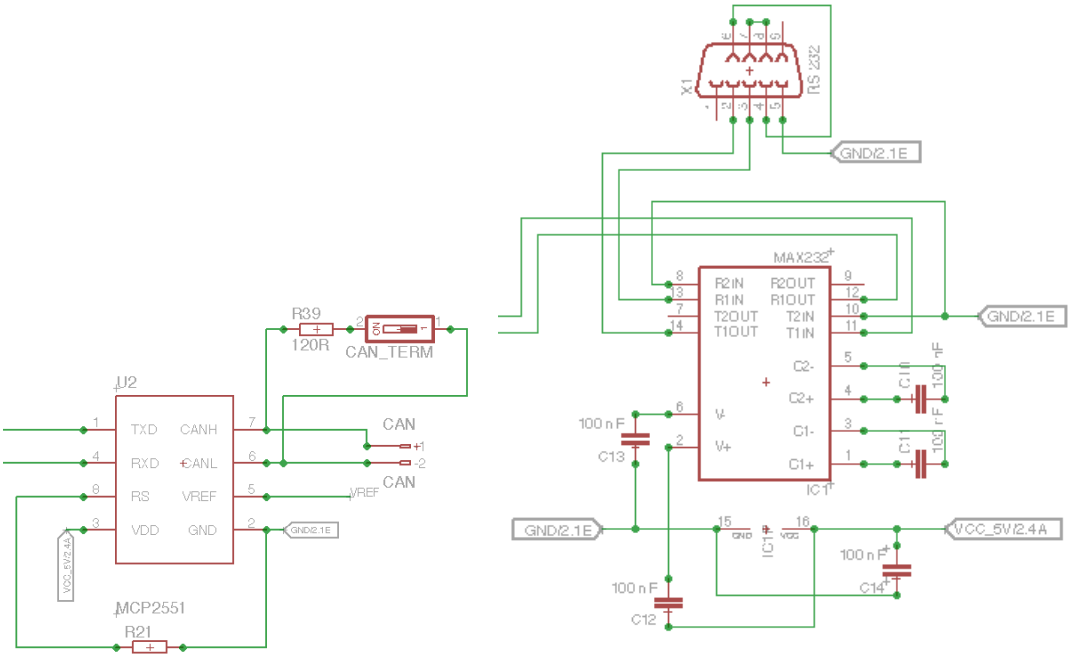


Figure 9 CAN and RS232 interface

- This assembly enables the master and slave boards to be wired on the same design. For the master board, solder the RS232 DB9 connector, the MAX232 transmitter and remove resistors R23 and R24 as well as the address selector. For the slave board, do not solder the RS232 DB9 connector, nor the MAX232 transmitter but keep C14.

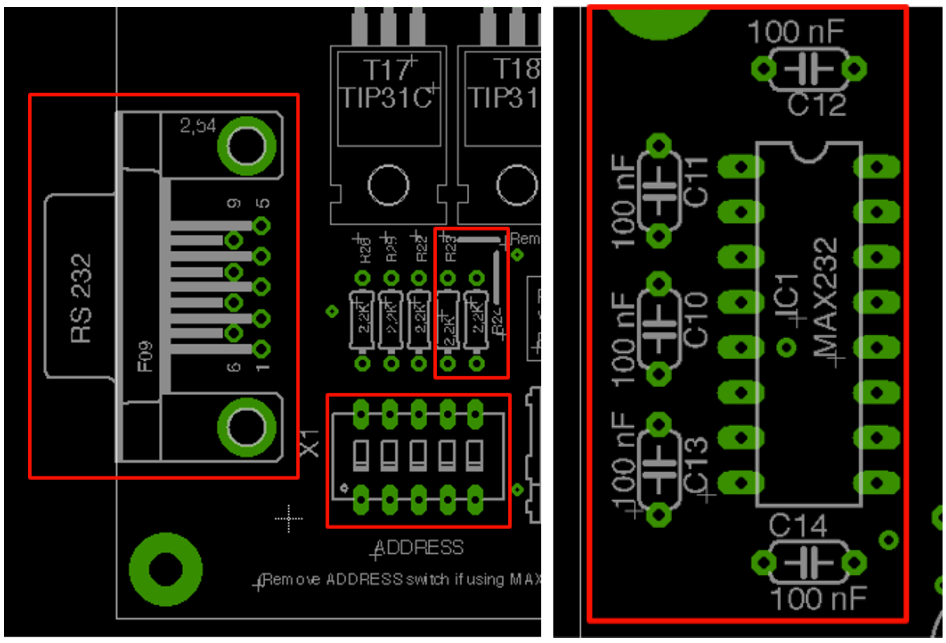


Figure 10 Master or Slave board

- The second contains the L298 power drivers and the 6 TIP31C transistors in Darlington circuit controlled by the PIC18F, as well as the power outputs to the rail or road network.

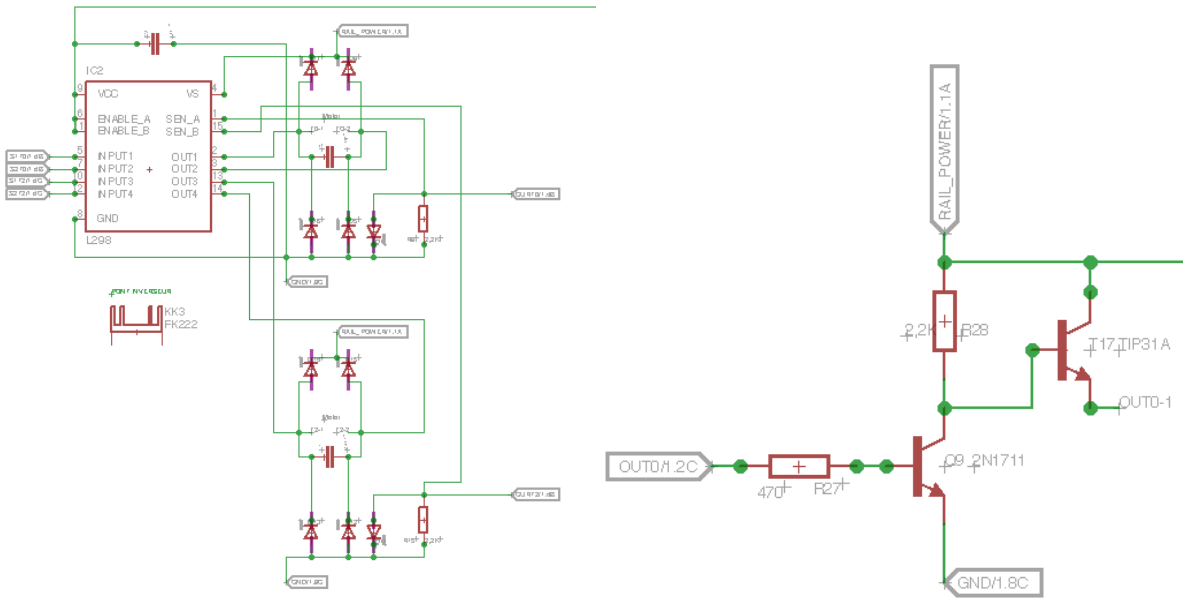


Figure 11Power boost

- Note that it is essential to place decoupling capacitors as close as possible to the components on the 5V supply, otherwise there is a risk of catastrophic loss of control due to PIC18F shutdown. These decoupling capacitors are 100nF on the 5V supply and 2500pF on the power outputs of the tracks to be supplied.

- Vehicle presence is detected by measuring the voltage across a diode at the output of the L298. When no load is present, a voltage close to 0V is measured at the terminal of a 2K2 resistor connected to ground, whereas when a load is present, the connection to ground is made via the 1N4004 diode, whose 0.7V bias voltage is then detected by the PIC18F.

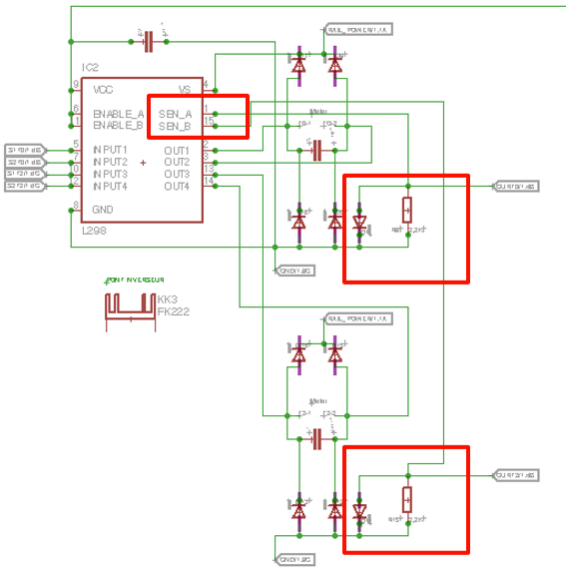
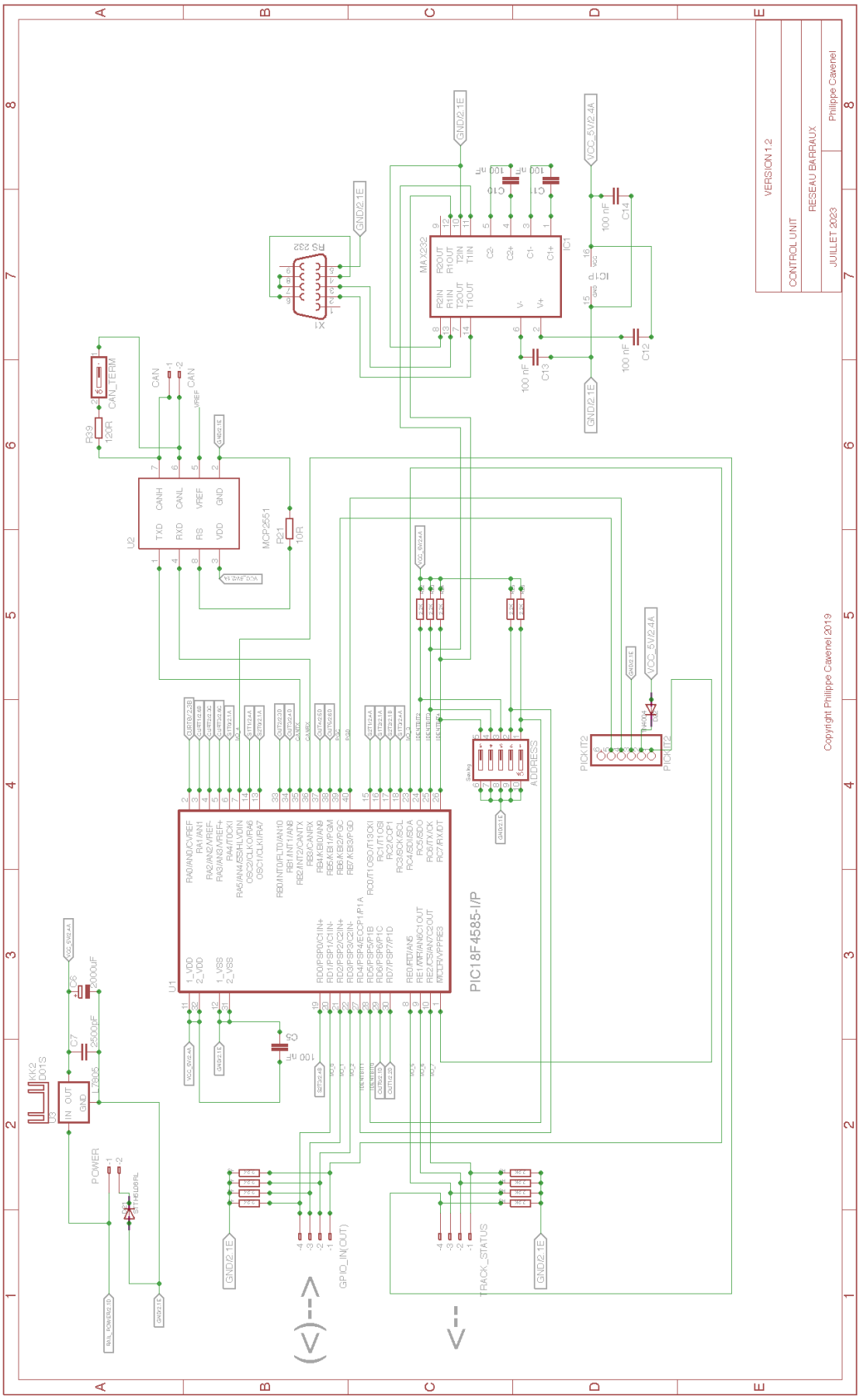
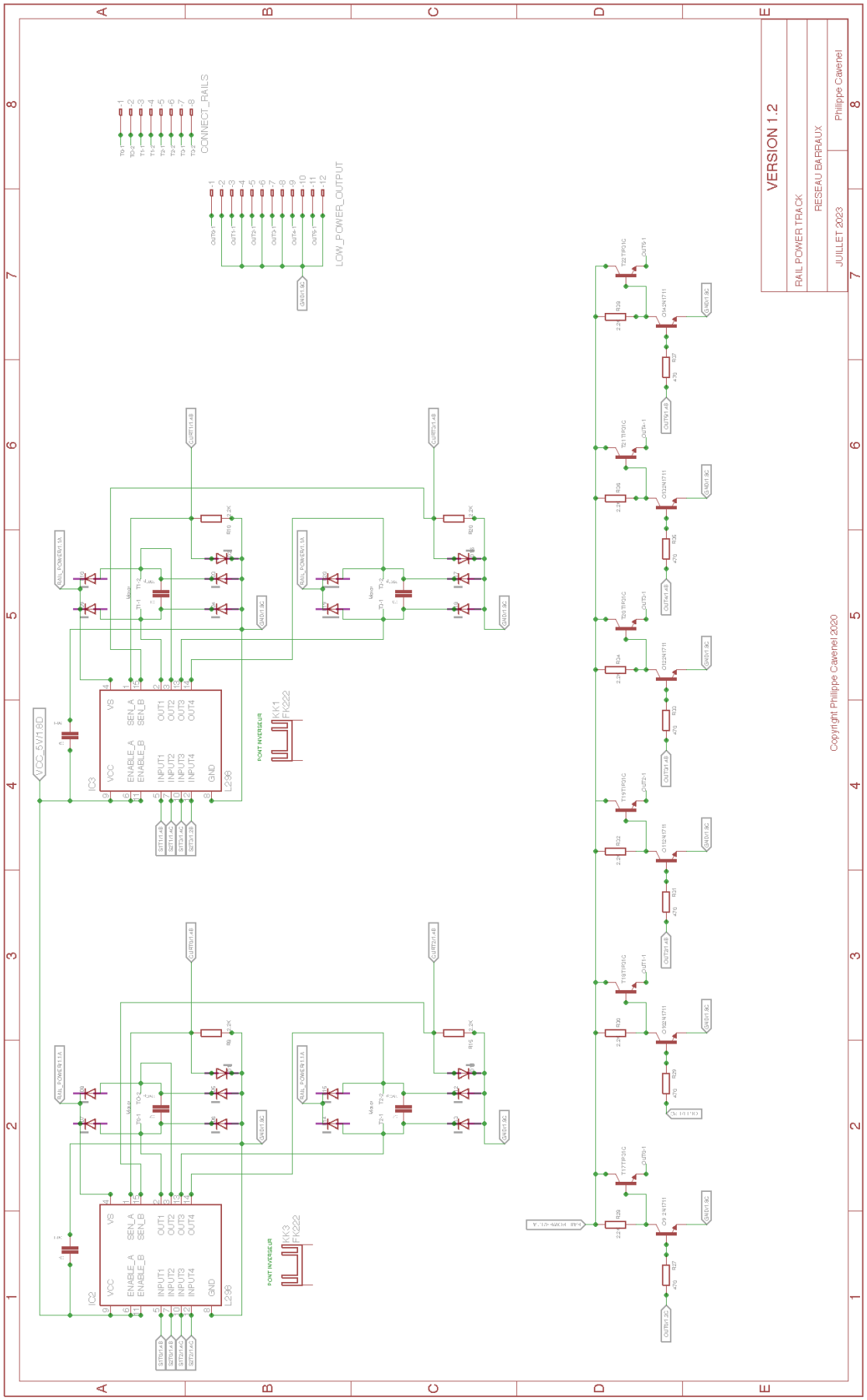


Figure 12 Vehicle Presence Detection


VERSION 1.2
CONTROL UNIT
RESEAU BARRAUX
JUILLET 2023
Philippe Cavenel
Copyright Philippe Cavenel 2019
PIC18F4585-I/P
MAX232
MCP2551
CAN_TERM
POWER
ADDRESS
PICKIT2
TRACK_STATUS
GPIO_IN(OUT)


VERSION 1.2
RAIL POWER TRACK
RESEAU BARRAUX
JUILLET 2023
Philippe Cavenel
CONNECT_RAILS
LOW_POWER_OUTPUT
POINT INVERSEUR
Copyright Philippe Cavenel 2020

4.5 Pin assignment

PIN	SIGNAL NAME	SIGNAL USED	SCHEMATIC	DIR	FONCTION
1	MCLR/Vpp/RE3	MCLR/Vpp	MCLR/Vpp	IN	PIC programming via PICkit2
2	RA0/AN0/Cvref	AN0	CURT0	IN	Track voltage measurement 0
3	RA1/AN1	AN1	CURT1	IN	Track voltage measurement1
4	RA2/AN2/Vref-	AN2	CURT2	IN	Track voltage measurement2
5	RA3/AN3/Vref+	AN3	CURT3	IN	Track voltage measurement
6	RA4/T0CKI	RA4	S1T0	OUT	S1 Track 0
7	RA5/AN4/SS/HLVDIN	AN4	I/O_4	IN	KNOB 0 analog value
8	RE0/RD/AN5	AN5	I/O_5	IN	KNOB 1 analog value
9	RE1/WR/AN6/C1OUT	RE1	I/O_6	OUT	TM1637 SCK I2C bus (mode)
10	RE2/CS/AN7/C2OUT	RE2	I/O_7	OUT	TM1637 SDA I2C bus (mode)
11	Vdd	Vdd	VCC_5V	IN	+5V
12	Vss	Vss	GND	IN	GND
13	OSC1/CLKI/RA7	RA7	S2T0	OUT	S2 Track 0
14	OSC2/CLKO/RA6	RA6	S1T1	OUT	S1 Track 1
15	RC0/T1OSO/T13CKI	RC0	S2T1	OUT	S2 Track 1
16	RC1/T1OSI	RC1	S1T2	OUT	S1 Track 2
17	RC2/CCP1	RC2	S2T2	OUT	S2 Track 2
18	RC3/SCK/SCL	RC3	S1T3	OUT	S1 Track 3
19	RD0/PSP0/C1IN+	RD0	S2T3	OUT	S2 Track 3
20	RD1/PSP1/C1IN-	RD1	I/O_0	IN	Global Purpose Input 0

PIN	SIGNAL NAME	SIGNAL USED	SCHEMATIC	DIR	FONCTION
21	RD2/PSP2/C2IN+	RD2	I/O_1	IN	Global Purpose Input 1
22	RD3/PSP3/C2IN-	RD3	I/O_2	IN	Global Purpose Input 2
23	RC4/SDI/SDA	RC4	I/O_3	IN	Global Purpose Input 3
24	RC5/SDO	RC5	IDENTBIT2	IN	BIT 2 card identifier
25	RC6/TX/CK	RC6	IDENTBIT3	IN	BIT 3 card identifier or TX line if MAX232 installed
26	RC7/RX/DT	RC7	IDENTBIT4	IN	BIT 4 card identifier or RX line if MAX232 installed
27	RD4/PSP4/ECCP1/P1A	RD4	IDENTBIT1	IN	BIT 1 card identifier
28	RD5/PSP5/P1B	RD5	IDENTBIT0	IN	BIT 0 card identifier
29	RD6/PSP6/P1C	RD6	OUT0	OUT	Low-power control (points, low beam, lighting, level crossings, various motors)
30	RD7/PSP7/P1D	RD7	OUT1	OUT	Low-power control (points, low beam, lighting, level crossings, various motors)
31	Vss	Vss	VCC_5V	IN	+5V
32	Vdd	Vdd	GND	IN	GND
33	RB0/INT0/FLT0/AN10	RB0	OUT2	OUT	Low-power control (points, low beam, lighting, level crossings, various motors)
34	RB1/INT1/AN8	RB1	OUT3	OUT	Low-power control (points, low beam, lighting, level crossings, various motors)
35	RB2/INT2/CANTX	CANTX	CANTX	IN/OUT	Bus CAN Transmission
36	RB3/CANRX	CANRX	CANRX	IN/OUT	Bus CAN Reception
37	RB4/KBI0/AN9	RB4	OUT4	OUT	Low-power control (points, low beam, lighting, level crossings, various motors)
38	RB5/KBI1/PGM	RB5	OUT5	OUT	Low-power control (points, low beam, lighting, level crossings, various motors)
39	RB6/KBI2/PGC	PGC	PGC	IN/OUT	PIC programming via PICkit2
40	RB7/KBI3/PGD	PGD	PGD	IN/OUT	PIC programming via PICkit2

4.6 Place and route

Components are placed as close as possible to each other to limit the distance between related components, especially for decoupling capabilities. Routing is performed automatically.

Top view

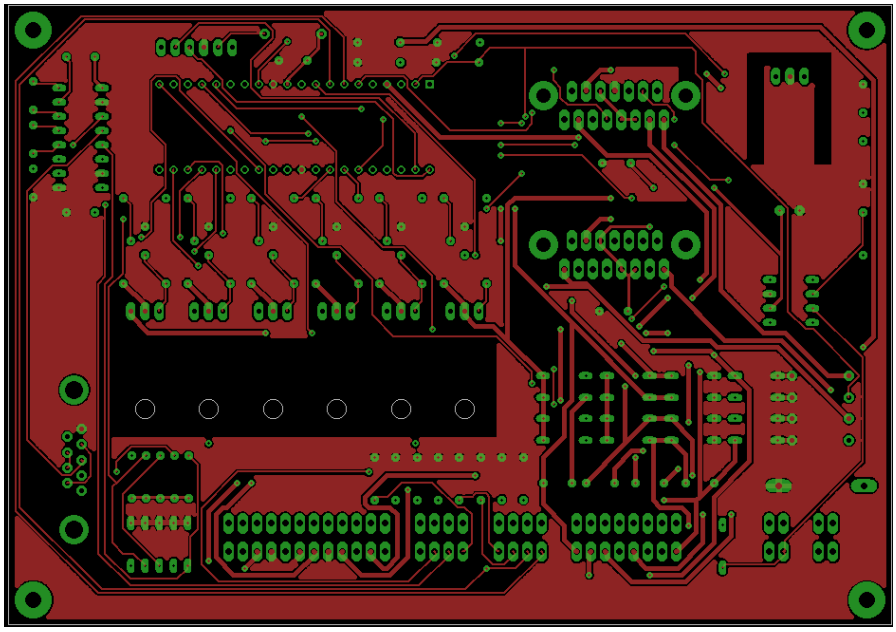


Figure 13 Top, Pads, Vias

Bottom view

Figure 14 Bottom, Pads, Vias

Components view

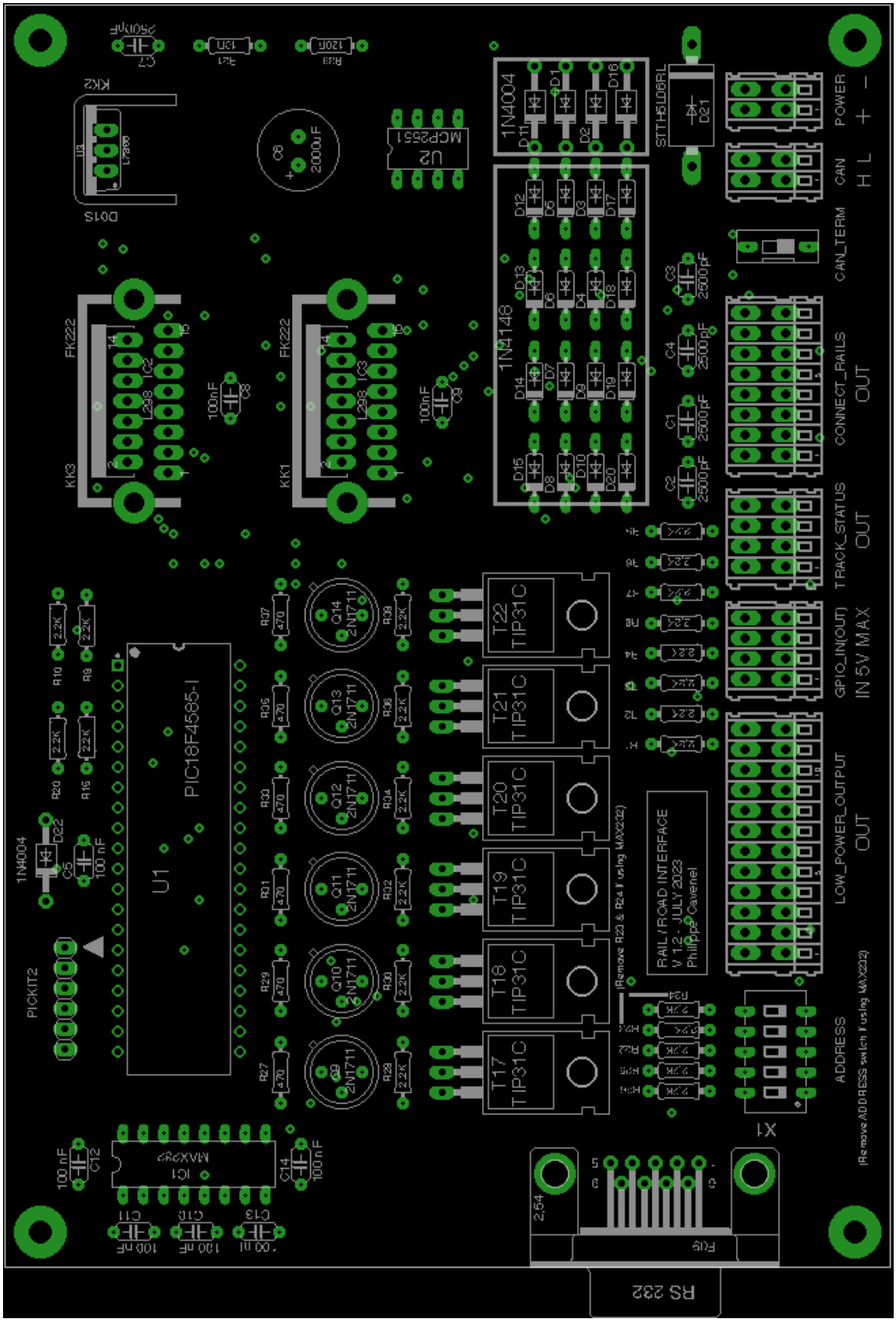


Figure 15 Pads, Vias, tPlace, tValues

3D view

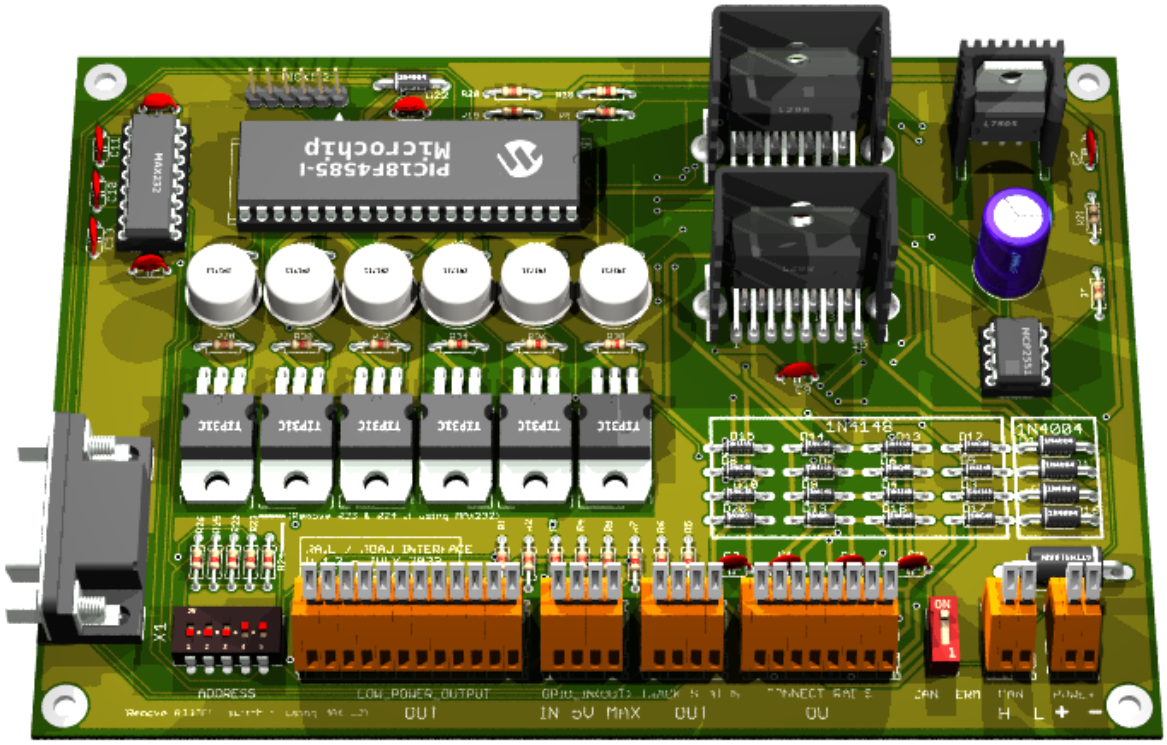


Figure 16 Top View (800 x 600)

Figure 17 Bottom View (800 x 600)

4.7 BOM

Qty	VALUE	DESCRIPTION	Farnell
1	Standard D-Sub connectors AMPL PLUG HD20, R/A 9P, B/L,4-40 INS	X1	2857982
8	100nF	C5, C8, C9, C10, C11, C12, C13, C14	2309064
1	MAX 232N	IC1	3121260
1	2000uF	C6	2766923
1	10R	R21	2329993
1	120R	R39	2329862
16	1N4148	D3, D4, D5, D6, D7, D8, D9, D10, D12, D13, D14, D15, D17, D18, D19, D20	9843680
5	1N4004	D22, D1, D11, D2, D16	1843708
1	STTH5L06RL	D21	2353682
23	2.2K	R1, R2, R3, R4, R5, R6, R7, R8, R9, R10, R15, R20, R22, R23, R24, R25, R26, R28, R30, R32, R34, R36, R38	9341536
5	2500pF	C1, C2, C3, C4, C7	2860175
6	2N1711	Q9, Q10, Q11, Q12, Q13, Q14	1611558
6	470	R27, R29, R31, R33, R35, R37	3496822
1	DIP / SIP Switch, 5 Circuit(s), Slide, Through Hole, SPST, 24 VDC, 100 mA	ADDRESS	3397711
1	Terminal block x2	CAN	1777096
1	Terminal block x2	POWER	1777096
1	DIP / SIP switch, 1 circuit	CAN TERMINATOR	1960919
1	Terminal block x12	LOW_POWER_OUTPUT	1777102
1	Terminal block x8	CONNECT_RAILS	1777101
2	Terminal block x4	TRACK_STATUS, GPIO_IN(OUT)	1777098
1	L7805 or DollaTek 5V 1A	U3	1467758
1	MCP2551P	U2	1439745
1	PIC18F4585-I (or PIC18F4680)	U1	1439547
1	PICKIT2	PICKIT2	1187827
6	TIP31C	T17, T18, T19, T20, T21, T22	9804145
2	L298N	IC2, IC3	403295
2	FK222	KK1, KK3	4621281

1	274-1AB	KK2	1611445

4.8 Generate the manufacturing files

First step: ERC on Eagle

- Generate schematic and validate circuit with ERC
- Pay particular attention to net class, especially for the power section (larger surface area required)

Step two: DRC and silkscreen on Eagle

- Component placement and automatic routing.
- To fill the top and bottom polygons, draw a polygon around the map, selecting the right layer (Top or Bottom) in the top left-hand corner after routing (otherwise automatic routing won't work), then restart routing.
- Validate with a DRC, selecting only the Top, Bottom, Via and Pad layers.
- Finalize silkscreen (layer 21) by placing all indications correctly, then generate drill legend on layer 144 by launching the ULP icon in the layout screen of Eagle on `drillegend-stack.ulp`

Step three: CAM processor on Eagle

- Generate the files for CAM Processor production by clicking on the `CAM_JOB/RailDriverCamJob.cam` file.
- In the top-left File menu, select the `.brd` file for the corresponding design.
- The directories and file names must be correct in the various tabs (check and correct if necessary).
- All that's left to do is run PROCESS JOB to produce the files.

Fourth stage: Final rendering

- To modify the selection of unknown boxes, modify the `3D_RENDERING/eagle3d/ulp/3dusrpac.dat` file.
- Update the configuration files directory in `3dconf.dat`
- Generate a 3D rendering of the design by launching the ULP icon in the layout screen of Eagle on `3D_RENDERING/eagle3d/ulp/3d50.ulp`
- Specify the `.brd` file, which will generate a `.pov` file that can be processed by POV-Ray to obtain the 3D rendering.
- Select "User-defined model" in the General tab
- Add layer 25 in Miscellaneous Case Design
- Modify Writing on plate to display layers 21 and 25 only.
- Add layers 25 and 27 in Case Reference
- Click on Create POVRay file and Exit.
- Copy the generated `EAGLE_FILES/RailDriver/RailDriver.pov` file into `3D_RENDERING/eagle3d/povray`
- Repeat the same operation, moving the camera to shoot from below (using Y) and/or from the other sides (using X and Z).

- Click on the `3D_RENDERING/eagle3d/povray/RailDriver.pov` file to start building the 3D view with POV-Ray (automatic launch on Windows). Use 800 x 600 AA 0.3 for a reasonable image creation time, or 1600 x 1200 A 0.3 for a better resolution, but the image creation time will take several tens of minutes (On old PC).
- In Eagle, make a screen copy of the various layers built during the construction of the manufacturing files. Place all views in a `PNGVIEW` directory

How to add a 3d model

- Retrieve the model in STL format from the net (snapeda for example on https://www.snapeda.com/). If the model is in STEP format, it must first be converted to STL, you can do that for example at on https://polyd.com/fr/convertir-step-en-stl-en-ligne
- Use the **stl2pov.exe** utility (launch a Windows shell)

```
stl2pov.exe myModel.stl > myModel.inc.
```

- Copy it to `3D_RENDERING/eagle3d/povray`. For example:

```
stl2pov.exe "TO220HeatSink\TO220_Heatsink.stp">"TO220HeatSink\TO220HeatSink.inc"
```

- Modify the `myModel.inc` file so that it is understood by POV-Ray, taking as an example `fk_222_sa.inc`

- Add the following include to the `3D_RENDERING/eagle3d/povray/e3d_user.inc` file:

```
#include "myModel.inc".
```

- Add the following information to the `3D_RENDERING/eagle3d/ulp/3dusrpac.dat` file:

```
MYMODEL:0:1:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0
:0:0:0:0:0:0:0:0:0:MYMODEL_GRND
0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0
:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:0:
```

[00] Eagle component package name
[01] Output name
[02] Output value
[03] Define color bands
[04] SMD offset (parts will be moved pcb_cuhight up/down)
[05] LED options (LED options dialog box will be displayed)
[06] Ready for sockets (see explanation)
[07] Quartz height request
[08] A part of a macro (e.g. SMD jumpers)
[09] SMD resistor, generates a combination of numbers
[10] Macro socket
[11] Socket height in 1/10mm
[14] Y-axis rotation correction
[15] Offset correction x
[16] Offset correction y
[17] Offset z correction
[18] Use Prefix from Part?
[19] Shunt on pin header (a dialog box will be displayed)
[20] Logo selection dialog box is displayed
[21] Reserved
[29] Minimum encircling box
[30] Maximum bounding box
[31] POV-Ray macro (Name of pov macro and left parenthesis)
[32] Package comments (German)
[33] Package comments

[12] Comments on socket
[13] Internal for administration (not currently used)

Fifth stage: BOM

- Update the BOM to purchase components on Farnell or Mouser or other dealers on the net.

Sixth step: send files for manufacturing

- Compress `CAM_JOB` and `PNGVIEW` directories to start manufacturing on a PCB manufacturing site such as AllPcb: https://www.allpcb.com/ or other manufacturer on the net.

5 Firmware design

5.1 Set PIC18F frequency to 32MHz

```
    OSCCON = 0x70;          no pre-divider => 8MHz // comment to set 8MHz
    OSCTUNE = 0x40;         activate PLL *4 => 32MHz // comment to set 8MHz
```

5.2 CAN bus control library

Introduction

This document describes programming interface to ECAN (Enhanced Controller Area Network) module. At the time of writing this document, only PIC18F family of microcontroller contained ECAN module. This module provides access to ECAN module in polling fashion. This module is completely written in 'C' language. It provides many customization options that may result in significant code reduction. To utilize this module, one must understand all options offered by ECAN module. This module is also available in Microchip Application Note AN878.

(source ECANPoll.ReadMe file)

Module Features

- Out-of-box support for Microchip C18 and HI-TECH PICC-18TM C compilers
- Offers simple abstract interface to ECAN module for most applications
- Additional functions/macros are available for advanced applications
- Supports all three functional modes
- Provides access to all ECAN features in Polling mode
- Easily modifiable to Interrupt-driven mode
- Operates in two main modes:
- Run-time Library Mode and Fixed Library Mode
- Various compile-time options to customization routines to a specific application
- Also available as Microchip Application Note AN878

(source ECANPoll.ReadMe file)

List of Component Modules

`ECAN.ex.txt`	This is main test file developed to demonstrate use of the library functions.
`ECANPoll.c`	This is ECAN code implementation file. One needs to include this file in their project.
`ECANPoll.h`	This file contains prototypes of functions and macros. One needs to include this file in every source file where ECAN functions will be called.
`ECANPoll.def`	This file contains all compile-time options for ECAN module. If you are using Maestro, this file will be created as per your option selections. This file is automatically included by `ECANPoll.h` file.

Functions

Refer to the `ECANPoll.ReadMe` file

CAN Initialization

In order to improve performance, data reception is interrupted.

```
// ECAN
    TRISBbits.TRISB3 = 1; // CANRX input setting
        gl_outputCANbufferCounter=0;
        for(bufferNumber=0;bufferNumber<MAXINPUTCANBUFFER;bufferNumber++) {
                gl_inputCANWriteBufferPointer[bufferNumber]=0;
                gl_inputCANReadBufferPointer[bufferNumber]=0;
                gl_inputCANmode[bufferNumber]=CAN_FREE;
        }
    ECANInitialize(); // init ECAN
        PIE3bits.RXB0IE=1; // enable interrupt for CAN
        PIE3bits.RXB1IE=1; // enable interrupt for CAN
```

CAN Reception

Can reception is shared with RS232 reception:

```
/* ============================================================================
 * Function: high_isr
 * Returns: void = ISR, no return.
 * Description: Hight priority interrupt service routine: receives CAN messages,
 * sync handling, buffers data, and UART RX.
 * ============================================================================ */
void high_isr(void){

        BYTE dataLen;                           // Number of bytes transmitted in the message
        BYTE dataCounter;
        ECAN_RX_MSG_FLAGS flags;  // Flags
        unsigned long    id;                    // Id of sender
        char     dataBuffered;
        char     bufferNumber;

        dataBuffered=FALSE;

        if(PIR3bits.RXB0IF ||PIR3bits.RXB1IF) {
                while(ECANReceiveMessage((unsigned long *)&id,(BYTE *)&gl_data[0], (BYTE *)
&dataLen,(ECAN_RX_MSG_FLAGS *) &flags));

           // BOARD SYNC
        if (id == SYNC_ID) {
                INTCONbits.TMR0IF       = 0;   // T0 int flag bit cleared before starting
                        TMR0H                                   = 0;   // re-phase Timer0
                        TMR0L                                   = 0;   // re-phase Timer0
                gl_speedCounter           = 1;   // Same value on each board
                        gl_syncRequested          = 1;   // For printing SYNC message on LED
                }
                // OTHER TRAMES
                else {
                        for(bufferNumber=0;bufferNumber<MAXINPUTCANBUFFER;bufferNumber++) {
                                if ((unsigned long)gl_currentCANid[bufferNumber]==(unsigned long)id &&
(char)gl_inputCANmode[bufferNumber]!=(char)CAN_FREE) {
                                        for(dataCounter=0;dataCounter<dataLen;dataCounter++) {

        gl_inputCANbuffer[bufferNumber][gl_inputCANWriteBufferPointer[bufferNumber]]=gl_data[dataCounter];
                                                gl_inputCANWriteBufferPointer[bufferNumber]++;

        if(gl_inputCANWriteBufferPointer[bufferNumber]>=MAXTRAMESIZE)gl_inputCANWriteBufferPointer[bufferNumber]-
=MAXTRAMESIZE;
                                        }
                                        dataBuffered=TRUE;
                                        break;
                                }
                        }

                        if ((char)dataBuffered==(char)FALSE) {
                                for(bufferNumber=0;bufferNumber<MAXINPUTCANBUFFER;bufferNumber++) {
                                        if (gl_inputCANmode[bufferNumber]==CAN_FREE) {
                                                for(dataCounter=0;dataCounter<dataLen;dataCounter++) {

        gl_inputCANbuffer[bufferNumber][gl_inputCANWriteBufferPointer[bufferNumber]]=gl_data[dataCounter];
```

```
                                                        gl_inputCANWriteBufferPointer[bufferNumber]++;

        if(gl_inputCANWriteBufferPointer[bufferNumber]>=MAXTRAMESIZE)gl_inputCANWriteBufferPointer[bufferNumber]-
=MAXTRAMESIZE;
                                                }
                                                gl_inputCANmode[bufferNumber]=CAN_GET_DATA;
                                                gl_currentCANid[bufferNumber]=id;
                                                dataBuffered=TRUE;
                                                break;
                                        }
                                }
                        }
                }
        }


        if ((char)gl_master==(char)FALSE)return;

    // Check if interrupt originates from USART reception
    if (PIR1bits.RCIF)
    {
        // Read received data
        gl_receivedUSARTData[gl_receivedUSARTPointer++] = RCREG;
                if (gl_receivedUSARTPointer>=USARTBUFFERSIZE)gl_receivedUSARTPointer=0;
    }
}
#pragma code
```

Can transmission is performed either by the user_putc() function when an ASCII message is to be sent to the master, or by a specific CAN routine to send compressed data.

```
/* ================================================================================
 * Function: CANsendDelay
 * Returns: void = no return.
 * Description: Small busy-wait delay between CAN frames.
 * ================================================================================ */
void CANsendDelay(void) {
        short   delay;
        for(delay=0;delay<WAITDELAYTRAMECAN;delay++);
}
/* ================================================================================
 * Function: sendPrintToCAN
 * Returns: void = no return.
 * Description: Packages the output buffer into CAN 'PRINT' frames with header/footer
 * and sends.
 * ================================================================================ */
void sendPrintToCAN(void){

        long    id;                     // Id of sender
    BYTE        dataOut[8];     // DATA to CAN
        char    dataCounter;
        char    dataOutCounter;
        char    trameComplete;

        BYTE dataLen;                           // Number of bytes transmitted in the gl_message
        ECAN_TX_MSG_FLAGS flags;  // Flags

        dataLen=8;
        flags=ECAN_TX_STD_FRAME;
        id=gl_boardNumber;

        // Watchdog off
        WDTCONbits.SWDTEN = 0;

        // header trame
        for(dataOutCounter=0;dataOutCounter<8;dataOutCounter++) dataOut[dataOutCounter]=TRAMEPRINTHEADER;

        while(!ECANSendMessage(id,dataOut,dataLen,flags));
        CANsendDelay();

        // trame
        trameComplete=FALSE;
        dataCounter=0;
        while(dataCounter<MAXMESSAGESIZE) {
                for(dataOutCounter=0;dataOutCounter<8;dataOutCounter++) {
                        if (dataCounter<gl_outputCANbufferCounter) {
                                dataOut[dataOutCounter]=gl_outputCANbuffer[dataCounter++];
                                if ((char)dataOut[dataOutCounter]==(char)ENDOFPRINTFTRAME) trameComplete=TRUE;
                        }
                        else {
                                dataOut[dataOutCounter]=ENDOFPRINTFTRAME;
                                if (dataCounter<MAXMESSAGESIZE)dataCounter++;
                                trameComplete=TRUE;
                        }
```

```
                }
                while(!ECANSendMessage(id,dataOut,dataLen,flags));
                CANsendDelay();

                if ((char)trameComplete==(char)TRUE) break;
        }
        gl_outputCANbufferCounter=0;

        // footer trame
        for(dataOutCounter=0;dataOutCounter<8;dataOutCounter++) dataOut[dataOutCounter]=TRAMEPRINTFOOTER;
        while(!ECANSendMessage(id,dataOut,dataLen,flags));
        CANsendDelay();

        // Watchdog on
        WDTCONbits.SWDTEN = 1;


}
/* ================================================================================
 * Function: _user_putc
 * Returns: int = the character written.
 * Description: Writes a character to UART on master or queues to CAN on slave;
 * triggers CAN send on buffer/full or terminator.
 * ================================================================================ */
int _user_putc (char c) {

        // On master board send to UART via standart putc()
        if ((char)gl_master==(char)TRUE){
                if (gl_inputCounter==0)sendUSART(c);
        }

        // Send on CAN bus
        else {
                if (gl_outputCANbufferCounter<MAXMESSAGESIZE)gl_outputCANbuffer[gl_outputCANbufferCounter++]=c;
                if ((gl_outputCANbufferCounter==MAXMESSAGESIZE || c==ENDOFPRINTFTRAME)) {
                        sendPrintToCAN();
                }
        }
        return(c);
}
/* ================================================================================
 * Function: sendRequestToCAN
 * Returns: void = no return.
 * Description: Compresses gl_request and transmits it as a CAN 'REQUEST' frame
 * sequence; restores request via RLE decode.
 * ================================================================================ */
void sendRequestToCAN(void) {

        long    id;                     // Id of sender
    BYTE        dataOut[8];     // DATA to CAN
        char    dataCounter;
        char    dataOutCounter;
        BYTE dataLen;                           // Number of bytes transmitted in the gl_message
        ECAN_TX_MSG_FLAGS flags;  // Flags
        char trameSize;

        // Convert request to dataOut using gl_dataStructure
        trameSize=compressData();

        dataLen=8;
        flags=ECAN_TX_STD_FRAME;
        dataCounter=0;
        id=gl_boardNumber;

        // header trame
        for(dataOutCounter=0;dataOutCounter<8;dataOutCounter++) {
                dataOut[dataOutCounter]=TRAMEREQUESTHEADER;
        }

        while(!ECANSendMessage(id,dataOut,dataLen,flags));
        CANsendDelay();

        // trame
        while(dataCounter<trameSize) {
                for(dataOutCounter=0;dataOutCounter<8;dataOutCounter++) {
                        if (dataCounter<trameSize) {
                                dataOut[dataOutCounter]=gl_request[dataCounter++];
                                if(dataCounter>=MAXTRAMESIZE) return; // Something wrong happened
                        }
                        else {
                                dataOut[dataOutCounter]=0;
                        }
                }
                while(!ECANSendMessage(id,dataOut,dataLen,flags));
                CANsendDelay();
        }
```

```
        // footer trame
        for(dataOutCounter=0;dataOutCounter<8;dataOutCounter++) {
                dataOut[dataOutCounter]=TRAMEREQUESTFOOTER;
        }
        while(!ECANSendMessage(id,dataOut,dataLen,flags));
        uncompressData(); // get back to initial data for other action in manageRequest()
}
/* ============================================================================
 * Function: CAN_SendSync
 * Returns: void = no return.
 * Description: Broadcasts a CAN SYNC frame and re-phases timers and counters
 * across boards.
 * ============================================================================ */
void CAN_SendSync(void){

        BYTE dataLen;                           // Number of bytes transmitted in the gl_message
        ECAN_TX_MSG_FLAGS flags;  // Flags
    BYTE        dataOut[8];      // DATA to CAN
        char    dataOutCounter;

        dataLen=8;
        flags=ECAN_TX_STD_FRAME;

        // sync trame
        for(dataOutCounter=0;dataOutCounter<8;dataOutCounter++) {
                dataOut[dataOutCounter]=TRAMESYNCTRACE;
        }

    while (!ECANSendMessage(SYNC_ID, dataOut, dataLen, flags));

    gl_mutexLowIsr = 1;
    INTCONbits.TMR0IE       = 0;   // Disable Timer0 Interrupt
    gl_speedCounter                     = 1;   // Same value on each board
        gl_syncRequested          = 1;   // For printing SYNC message on LED
    INTCONbits.TMR0IF       = 0;   // T0 int flag bit cleared before starting
        TMR0H                                   = 0;   // re-phase Timer0
        TMR0L                                   = 0;   // re-phase Timer0
    INTCONbits.TMR0IE       = 1;   // Enable Timer0 Interrupt
    T0CONbits.TMR0ON        = 1;   // timer0 START
    gl_mutexLowIsr = 0;
        CANsendDelay();
}
/* ============================================================================
 * Function: getInputRequestFromCAN
 * Returns: char = TRUE when a request was assembled, FALSE otherwise.
 * Description: Parses incoming CAN buffers, extracting REQUEST or PRINT trames;
 * fills gl_request or prints content.
 * ============================================================================ */
char getInputRequestFromCAN(void) {

        char    requestHeaderTrameDetected;
        char    printHeaderTrameDetected;
        char    requestFooterTrameDetected;
        char    printFooterTrameDetected;
        char    requestTrameEnd;
        char    printTrameEnd;

        char    dataInCounter;
        char    dataStructureCounter;

        char    bufferNumber;
        char    data;

        mySprintf((char *)gl_message,"");
        for(bufferNumber=0;bufferNumber<MAXINPUTCANBUFFER;bufferNumber++) {

                requestHeaderTrameDetected=0;
                printHeaderTrameDetected=0;
                requestFooterTrameDetected=0;
                printFooterTrameDetected=0;

                while (gl_inputCANReadBufferPointer[bufferNumber]!=gl_inputCANWriteBufferPointer[bufferNumber]) {
                        data=gl_inputCANbuffer[bufferNumber][gl_inputCANReadBufferPointer[bufferNumber]];

                                if ((char)data==(char)TRAMEREQUESTHEADER) requestHeaderTrameDetected++;      else
requestHeaderTrameDetected=0;
                                if ((char)data==(char)TRAMEREQUESTFOOTER) requestFooterTrameDetected++;      else
requestFooterTrameDetected=0;

                                if ((char)data==(char)TRAMEPRINTHEADER) printHeaderTrameDetected++;          else
printHeaderTrameDetected=0;
                                if ((char)data==(char)TRAMEPRINTFOOTER) printFooterTrameDetected++;          else
printFooterTrameDetected=0;

                                // REQUEST HEADER
```

```
                                if (requestHeaderTrameDetected==8) {
                                        gl_inputCANmode[bufferNumber]=CAN_REQUEST;
                                        requestHeaderTrameDetected=0;

        gl_requestInputCANtrameStart[bufferNumber]=gl_inputCANReadBufferPointer[bufferNumber]+1;
                                        if
((int)gl_requestInputCANtrameStart[bufferNumber]>=MAXTRAMESIZE)gl_requestInputCANtrameStart[bufferNumber]=0;
                                }
                                else if (printHeaderTrameDetected==8) {
                                        gl_inputCANmode[bufferNumber]=CAN_PRINT;
                                        printHeaderTrameDetected=0;

        gl_printInputCANtrameStart[bufferNumber]=gl_inputCANReadBufferPointer[bufferNumber]+1;
                                        if
((char)gl_printInputCANtrameStart[bufferNumber]>=(char)MAXTRAMESIZE)gl_printInputCANtrameStart[bufferNumber]=0;
                                }
                                else if (requestFooterTrameDetected==8 &&
gl_inputCANmode[bufferNumber]==CAN_REQUEST) {
                                        gl_inputCANmode[bufferNumber]=CAN_FREE;
                                        requestTrameEnd=gl_inputCANReadBufferPointer[bufferNumber]-7;
                                        if (requestTrameEnd<0)requestTrameEnd+=MAXTRAMESIZE;
                                        dataInCounter=gl_requestInputCANtrameStart[bufferNumber];
                                        dataStructureCounter=0;
                                        while(dataInCounter!=requestTrameEnd) {

        gl_request[dataStructureCounter++]=gl_inputCANbuffer[bufferNumber][dataInCounter++];
                                                if (dataInCounter>=MAXTRAMESIZE)dataInCounter=0;
                                                if (dataStructureCounter>=REQUESTSIZE){
                                                        return(FALSE);
                                                }
                                        }

                                        gl_inputCANReadBufferPointer[bufferNumber]++;
                                        if
(gl_inputCANReadBufferPointer[bufferNumber]>=MAXTRAMESIZE)gl_inputCANReadBufferPointer[bufferNumber]=0;
                                                return(uncompressData()); // Mean request available to proceed
                                }
                                else if (printFooterTrameDetected==8 && gl_inputCANmode[bufferNumber]==CAN_PRINT)
{
                                        gl_inputCANmode[bufferNumber]=CAN_FREE;
                                        printTrameEnd=gl_inputCANReadBufferPointer[bufferNumber]-7;
                                        if (printTrameEnd<0)printTrameEnd+=MAXTRAMESIZE;
                                        dataInCounter=gl_printInputCANtrameStart[bufferNumber];

                                        // Watchdog off
                                         WDTCONbits.SWDTEN = 0;

                                        while(dataInCounter!=printTrameEnd) {
                                                if ((char)gl_master==(char)TRUE &&
(char)gl_inputCANbuffer[bufferNumber][dataInCounter]!=(char)0)
_user_putc(gl_inputCANbuffer[bufferNumber][dataInCounter]);
                                                dataInCounter++;
                                                if (dataInCounter>=MAXTRAMESIZE)dataInCounter=0;
                                        }

                                        // Watchdog on
                                         WDTCONbits.SWDTEN = 1;

                                        gl_inputCANReadBufferPointer[bufferNumber]++;
                                        if
(gl_inputCANReadBufferPointer[bufferNumber]>=MAXTRAMESIZE)gl_inputCANReadBufferPointer[bufferNumber]=0;
                                        return(FALSE); // Mean no more data to print
                                }

                                // read new data
                                gl_inputCANReadBufferPointer[bufferNumber]++;
                                if
(gl_inputCANReadBufferPointer[bufferNumber]>=MAXTRAMESIZE)gl_inputCANReadBufferPointer[bufferNumber]=0;

                }
        }
        return(FALSE); // Nothing to do
}
```

5.3 PWM management in analog mode, DCC management in NMRA digital mode and TM1637 device management

PWM or DCC signal management is performed under interrupt, including TM1637 7 segment display management

When the bogie wheels bridge two track sections, the current briefly passes from one section to the other through the wheels. At that moment, if the H-bridges are not perfectly synchronized, this can cause a short transient short-circuit, which in the best case may create interference, and in the worst case a voltage drop that will reboot the board. It is therefore very important to properly synchronize the signal between the sections. This is done by updating the bridge states only after all track section speeds have been computed. The setting is applied in a single step via **setPort()**.

```
// Set Port Value when all tracks have been updated
                        if (gl_trackNumber==0) {
                                for(gl_trackNumber=0;gl_trackNumber<4;gl_trackNumber++) {
                                        if (gl_speed[gl_trackNumber]>=gl_speedCounter) {
                                                if (gl_direction[gl_trackNumber]==TRACK_FORWARD) {
                                                switch (gl_trackNumber) {
                                                        case 0:gl_S1T0char=1; gl_S2T0char=0;break;
                                                        case 1:gl_S1T1char=1; gl_S2T1char=0;break;
                                                        case 2:gl_S1T2char=1; gl_S2T2char=0;break;
                                                        case 3:gl_S1T3char=1; gl_S2T3char=0;break;
                                        }
                                        }
                                        if (gl_direction[gl_trackNumber]==TRACK_BACKWARD) {
                                        switch (gl_trackNumber) {
                                                case 0:gl_S1T0char=0; gl_S2T0char=1;break;
                                                case 1:gl_S1T1char=0; gl_S2T1char=1;break;
                                                case 2:gl_S1T2char=0; gl_S2T2char=1;break;
                                                case 3:gl_S1T3char=0; gl_S2T3char=1;break;
                                        }
                                        }
                                        if (gl_direction[gl_trackNumber]==TRACK_STOP) {
                                        switch (gl_trackNumber) {
                                                case 0:gl_S1T0char=0; gl_S2T0char=0;break;
                                                case 1:gl_S1T1char=0; gl_S2T1char=0;break;
                                                case 2:gl_S1T2char=0; gl_S2T2char=0;break;
                                                case 3:gl_S1T3char=0; gl_S2T3char=0;break;
                                        }
                                        }
                                        }
                                        else {
                                                switch (gl_trackNumber) {
                                        case 0:gl_S1T0char=0; gl_S2T0char=0;break;
                                        case 1:gl_S1T1char=0; gl_S2T1char=0;break;
                                        case 2:gl_S1T2char=0; gl_S2T2char=0;break;
                                        case 3:gl_S1T3char=0; gl_S2T3char=0;break;
                                        }
                                        }
                                }
                                gl_trackNumber=0;
                                setPort();
                        }
```

Here is the complete code related to the low-level interrupt handled by TMR0.

```
/* ============================================================================
 * Function: set7segmentPort
 * Returns: void = no return.
 * Description: Sets the TM1637 lines (CLK,DIO) on a shared port mapping.
 * ============================================================================ */
void set7segmentPort(char CLK, char DIO) {

        char myPortE; // used to better synchronised output updates
        char delay;

    myPortE= (CLK<<1) + (DIO<<2);
```

```
    LATE=myPortE;
}
/* ============================================================================
 * Function: twoWire_init
 * Returns: void = no return.
 * Description: Initializes pseudo two-wire interface for TM1637.
 * ============================================================================ */
void twoWire_init(void) {
        set7segmentPort(0,0);
}
/* ============================================================================
 * Function: twoWire_start
 * Returns: void = no return.
 * Description: Generates TM1637 start sequence.
 * ============================================================================ */
void twoWire_start(void){

        set7segmentPort(1,1);
        set7segmentPort(1,0);
}
/* ============================================================================
 * Function: twoWire_stop
 * Returns: void = no return.
 * Description: Generates TM1637 stop sequence.
 * ============================================================================ */
void twoWire_stop(void){
        set7segmentPort(0,0);
        set7segmentPort(1,0);
        set7segmentPort(1,1);
}
/* ============================================================================
 * Function: twoWire_ack
 * Returns: void = no return.
 * Description: Generates a TM1637 dummy ACK pulse.
 * ============================================================================ */
void twoWire_ack(void){

        set7segmentPort(0,0);
        set7segmentPort(1,0);
}
/* ============================================================================
 * Function: twoWire_write
 * Returns: char = unspecified / not used.
 * Description: Clocks out 8 bits LSB-first on TM1637 (bit-bang).
 * ============================================================================ */
char twoWire_write(char data){

        char tx;
        char DIO;
        for(tx = 0 ; tx < 8 ; tx++) {
                DIO = ((data >> tx) & 0x01) ? 1 : 0 ; //LSB first (Real 12c sends MSB first)
                set7segmentPort(0,DIO);
                set7segmentPort(1,DIO);
                set7segmentPort(0,DIO);
        }
}
/* ============================================================================
 * Function: TM1637_init
 * Returns: void = no return.
 * Description: Initializes the TM1637 display driver.
 * ============================================================================ */
void TM1637_init(void){
    twoWire_init();
}
/* ============================================================================
 * Function: TM1637_write
 * Returns: void = no return.
 * Description: Converts two 3-digit numbers to 6 digits and writes segment bytes.
 * ============================================================================ */
void TM1637_write(short number1,short number2){

    char str1Num[4];
    char str2Num[4];
    char strNum[8];
    char size;

        mySprintf(str1Num,"%3d",number1);
        mySprintf(str2Num,"%3d",number2); // 3 characters
        mySprintf(strNum,"%s%s",str2Num,str1Num);

    for(size=5;size>=0;size--) {
                if ((char)strNum[size]==(char)' ')twoWire_write(digits[11]);
                else if ((char)strNum[size]==(char)'-')twoWire_write(digits[10]);
        else {
                        char i = strNum[size] - '0';  //Get index 0 - 9
        twoWire_write(digits[i]);
```

```
                }
        twoWire_ack();
    }
}
/* ============================================================================
 * Function: TM1637_display
 * Returns: void = no return.
 * Description: Configures TM1637 and displays two numbers with given brightness.
 * ============================================================================ */
void TM1637_display(short number1,short number2){

        TM1637_setBrightness(4);

    twoWire_start();
    twoWire_write(0x40);
    twoWire_ack();
    twoWire_stop();

    twoWire_start();
    twoWire_write(0xC0);
    twoWire_ack();
    TM1637_write(number1,number2);

    twoWire_stop();

}
/* ============================================================================
 * Function: charToSegments
 * Returns: char = segment bitmap.
 * Description: Maps ASCII char to 7-segment encoding for common-anode TM1637.
 * ============================================================================ */
char charToSegments(char c) {

    if((char)c >= (char)'0' && (char)c <= (char)'9')        return digits[c - '0'];
    else if((char)c == (char)'-')               return digits[10];
    else if((char)c == (char)' ')               return digits[11];
    else if((char)c >= (char)'A' && (char)c <= (char)'Z')   return digits[12 + (c - 'A')];
    else if((char)c >= (char)'a' && (char)c <= (char)'z')   return digits[12 + (c - 'a')];
    else                            return 0x00;
}
/* ============================================================================
 * Function: TM1637_writeStringWindow
 * Returns: void = no return.
 * Description: Writes exactly 6 characters to TM1637 using physical position mapping.
 * ============================================================================ */
void TM1637_writeStringWindow(const char *s, char start, char len) {

        // physical order of the positions on screen (0 = left, 5 = right)
        // adjust this array if the order is different

    const char mapping[6] = {2,1,0,5,4,3};

    char i;
    for (i = 0; i < 6; i++) {
        char idx = start + mapping[i];
        char ch;
        if (idx < len) ch = s[idx];
        else ch = ' ';
        twoWire_write(charToSegments(ch));
        twoWire_ack();
    }
}
/* ============================================================================
 * Function: TM1637_displayString
 * Returns: void = no return.
 * Description: Displays a string on 6 digits; scrolls if longer than 6.
 * ============================================================================ */
void TM1637_displayString(char *string) {

    char len;
    char start;

        // init
        len = strlen(string);

        TM1637_setBrightness(4);

    if (len <= 6) {
        twoWire_start();
        twoWire_write(0x40);
        twoWire_ack();
        twoWire_stop();

        twoWire_start();
        twoWire_write(0xC0);
        twoWire_ack();
```

```
        TM1637_writeStringWindow(string, 0, len);
        twoWire_stop();
    }
    else {
        for (start = 0; start <= (len - 6); start++) {
            twoWire_start();
            twoWire_write(0x40);
            twoWire_ack();
            twoWire_stop();

            twoWire_start();
            twoWire_write(0xC0);
            twoWire_ack();
            TM1637_writeStringWindow(string, start, len);
            twoWire_stop();

            delayMainLoop(1);
        }
    }
}
/* ================================================================================
 * Function: TM1637_setBrightness
 * Returns: void = no return.
 * Description: Sets TM1637 brightness (0..8). 0 turns display off.
 * ================================================================================ */
void TM1637_setBrightness(char level){

    twoWire_start();
    twoWire_write(0x87 + level);
    twoWire_ack();
    twoWire_stop();
}
/* ================================================================================
 * Function: setPort
 * Returns: void = no return.
 * Description: Updates microcontroller output latches (A..D) from current state
 * variables; handles stopAll.
 * ================================================================================ */
void setPort(void){

        char myPortA; // used to better synchronised output updates
        char myPortB; // used to better synchronised output updates
        char myPortC; // used to better synchronised output updates
        char myPortD; // used to better synchronised output updates

        if ((char)gl_stopAll==(char)TRUE) {
                gl_S1T0char=0; gl_S2T0char=0;
                gl_S1T1char=0; gl_S2T1char=0;
                gl_S1T2char=0; gl_S2T2char=0;
                gl_S1T3char=0; gl_S2T3char=0;
                LATA=0xFF;
                LATB=0xFF;
                LATC=0xFF;
        LATD=0xFF;
        }
        else {
            myPortA=(gl_S1T0char<<4) + (gl_S1T1char<<6) + (gl_S2T0char<<7);
            myPortB=(gl_OUTchar[2]&1) + ((gl_OUTchar[3]&1)<<1) + ((gl_OUTchar[4]&1)<<4) + ((gl_OUTchar[5]&1)<<5);
                myPortC=(gl_S2T1char) + (gl_S1T2char<<1) + (gl_S2T2char<<2) + (gl_S1T3char<<3)
+((char)TRISCbits.RC4==(char)0 ? gl_GPIOchar[3] <<4 : 0);
                myPortD=(gl_S2T3char) + ((char)TRISDbits.RD1==(char)0 ? gl_GPIOchar[0] <<1 :0) +
((char)TRISDbits.RD2==(char)0 ? gl_GPIOchar[1] <<2:0) +((char)TRISDbits.RD3==(char)0 ? gl_GPIOchar[2] <<3:0) +
((gl_OUTchar[0]&1)<<6) +  ((gl_OUTchar[1]&1)<<7);
                LATA=myPortA;
            LATC=myPortC;
            LATD=myPortD;
            LATB=myPortB;
        }

}

/*******************************************************************************
 * low_interrupt
 *******************************************************************************/
#pragma code low_vector=0x18
/* ================================================================================
 * Function: low_interrupt
 * Returns: void = vector stub.
 * Description: Low-priority interrupt vector stub that jumps to low_isr.
 * ================================================================================ */
void low_interrupt (void){
    _asm goto low_isr _endasm
}
#pragma code

#pragma interruptlow low_isr
```

```
/* ============================================================================
 * Function: low_isr_task
 * Returns: void = no return.
 * Description: Low-priority background task called from ISR: manages flashing,
 * reads knobs/ADC and computes averages.
 * ============================================================================ */
void low_isr_task(void) {

char    bitStateCounter;
char    delay;
char    selectBitDelay;
char    bitNumber;
char    bitValue;
short   ADC;
int     knobValue1;

        if (gl_goFlashingCounter==1)gl_flashingCounter++;
        if (((gl_flashingCounter & 0xFFF) == 0) || ((gl_flashingCounter & 0xFFF) == 0x7FF))gl_goFlashingCounter=0;

        // KNOBS MANAGEMENT
        gl_getKnobValue++;
        if (gl_getKnobValue==0) {

                gl_numberKnobData++;

                // KNOB VALUE
                ADCON0=INKNOB0;
                ADCON0bits.GO = 1;                              // ADCON0.GODONE = 1
                while((char)ADCON0bits.GO == (char)1);          // wait till GODONE bit is zero
                ADC = ADRESH;    //Read converted result
                ADC = (ADC<<8) + ADRESL;

                gl_adcKnobValue0+=ADC;

                ADCON0=INKNOB1;
                ADCON0bits.GO = 1;                              // ADCON0.GODONE = 1
                while((char)ADCON0bits.GO == (char)1);          // wait till GODONE bit is zero
                ADC = ADRESH;    //Read converted result
                ADC = (ADC<<8) + ADRESL;

                gl_adcKnobValue1+=ADC;

                if(gl_numberKnobData>=40) {
                        gl_numberKnobData=0;
                        gl_adcKnobValue0=gl_adcKnobValue0/20;
                        gl_adcKnobValue1=gl_adcKnobValue1/20;

                        if (gl_calibKnob==1) {
                                if (gl_minAdcKnobValue0>gl_adcKnobValue0)gl_minAdcKnobValue0=gl_adcKnobValue0;
                                if (gl_minAdcKnobValue1>gl_adcKnobValue1)gl_minAdcKnobValue1=gl_adcKnobValue1;
                                if (gl_maxAdcKnobValue0<gl_adcKnobValue0)gl_maxAdcKnobValue0=gl_adcKnobValue0;
                                if (gl_maxAdcKnobValue1<gl_adcKnobValue1)gl_maxAdcKnobValue1=gl_adcKnobValue1;
                                gl_deltaKnob0=gl_maxAdcKnobValue0-gl_minAdcKnobValue0;
                                gl_deltaKnob1=gl_maxAdcKnobValue1-gl_minAdcKnobValue1;

                        }

                        // Speed
                        gl_knobValue0=(31*((long)(gl_adcKnobValue0-gl_minAdcKnobValue0))/(long)(gl_deltaKnob0))-
15;
                        if (gl_knobValue0>15)gl_knobValue0=15;
                        else if (gl_knobValue0==-1 || gl_knobValue0==1)gl_knobValue0=0; // For stability around 0

                        // Inertia
                        knobValue1=(101*(long)(gl_adcKnobValue1-gl_minAdcKnobValue1))/(long)(gl_deltaKnob1);
                        if (knobValue1>100)knobValue1=100;
                        if(gl_knobValue1==0 && knobValue1==100)knobValue1=0; // Should happen if potentiometer
lost contact
                        gl_knobValue1=knobValue1;

                        gl_adcKnobValue0=0;
                        gl_adcKnobValue1=0;
                }
        }

        // GPIO IN Detection
        if((char)TRISDbits.RD1==(char)1 && (char)PORTDbits.RD1!=(char)gl_GPIOchar[0]){
                gl_GPIOstabilized[0]++;
                if (gl_GPIOstabilized[0]>(unsigned int)GPIOTHRESHOLD) {
                        gl_GPIOchar[0]=PORTDbits.RD1;
                        if (gl_GPIOchar[0]==1)gl_GPIOcounter[0]++;
                        gl_GPIONotification[0]=TRUE;
                        gl_GPIOstabilized[0]=0;
                }
        }
        else gl_GPIOstabilized[0]=0;
```

```
        if((char)TRISDbits.RD2==(char)1 && (char)PORTDbits.RD2!=(char)gl_GPIOchar[1]){
                gl_GPIOstabilized[1]++;
                if (gl_GPIOstabilized[1]>(unsigned int)GPIOTHRESHOLD) {
                        gl_GPIOchar[1]=PORTDbits.RD2;
                        if (gl_GPIOchar[1]==1)gl_GPIOcounter[1]++;
                        gl_GPIONotification[1]=TRUE;
                        gl_GPIOstabilized[1]=0;
                }
        }
        else gl_GPIOstabilized[1]=0;

        if((char)TRISDbits.RD3==(char)1 && (char)PORTDbits.RD3!=(char)gl_GPIOchar[2]){
                gl_GPIOstabilized[2]++;
                if (gl_GPIOstabilized[2]>(unsigned int)GPIOTHRESHOLD) {
                        gl_GPIOchar[2]=PORTDbits.RD3;
                        if (gl_GPIOchar[2]==1)gl_GPIOcounter[2]++;
                        gl_GPIONotification[2]=TRUE;
                        gl_GPIOstabilized[2]=0;
                }
        }
        else gl_GPIOstabilized[2]=0;

        if((char)TRISCbits.RC4==(char)1 && (char)PORTCbits.RC4!=(char)gl_GPIOchar[3]){
                gl_GPIOstabilized[3]++;
                if (gl_GPIOstabilized[3]>(unsigned int)GPIOTHRESHOLD) {
                        gl_GPIOchar[3]=PORTCbits.RC4;
                        if (gl_GPIOchar[3]==1)gl_GPIOcounter[3]++;
                        gl_GPIONotification[3]=TRUE;
                        gl_GPIOstabilized[3]=0;
                }
        }
        else gl_GPIOstabilized[3]=0;

        if ((char)gl_stopAll==(char)FALSE) {

                gl_trackNumber=gl_trackNumber+1;
                if (gl_trackNumber>3)gl_trackNumber=0;

                // TIMER
                gl_timer--;
                if ((unsigned short)gl_timer==(unsigned short)0) {
                        gl_timer=INITTIMERVALUE;
                        if (gl_TIMERValue[gl_timerNumber]>0){
                                gl_TIMERValue[gl_timerNumber]--;
                                if (gl_TIMERValue[gl_timerNumber]==0)gl_TIMERNotification[gl_timerNumber]=TRUE;
                        }
                        gl_timerNumber++;
                        if(gl_timerNumber>MAXTIMER)gl_timerNumber=0;
                }

                /////////////// MODE ANALOG /////////////////
                if(gl_boardMode==ANAValue) {
                        gl_speedCounter++;
                        if (gl_speedCounter>MAX_INERTIA_COUNTER) {
                                gl_speedCounter=1;
                        }

                        if (gl_curSpeed[gl_trackNumber]!=gl_setPoint[gl_trackNumber]) {
                                gl_setStepCounter[gl_trackNumber]--;
                                if (gl_setStepCounter[gl_trackNumber]<=0) {
                                        gl_setStepCounter[gl_trackNumber]=gl_setStep[gl_trackNumber];
                                        if (gl_curSpeed[gl_trackNumber]>gl_setPoint[gl_trackNumber]){
                                                gl_curSpeed[gl_trackNumber]-=MAX_STEP;

        if(gl_curSpeed[gl_trackNumber]<gl_setPoint[gl_trackNumber])gl_curSpeed[gl_trackNumber]=gl_setPoint[gl_track
Number];
                                        }
                                        else {
                                                gl_curSpeed[gl_trackNumber]+=MAX_STEP;

        if(gl_curSpeed[gl_trackNumber]>gl_setPoint[gl_trackNumber])gl_curSpeed[gl_trackNumber]=gl_setPoint[gl_track
Number];
                                        }
                                }

                                if (gl_curSpeed[gl_trackNumber]>0) {
                                        gl_speed[gl_trackNumber]=gl_curSpeed[gl_trackNumber]/(MAXINTERNALSPEED);
                                        gl_direction[gl_trackNumber]=TRACK_BACKWARD;
                                }
                                else if (gl_curSpeed[gl_trackNumber]<0) {
                                        gl_speed[gl_trackNumber]=-gl_curSpeed[gl_trackNumber]/(MAXINTERNALSPEED);
                                        gl_direction[gl_trackNumber]=TRACK_FORWARD;
                                }
                                else {
                                        gl_direction[gl_trackNumber]=TRACK_STOP;
```

```
                                        gl_speed[gl_trackNumber]=0;
                                }
                        }

                        // Set Port Value when all tracks have been updated
                        if (gl_trackNumber==0) {
                                for(gl_trackNumber=0;gl_trackNumber<4;gl_trackNumber++) {
                                        if (gl_speed[gl_trackNumber]>=gl_speedCounter) {
                                                if (gl_direction[gl_trackNumber]==TRACK_FORWARD) {
                                                switch (gl_trackNumber) {
                                                        case 0:gl_S1T0char=1; gl_S2T0char=0;break;
                                                        case 1:gl_S1T1char=1; gl_S2T1char=0;break;
                                                        case 2:gl_S1T2char=1; gl_S2T2char=0;break;
                                                        case 3:gl_S1T3char=1; gl_S2T3char=0;break;
                                        }
                                        }
                                        if (gl_direction[gl_trackNumber]==TRACK_BACKWARD) {
                                        switch (gl_trackNumber) {
                                                case 0:gl_S1T0char=0; gl_S2T0char=1;break;
                                                case 1:gl_S1T1char=0; gl_S2T1char=1;break;
                                                case 2:gl_S1T2char=0; gl_S2T2char=1;break;
                                                case 3:gl_S1T3char=0; gl_S2T3char=1;break;
                                        }
                                        }
                                        if (gl_direction[gl_trackNumber]==TRACK_STOP) {
                                        switch (gl_trackNumber) {
                                                case 0:gl_S1T0char=0; gl_S2T0char=0;break;
                                                case 1:gl_S1T1char=0; gl_S2T1char=0;break;
                                                case 2:gl_S1T2char=0; gl_S2T2char=0;break;
                                                case 3:gl_S1T3char=0; gl_S2T3char=0;break;
                                        }
                                        }
                                        }
                                        else {
                                                switch (gl_trackNumber) {
                                        case 0:gl_S1T0char=0; gl_S2T0char=0;break;
                                        case 1:gl_S1T1char=0; gl_S2T1char=0;break;
                                        case 2:gl_S1T2char=0; gl_S2T2char=0;break;
                                        case 3:gl_S1T3char=0; gl_S2T3char=0;break;
                                        }
                                        }
                                }
                                gl_trackNumber=0;
                                setPort();
                        }
                }
                else if(gl_boardMode==DCCValue && gl_dcc_ready==0) {

                        for (bitStateCounter=0;bitStateCounter<FRAME_SIZE;bitStateCounter++) {

                                if (gl_dcc[bitStateCounter]==0) selectBitDelay=DCC_0;
                                else selectBitDelay=DCC_1;

                                gl_S1T0char=0;gl_S1T1char=0;
                                gl_S1T2char=0;gl_S1T3char=0;
                                gl_S2T0char=1;gl_S2T1char=1;
                                gl_S2T2char=1;gl_S2T3char=1;
                        setPort();
                                for (delay=0;delay<selectBitDelay;delay++);

                        gl_S2T0char=0;gl_S2T1char=0;
                                gl_S2T2char=0;gl_S2T3char=0;
                                gl_S1T0char=1;gl_S1T1char=1;
                                gl_S1T2char=1;gl_S1T3char=1;
                        setPort();
                                for (delay=0;delay<selectBitDelay;delay++);
                        }

                        gl_S1T0char=0; gl_S2T0char=1;
                        gl_S1T1char=0; gl_S2T1char=1;
                        gl_S1T2char=0; gl_S2T2char=1;
                        gl_S1T3char=0; gl_S2T3char=1;
                setPort();

                        for (delay=0;delay<selectBitDelay;delay++);
                        for
(bitStateCounter=0;bitStateCounter<FRAME_SIZE;bitStateCounter++)gl_dcc[bitStateCounter]=1;
                }
                gl_dcc_ready--;
                if (gl_dcc_ready<0) gl_dcc_ready=INITWAITDCCCOUNTER;


                // TRACK DETECTION

                switch(gl_trackNumber) {
                        case 0 : ADCON0=CURT0;break;
```

```
                case 1 : ADCON0=CURT1;break;
                case 2 : ADCON0=CURT2;break;
                        case 3 : ADCON0=CURT3;break;
                }
                // NEED TO GET LOW VOLTAGE VALUE WHEN TRACK IS OFF FOR CALIBRATION AT POWER ON
                if ((char)gl_trackCalibration==(char)TRUE) {
                        ADCON0bits.GO = 1;                              // ADCON0.GODONE = 1
                        while((char)ADCON0bits.GO ==(char) 1);                  // wait till GODONE bit is zero
                        ADC = 0;
                        ADC = ADRESH;    //Read converted result
                        ADC = (ADC<<8) + ADRESL;

        gl_average[gl_trackNumber]=(SAMPLEFORCALIBRATION*gl_average[gl_trackNumber]+ADC)/(SAMPLEFORCALIBRATION+1);
                        if
(gl_noVehicule[gl_trackNumber]>gl_average[gl_trackNumber])gl_noVehicule[gl_trackNumber]=gl_average[gl_trackNumber];
                }

                if(((char)gl_speed[gl_trackNumber]==(char)gl_speedCounter && (char)gl_boardMode==(char)ANAValue) ||
((char)gl_dcc_ready==(char)INITWAITDCCCOUNTER && (char)gl_boardMode==(char)DCCValue)) { // ONLY WHEN POWER ON
                        ADCON0bits.GO = 1;                              // ADCON0.GODONE = 1
                        while((char)ADCON0bits.GO == (char)1);                  // wait till GODONE bit is zero
                        ADC = 0;
                        ADC = ADRESH;    //Read converted result
                        ADC = (ADC<<8) + ADRESL;

                        if (gl_average[gl_trackNumber]<ADC) gl_average[gl_trackNumber]=ADC; // TRAP THE EVENT
                        else
gl_average[gl_trackNumber]=(SAMPLEFORAVERAGE*gl_average[gl_trackNumber]+ADC)/(SAMPLEFORAVERAGE+1);

                        if
(((int)(10*gl_average[gl_trackNumber])>(int)((10+HYSTERERISHIGH)*gl_noVehicule[gl_trackNumber])) &&
((char)gl_OUTSTATchar[gl_trackNumber]==(char)0)  && ((char)gl_trackNotification[gl_trackNumber]==(char)FALSE)) {
                                gl_OUTSTATchar[gl_trackNumber]=1;
                                gl_trackNotification[gl_trackNumber]=TRUE;
                        }
                        else if
(((int)(10*gl_average[gl_trackNumber])<(int)((10+HYSTERERISLOW)*gl_noVehicule[gl_trackNumber])) &&
((char)gl_OUTSTATchar[gl_trackNumber]==(char)1) && ((char)gl_trackNotification[gl_trackNumber]==(char)FALSE)) {
                                gl_OUTSTATchar[gl_trackNumber]=0;
                                gl_trackNotification[gl_trackNumber]=TRUE;
                        }
                }
        }
        else setPort();
}
/* ============================================================================
 * Function: low_isr
 * Returns: void = no return.
 * Description: Call low interrupt code
 * ============================================================================ */
void low_isr(void){

        if (!gl_mutexLowIsr) {
                low_isr_task();
        }

    // INTERRUPT RESET
    if((char)INTCONbits.TMR0IF==(char)1){
             INTCONbits.TMR0IF = 0;
                T0CONbits.PSA                   = 0;   // Timer0 prescaler is assigned
                T0CONbits.T0PS0                 = 0;   // Prescale value
                T0CONbits.T0PS1                 = 0;   // Prescale value
                T0CONbits.T0PS2                 = 0;   // Prescale value
    }
}
#pragma code
```

The DCC frame in standard NMRA format is generated by the setDcc() function and sent by `low_isr(void)` shown above . Please note that the timings have been adjusted by the following define values:

```
// DELAY FOR DCC SIGNAL
#define DCC_0 48
#define DCC_1 18
```

Any modification to the code, such as the use of `int` instead of `char` for example, may modify the timing and cause the DCC signal to become invalid. In this case, it is necessary to check the correct timing values of the DCC signal with an oscilloscope (58us for a 1 and 100us for a 0).

```
/* ================================================================================
 * Function: setDcc
 * Returns: void = no return.
 * Description: Builds a full DCC packet from address and command into gl_dcc buffer.
 * ================================================================================ */
void setDcc(char address, char command) {

         char i;
         char bitNumber;
         char control;

        control=address ^ command; // Control
        bitNumber=0;

        // PREAMBULE
    for(i=0;i<PREAMBLE_SIZE;i++) {
                gl_dcc[bitNumber++]=1;
        }
        // 0
        gl_dcc[bitNumber++]=0;

        // ADDRESS
    for(i=0;i<8;i++) {
                gl_dcc[bitNumber++]=address >> (7-i) & 1;
        }
        // 0
        gl_dcc[bitNumber++]=0;

        // COMMAND
    for(i=0;i<8;i++) {
                gl_dcc[bitNumber++]=command >> (7-i) & 1;
        }
        // 0
        gl_dcc[bitNumber++]=0;

        // CONTROL
    for(i=0;i<8;i++) {
                gl_dcc[bitNumber++]=control >> (7-i) & 1;
        }
        // 1
        gl_dcc[bitNumber++]=1;
        gl_dcc[bitNumber++]=1; // Only one is enough, but in case of....
}
```

5.4 Flash storage of programming information

EEPROM data storage is managed by a set of flash read/write functions. In order to get more space, automations are compressed inside EEPROM.

```
/* ================================================================================
 * Function: ReadEEPROM
 * Returns: char = SUCCESS on read, ERROR if address invalid.
 * Description: Reads one byte from data EEPROM at the given address into *data.
 * Checks bounds and selects data EEPROM space.
 * ================================================================================ */
char ReadEEPROM(int adr, char *data){

        if(adr > 0x3FF){
                return(ERROR);
        }
        else{
                EEADR = adr&0xFF;
                EEADRH = (adr>>8) & 0x3;
                EECON1bits.EEPGD = 0;     // Point to data memory
                EECON1bits.CFGS = 0;      // Access EEPROM
                EECON1bits.RD = 1;                // Read data
                *data = EEDATA;                   // Load data
                return(SUCCESS);
        }
}
/* ================================================================================
 * Function: ResetEEPROM
 * Returns: void = no return.
 * Description: Initializes EEPROM to default values (magic number, ANA mode,
```

```
 * clears automation, sets GPIO directions) and syncs globals.
 * ============================================================================ */
void ResetEEPROM(void){

         char    value;
         short adr;
         char  checkMagicNumberCounter;

        // Init EEPROM after flashing the board
        adr=(short)MAGICNUMBER_ADDRESS;
        for (checkMagicNumberCounter=0;checkMagicNumberCounter<MAGICNUMBERSIZE;checkMagicNumberCounter++) {
                WriteEEPROM(adr++,checkMagicNumberCounter);
        }
        // Set ANA mode
        adr=(short)MODE_ADDRESS;
        value=ANAValue;
        WriteEEPROM(adr,value);

        // No automation
        adr=(short)NEXTTAUTOMATION_ADDRESS;
        value=0;
        WriteEEPROM(adr,value);

        gl_mutexLowIsr=1;gl_boardMode = ANAValue;gl_nexAvailableAutomation=0;gl_mutexLowIsr=0;

        // GPIO IN
        for(adr=(short)GPIO0DIR_ADDRESS;adr<=(short)GPIO0DIR_ADDRESS+3;adr++)WriteEEPROM(adr,1);

}
/* ============================================================================
 * Function: ReadEEPROMConfig
 * Returns: void = no return.
 * Description: Loads configuration from EEPROM: validates magic, board mode,
 * GPIO directions, knob calibration, and automation cache.
 * ============================================================================ */
void ReadEEPROMConfig(void) {

         char    value;
         char    dummy;
         short adr;
         char  automationCounter;
         char  automationDataCounter;
         char  checkMagicNumberCounter;

        // Read MAGIC NUMBER
        adr=(char)MAGICNUMBER_ADDRESS;
        for (checkMagicNumberCounter=0;checkMagicNumberCounter<MAGICNUMBERSIZE;checkMagicNumberCounter++) {
                ReadEEPROM(adr++,&value);
                if (value!=checkMagicNumberCounter) {

                        ResetEEPROM();
                        return;
                }
        }

        // Read in EEPROM MODE
        adr=(short)MODE_ADDRESS;
        ReadEEPROM(adr,&value);
        gl_mutexLowIsr=1;gl_boardMode=value;gl_mutexLowIsr=0;

        // Read in GPIO dir
        adr=(short)GPIO0DIR_ADDRESS;
        ReadEEPROM(adr++,&value);
        TRISDbits.RD1=value;
        ReadEEPROM(adr++,&value);
        TRISDbits.RD2=value;
        ReadEEPROM(adr++,&value);
        TRISDbits.RD3=value;
        ReadEEPROM(adr,&value);
        TRISCbits.RC4=value;

        // Knob min & max
        adr=(short)KNOB_ADDRESS;
        ReadEEPROM(adr++,&value);
        gl_mutexLowIsr=1;gl_minAdcKnobValue0=value;gl_mutexLowIsr=0;
        ReadEEPROM(adr++,&value);
        gl_mutexLowIsr=1;gl_minAdcKnobValue0=value+(gl_minAdcKnobValue0<<8);gl_mutexLowIsr=0;
        ReadEEPROM(adr++,&value);
        gl_mutexLowIsr=1;gl_maxAdcKnobValue0=value;gl_mutexLowIsr=0;
        ReadEEPROM(adr++,&value);
        gl_mutexLowIsr=1;gl_maxAdcKnobValue0=value+(gl_maxAdcKnobValue0<<8);gl_mutexLowIsr=0;
        ReadEEPROM(adr++,&value);
        gl_mutexLowIsr=1;gl_minAdcKnobValue1=value;gl_mutexLowIsr=0;
        ReadEEPROM(adr++,&value);
        gl_mutexLowIsr=1;gl_minAdcKnobValue1=value+(gl_minAdcKnobValue1<<8);gl_mutexLowIsr=0;
        ReadEEPROM(adr++,&value);
```

```
        gl_mutexLowIsr=1;gl_maxAdcKnobValue1=value;gl_mutexLowIsr=0;
        ReadEEPROM(adr,&value);
        gl_mutexLowIsr=1;gl_maxAdcKnobValue1=value+(gl_maxAdcKnobValue1<<8);gl_mutexLowIsr=0;

        gl_mutexLowIsr=1;gl_deltaKnob0=gl_maxAdcKnobValue0-gl_minAdcKnobValue0;gl_mutexLowIsr=0;
        gl_mutexLowIsr=1;gl_deltaKnob1=gl_maxAdcKnobValue1-gl_minAdcKnobValue1;gl_mutexLowIsr=0;

        // Read in EEPROM last automation
        dummy=getAutomationFromEEPROM();
}
/* ============================================================================
 * Function: WriteCompletedEEPROM
 * Returns: char = SUCCESS when complete, ERROR otherwise.
 * Description: Reports whether an EEPROM write cycle has completed and clears flags.
 * ============================================================================ */
char WriteCompletedEEPROM(void) {

        if(PIR2bits.EEIF){
                PIR2bits.EEIF=0;          // Clear write complete flag
                EECON1bits.WREN = 0;      // Disable write
                return(SUCCESS);          // Write to EEPROM completed
        }
        else {
                return(ERROR);                    // Write to EEPROM not completed
        }
}
/* ============================================================================
 * Function: WriteRdyEEPROM
 * Returns: char = SUCCESS when ready, ERROR when busy.
 * Description: Indicates if EEPROM is ready for a new write (no write in progress).
 * ============================================================================ */
char WriteRdyEEPROM(void){

        if(!EECON1bits.WR) {
                return(SUCCESS); // New Write Enabled
        }
        else {
                return(ERROR);            // new Write Disabled
        }
}
/* ============================================================================
 * Function: WriteEEPROM
 * Returns: char = SUCCESS on write, ERROR if address invalid.
 * Description: Writes one byte to data EEPROM at the given address with unlock
 * sequence; skips write if data unchanged.
 * ============================================================================ */
char WriteEEPROM(short adr, char data){

        char checkValue;
        if(adr > 0x3FF){
                return(ERROR);
        }
        else{

                // Wait eeprom ready to be written
                while (WriteRdyEEPROM()==(char) ERROR);

                // Check if this value already exists at that address
                ReadEEPROM(adr,&checkValue);
                if (checkValue==data)return(SUCCESS);

                EEADR = adr&0xFF;                 // Address of the data in EEPROM
                EEADRH = (adr>>8) & 0x3;  // Address of the data in EEPROM
                EEDATA = data;                          // Data to write in EEPROM
                EECON1bits.EEPGD = 0;             // Point to data memory
                EECON1bits.CFGS = 0;              // Access EEPROM
                EECON1bits.WREN = 1;              // Enable write
                INTCONbits.GIE   = 0;             // Disable Interrupt
                EECON2 = 0x55;
                EECON2 = 0x0AA;
                EECON1bits.WR = 1;                        // Begin write

                // Wait data written
                while (WriteCompletedEEPROM()==(char) IN_PROGRESS);
                while (WriteRdyEEPROM()==(char) ERROR);

                INTCONbits.GIE            = 1;    // Enable Interrupt

                return(SUCCESS);
        }
}
/* ============================================================================
 * Function: getAutomationFromEEPROM
 * Returns: char = percentage 0..100 (100 if none).
 * Description: Reads automation entries from EEPROM into RAM and returns free
 * space percentage.
```

```
 * ============================================================================ */
char getAutomationFromEEPROM(void) {

    char  automationCounter;
    char  automationDataCounter;
        char  value;
        short adr;
        long                percentage;

        // Get next automation address
        adr=(short)NEXTTAUTOMATION_ADDRESS;
        ReadEEPROM(adr,&value);
        gl_mutexLowIsr=1;gl_nexAvailableAutomation=value;gl_mutexLowIsr=0;
        if (gl_nexAvailableAutomation==0) return((double)100); // nothing to do

        // Read and uncompress automation from EEPROM
        adr=(short)AUTOMATION_ADDRESS;
        for(automationCounter=0;automationCounter<gl_nexAvailableAutomation;automationCounter++) {
                for(automationDataCounter=0;automationDataCounter<NEW_AUTOMATIONSIZE;automationDataCounter++) {
                        ReadEEPROM(adr++,&value);

        gl_mutexLowIsr=1;gl_automation[automationCounter][automationDataCounter]=value;gl_mutexLowIsr=0;
                }
        }
        percentage=100*(1024-(long)adr)/(1024-(long)AUTOMATION_ADDRESS);
        return((char)percentage);
}
/* ============================================================================
 * Function: setAutomationToEEPROM
 * Returns: char = TRUE on success, FALSE on overflow.
 * Description: Writes automation entries from RAM to EEPROM and updates the
 * 'next automation' pointer.
 * ============================================================================ */
char setAutomationToEEPROM(void) {

    char  automationCounter;
    char  automationDataCounter;
        short adr;
        char  value;

        adr=(short)AUTOMATION_ADDRESS;

        for(automationCounter=0;automationCounter<gl_nexAvailableAutomation;automationCounter++) {
                if (adr+NEW_AUTOMATIONSIZE>=1024) {
                        mySprintf((char *)gl_errorInfo,"limit is %d",(int)gl_nexAvailableAutomation);
                        gl_parserErrorCode=AUTOMATIONSIZELIMIT;
                        return(FALSE);
                }
                for(automationDataCounter=0;automationDataCounter<NEW_AUTOMATIONSIZE;automationDataCounter++) {
                        value=gl_automation[automationCounter][automationDataCounter];
                        WriteEEPROM(adr++,value);
                }
        }

        // update in EEPROM next automation value
        adr=(short)NEXTTAUTOMATION_ADDRESS;
        value=gl_nexAvailableAutomation;
        WriteEEPROM(adr,value);
        return(TRUE);
}

/* ============================================================================
 * Function: uncompressData
 * Returns: char = TRUE on success, FALSE on overflow.
 * Description: Run-length decodes gl_request buffer from (count,value) pairs
 * into raw bytes.
 * ============================================================================ */
char uncompressData(void) {
        char dataCounter;
        char tmpDataCounter;
        char repeat;

        // Codage is simply a list of (X,Y) where X is the number of Y. If X = 0 there is no more data

        tmpDataCounter=0;
        for(dataCounter=0;dataCounter<REQUESTSIZE;dataCounter++) gl_tmpBuffer[dataCounter]=0;

        for(dataCounter=0;dataCounter<MAXTRAMESIZE-2;dataCounter+=2) {
                if(gl_request[dataCounter]==0) break;
                for(repeat=0;repeat<gl_request[dataCounter];repeat++) {
                        if (tmpDataCounter>=REQUESTSIZE) {
                                initRequest();
                                return(FALSE);
                        }
                        gl_tmpBuffer[tmpDataCounter++]=gl_request[dataCounter+1];
                }
```

```
        }
        for(dataCounter=0;dataCounter<REQUESTSIZE;dataCounter++) gl_request[dataCounter]=gl_tmpBuffer[dataCounter];
        return(TRUE);
}
/* ================================================================================
 * Function: compressData
 * Returns: char = number of bytes produced (0 on failure).
 * Description: Run-length encodes gl_request into (count,value) pairs for
 * CAN transmission.
 * ================================================================================ */
char compressData(void) {

        char tmpDataCounter;
        char dataCounter;
        char quantityValue;
        char curValue;

        // Init
        dataCounter=0;
        tmpDataCounter=0;
        curValue=gl_request[0];
        quantityValue=0;

        while(dataCounter<REQUESTSIZE) {
                while(gl_request[dataCounter]==curValue) {
                        quantityValue++;
                        dataCounter++;
                        if (dataCounter>=REQUESTSIZE) break;
                }
                if (tmpDataCounter<MAXTRAMESIZE-1 && tmpDataCounter<REQUESTSIZE-2) {
                        gl_tmpBuffer[tmpDataCounter++]=quantityValue;
                        gl_tmpBuffer[tmpDataCounter++]=curValue;
                }
                if (tmpDataCounter>=MAXTRAMESIZE) {
                        for(dataCounter=0;dataCounter<REQUESTSIZE;dataCounter++) gl_request[dataCounter]=0;
                        return(0); // We loose the trame, because we can't compress it, should never happen.....
                }
                curValue=gl_request[dataCounter];
                quantityValue=0;
        }
        for(dataCounter=0;dataCounter<tmpDataCounter;dataCounter++)
gl_request[dataCounter]=gl_tmpBuffer[dataCounter];
        for(;dataCounter<REQUESTSIZE;dataCounter++) gl_request[dataCounter]=0;
        return(tmpDataCounter);
}
```

5.5 Automation management

The system is automated using a language that allows commands to be given or actions to be programmed on receipt of events. Syntax is parsed using a parser() function, and semantics and automation management are handled by a function for managing the system's various requests (from the RS232 link or the CAN bus).

A main loop in the `main()` function analyzes data from the RS232 link, CAN bus, internal timers, GPIOs or vehicle presence detection on the tracks.

5.6 Software design

5.7 Protocol on serial link

The protocol describes all the data exchanged between a PC and a master board. All data exchanged on the serial bus are in ASCII format, so the value 25 is transmitted with the letter "2" followed by the letter "5". This allows the board to be controlled directly from a terminal. Each transmission could be followed by one or more return messages.

- If message sent to the master needs a specific response
  - a status of a board (DCC or ANA)
  - a list of automation (value and name)
  - a status of a track
  - a status of GPIO
  - a status of a board
- Error: the board number with an error message

5.8 BNF Grammar

Sending a command on the serial bus uses the following BNF grammar:

```
Command         ::= <Mode> <Board Number> <Control> |
                    <Global Command>
Fashion         ::= PROG | COM
Board Number    ::= <between 0 and 31>
Control         ::= <Program> | <Command>
Global Command ::= STOP | RUNALL | RUN <Board Number> |RESET <Board Number>
```

For Programing Mode (P)

```
Program         ::= <Board Mode> |
                    <GPIO Setting> |
                    <Automation> |
                    <Del Automation>
Board Mode      ::= DCC | ANA
GPIO Setting    ::= GPIO <GPIO Number> <GPIO Dir>
GPIO number     ::= 0 | 1 | 2 | 3
GPIO Dir        ::= IN | OUT
Automation      ::= AUT <Identifier> <Manual Status> <Event> ACT <Action>
Manual Status   ::= AUTOFF | AUTON
Identify        ::= <List of 2 characters max between 2 spaces>
Event           ::= BOARD < Board Number> TIMER <Timer Number>
                    BOARD < Board Number> GPIO < Board Number> <GPIO Level>|
                    BOARD < Board Number> TRACK < Track Number> STA <vehicle status>
Board Number    ::= <between 0 and 31>
Timer number    ::= <A value between 0 and 15>
GPIO Level      ::= <0 or 1 for output, a value between 0 and 255 and input, counter mode>
Vehicle Status ::= ONTRACK | OFFTRACK
Action          ::= TIMER <Timer Number> <Timer Delay>
                    GPIO <GPIO Number> <GPIO Level> |
                    LPO <LPO Number> <LPO Level> |
                    TRACK <Track Number> SPEED <Track Speed> <Track Dir> INERTIA <Inertia>|
                    DCC <DCC NMRA>|
                    MANUAL | AUTOMATIC |
                    AUTOFF < identify> | AUTON < identify>
Timer delay ::= <A value between 0 and 255>
LPO Number      ::= 0 | 1 | 2 | 3 | 4 | 5
LPO Level       ::= 0 | 1 | 2
Track Number    ::= 0 | 1 | 2 | 3 | 4
Track Speed     ::= <A value between 0 and 99> | KNOB0 | KNOB1
Track Dir       ::= FORW | BACK
Inertia         ::= <A value between 0 and 99> | KNOB0 | KNOB1
Del Automation ::= DEL <Automation Number>
```

For Command Mode (C)

```
Command         ::= < > Action | <GPIO Status> | <LPO Status> | <Track Status> |
                    <Board Status> |<Automation List> | <Dump Memory> | <Knob Calib>
Action ::= TIMER<Timer Number> <Timer delay>
                    GPIO <GPIO Number> <GPIO Level> |
                    LPO <LPO Number> <LPO Level> |
                    TRACK <Track Number> SPEED <Track Speed> <Track Dir> INERTIA <Inertia>|
                    DCC <DCC NMRA> |
                    MANUAL | AUTOMATIC
                    AUTOFF < identify> | AUTON < identify>
Timer number    ::= <A value between 0 and 15>
Timer delay ::= <A value between 0 and 255>
GPIO Number     ::= 0 | 1 | 2 | 3
GPIO Level      ::= <0 or 1 for output, a value between 0 and 255 and input, counter mode>
LPO Number      ::= 0 | 1 | 2 | 3 | 4 | 5
LPO Level       ::= 0 | 1 | 2
Track Number    ::= 0 | 1 | 2 | 3 | 4
Track Speed     ::= <A value between 0 and 99> | KNOB0 | KNOB1
Inertia         ::= <A value between 0 and 99> | KNOB0 | KNOB1
AFC NMRA        ::= <See NMRA standard specification>
GPIO Status     ::= GSTAT
LPO Status      ::= LSTAT
Track Status    ::= TSTAT
Board Status ::= BSTAT
Automation List::= AUTLIST
Dump Memory     ::= DUMP
Knob Calib      ::= CALIB
```

Any other syntax in command or programing mode should produce an error.

Response from the master

```
Response          ::= <Acknowledge> |
                      Board <Board Number> : GPIO <GPIO Number> VAL <GPIO Level> <GPIO Dir> |
                      Board <Board Number> : LPO <LPO Number> VAL <LPO Level> |
                      Board <Board Number> : KNOB0 VAL <knob 0 value> |
                      Board <Board Number> : KNOB1 VAL <knob 1 value> |
                      Board <Board Number> : TRACK <Track Number> SPEED <Track Speed> <Track Dir><vehicle status>
                      Board <Board Number> : DCC |
                      Board <Board Number> : ANA |
                      Board <Board Number>: AUT <Number> <Identifier> ON | AUT <Number> <Identifier> OFF |
                      Board <Board Number> : <Memory content>
Memory content    ::= (<Automation number>,<Data index>,<Data>)<Memory Content>| <empty>
Vehicle Status    ::= ONTRACK | OFFTRACK
Acknowledge       ::= Board <Board Number> : <Error Message>
```

5.9 Protocol on CAN bus

Each data exchange on the CAN bus is made up of a frame containing the sender identifier (address value defined with the switch) and a set of data grouped on 8 bytes (Data Field); the other frame information is of no importance here. These fields are described below:

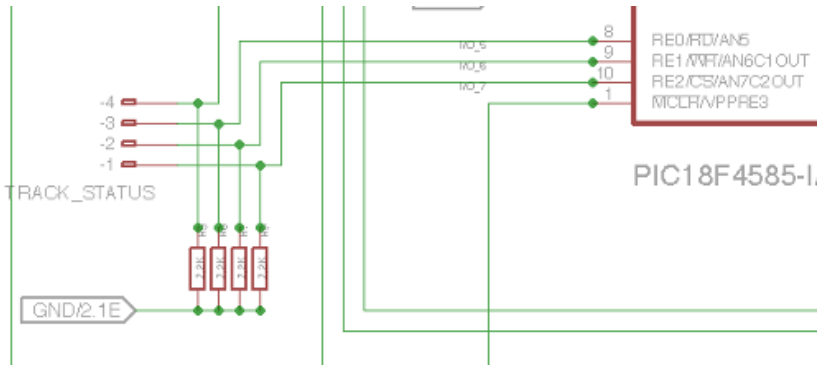
```
SOF (START OF FRAME): 1 BIT.
Frame start field always equal to 0.

IDENTIFIER: 11-BIT.
Identifies the message sender.

RTR (REMOTE TRANSMISSION REQUEST): 1 BIT.
Usually 0 except in the case of a request frame.

COMMAND: 6-BIT.
Contains the DLC (Data Length Code), the length of data transmitted in of bytes. For
example, if 4 bytes of data: DLC = 001000.

DATA : FROM 0 TO 8 BYTES.
```

```
CRC (CYCLIC REDUNDANCY CHECK): 16-BIT.
A calculation algorithm is used to check for transmission errors.

ACK (ACKNOWLEDGE): 2 BITS.
Acknowledges whether the frame has been read by a node.

EOF (END OF FRAME): 7 BITS.
Indicates the end of message transmission, 7 bits to 1: 1111111.
```

The data exchanged on the CAN bus is either text between two 8-byte frames (header 0xCC and footer 0xDD) or compressed data between two 8-byte frames (header 0xEE and footer 0xFF).

5.10 Output display and knob controls

We can connect 2 knobs and an output display of 6 7-segments LEDS compliant with TM1637 protocol on track status GPIO



- GPIO 4 is used for KNOB0 where a value between 0 and 5V must be read.
- GPIO 3 is used for KNOB1 where a value between 0 and 5V must be read.
- GPIO 2 is used for CLK (or SCK) line.
- GPIO 1 is used for DIO (or SDA) line.

On the SPEED and INERTIA command or program, the numerical value can be replaced by KNOB0 or KNOB1, in which case the value read when the command is executed on GPIO 4 or 3 is used.

KNOB0 has a value between -15 and 15, where -15 is 0V and 15 is 5V
KNOB1 has a value between 0 and 100, where 0 is 0V and 100 is 5V

Negative value used on SPEED reverses the running direction. The lowest inertia value corresponds to the shortest time needed to change speed.

Warning: A FORW command with a negative value will result in a BACK side and a BACK command with a negative value will result in a FORW command

The corresponding values are sent to the 7-segments LEDS display

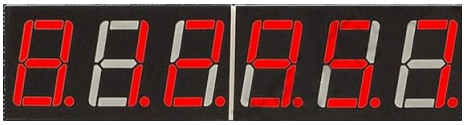


Figure 18 Led Display on TCO

The left side shows the value of KNOB0 and the right side of KNOB1
In MANUAL mode, speed and inertia have an immediate action on all the tracks
.

To calibrate the knobs, turn the min to max values of the two knobs and run the CALIB command. Calibration is saved in EEPROM**. Need to be done once after firmware update**.

6 Command

Programming DCC mode on a board

Description	This command allows you to configure a board to operate at the NMRA DCC standard. This action remains memorized when the power is off
Syntax	`PROG <Board Number> DCC`
Response	`No response`
Example	To configure board number 5 in DCC mode: **`PROG 5 DCC`**

Programming ANA mode on a board

Description	This command allows you to configure a board to operate in analogic mode using PWM on track for speed setting. This action remains memorized when the power is off
Syntax	`PROG <Board Number> ANA`
Response	`No response`
Example	To configure board number 5 in ANA mode: **`PROG 5 ANA`**

Programming AUTOMATIC mode on a board

Description	This command reactivates all AUTOFF-type commands and disables track control from the speed and inertia knobs.
Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status> <Event> ACT AUTOMATIC`
Response	`No response`
Example	To configure board number 2 in AUTOMATIC mode based on GPIO 3 status **`PROG 2 AUT MC AUTON BOARD 2 GPIO 3 0 ACT AUTOMATIC`**

Programming MANUAL mode on a board

Description	This command deactivates all AUTOFF-type commands and enables track control from the speed and inertia knobs. MANUAL0 also allows controlling only channel 0; otherwise, all four channels will be controlled by the knobs in this mode.
Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status> <Event> ACT MANUAL`
Response	`No response`
Example	To configure board number 2 in MANUAL mode based on GPIO 3 status **`PROG 2 AUT MC AUTON BOARD 2 GPIO 3 0 ACT MANUAL`**

Initialize a GPIO as output on a board

Description	This command allows you to configure a GPIO in output mode. This action remains memorized when the power is off (By default, the 4 GPIO are in input mode). Default value in output mode is 1.
Syntax	`PROG <Board Number> GPIO <GPIO Number> OUT`
Response	`No response`
Example	To configure GPIO 3 on board 5 in output mode **`PROG 5 GPIO 3 OUT`**

Initialize a GPIO as input on a board

Description	This command allows you to configure a GPIO in input mode (default mode). This action remains memorized when the power is off In input mode, the GPIO counts level changes from 0 to 255. To reset this counter to 0, simply initialize the GPIO to 0 as if it were an output GPIO, and the counter will be reset to zero.
Syntax	`PROG <Board Number> GPIO <GPIO Number> IN`
Response	`No response`
Example	To configure GPIO 3 on board 5 in input mode **`PROG 5 GPIO 3 IN`**

Program an automatic action on a GPIO driven by a timer

Description	This command allows you to program an automatic change on GPIO output level when a timer is triggered. This action remains memorized when the power is off
Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status> BOARD <Board Number> TIMER <Timer Number> ACT GPIO <GPIO Number> <GPIO Level>`
Response	`No response`
Example	To automatically switch the level of GPIO 3 of board 5 to level 1 when the TIMER 2 of board 2 is triggered **`PROG 5 AUT Test AUTOFF BOARD 2 TIMER 2 ACT GPIO 3 1`**
Detail	**`PROG 5`** `=> programming board number 5` **`AUT Test AUTOFF BOARD 2 TIMER 2`** `=> when TIMER 2 of board 2 is triggered. This action is not active in manual mode` **`ACT GPIO 3 1`** `=> GPIO 3 of board 5 is set to 1`

Program an automatic action on a LPO driven by a timer

Description	This command allows you to program an automatic change on LPO output level when a timer is triggered. This action remains memorized when the power is off. (LPO value 2 is used for automatic flashing).
Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status> BOARD <Board Number> TIMER <Timer Number> ACT LPO <LPO Number > <LPO Level>`
Response	`No response`
Example	To automatically switch the level of GPIO 3 of board 5 to level 1 when the TIMER 2 of board 2 is triggered **`PROG 5 AUT Test AUTON BOARD 2 TIMER 2 ACT LPO 3 1`**
Detail	**`PROG 5`** `=> programming board number 5` **`AUT Test AUTON BOARD 2 TIMER 2`** `=> when TIMER 2 of board 2 is triggered. This action is active in manual mode` **`ACT LPO 3 1`** `=> LPO 3 of board 5 is set to 1`

Program an automatic action on a track driven by a timer

Description	This command allows you to program an automatic change of track speed and direction when a timer is triggered. This action remains memorized when the power is off
Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status> BOARD <Board Number> TIMER <TIMER Number> ACT TRACK <Track Number> SPEED <Track Speed> <Track Dir> INERTIA <Inertia>`
Response	`No response`
Example	To slowly change speed and direction of track 2 of board 5 to speed 10 and travel forward when the TIMER 2 of board 2 is triggered **`PROG 5 AUT Test1 AUTOFF BOARD 2 TIMER 2 ACT TRACK 2 SPEED 10 FORW INERTIA 15`**
Detail	**`PROG 5`** `=> programming board number 5` **`AUT Test1 AUTOFF BOARD 2 TIMER 2`** `=> when TIMER 2 of board 2 is triggered. This action is not active in manual mode` **`ACT TRACK 2 SPEED 10 FORW INERTIA 15`** `=> set track 2 for board 5 at speed 10 forward, 15 steps inertia to change speed`
How	**Speed and/or Inertia value(s) could be replaced by KNOB0 or KNOB1, in this case the value of the knob is read when the action is performed**

Program on a timer driven by a timer

Description	This command allows you to program a timer when a timer (the same or another) is triggered. This action remains memorized when the power is off
Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status> BOARD <Board Number> TIMER <TIMER Number> ACT TIMER <TIMER Number> <TIMER delay>`
Response	`No response`
Example	To set TIMER 3 on board 5 with a delay of 100 when the TIMER 2 of board 2 is triggered **`PROG 5 AUT Test AUTOFF BOARD 2 TIMER 2 ACT TIMER 3 100`**
Detail	**`PROG 5`** `=> programming board number 5` **`AUT Test AUTOFF BOARD 2 TIMER 2`** `=> when TIMER 2 of board 2 is triggered. This action is not active in manual mode` **`ACT TIMER 3 100`** `=> TIMER 3 is set up at value 100`

Turn On or Off an automation driven by a timer

Description	This command allows you to turn on or off an automation when a timer (the same or another) is triggered. This action remains memorized when the power is off

Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status> BOARD <Board Number> TIMER <TIMER Number> ACT AUTOFF <Identifier>`
	
	`PROG <Board Number> AUT <Identifier> BOARD <Board Number> TIMER <TIMER Number> ACT AUTON <Identifier>`

Response	`No response`
Example	To turn on automation Test1 on board 5 when the TIMER 2 of board 2 is triggered **`PROG 5 AUT Test AUTOFF BOARD 2 TIMER 2 ACT AUTON Test1`**
Detail	**`PROG 5`** `=> programming board number 5` **`AUT Test AUTOFF BOARD 2 TIMER 2`** `=> when TIMER 2 of board 2 is triggered. This action is not active in manual mode` **`ACT AUTON Test1`** `=> turn on Test1`

Program an automatic action on a GPIO driven by a change of level on a GPIO

Description	This command allows you to program an automatic change on GPIO output level when the level of a GPIO input has changed on a board. This action remains memorized when the power is off
Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status> BOARD <Board Number> GPIO <GPIO Number> <GPIO Level> ACT GPIO <GPIO Number> <GPIO Level>`
Response	`No response`
Example	To automatically switch the level of GPIO 3 of board 5 to level 1 when the GPIO 2 of board 2 changes to level 0 **`PROG 5 AUT Test AUTOFF BOARD 2 GPIO 2 0 ACT GPIO 3 VAL 1`**
Detail	**`PROG 5`** `=> programming board number 5` **`AUT Test AUTOFF BOARD 2 GPIO 2 0`** `=> when GPIO 2 of board 2 is set to 0. This action is not active in manual mode` **`ACT GPIO 3 1`** `=> GPIO 3 of board 5 is set to 1`

Program an automatic action on a LPO driven by a change of level on a GPIO

Description	This command allows you to program an automatic change on LPO output level when the level of a GPIO input has changed on a board. This action remains memorized when the power is off. (LPO value 2 is used for automatic flashing).

Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status>`
	`BOARD <Board Number> GPIO <GPIO Number> <GPIO Level>`
	`ACT LPO <LPO Number> VAL <LPO Level>`

Response	`No response`
Example	To automatically switch the level of LPO 3 of board 5 to level 1 when the GPIO 1 of board 2 changes to level 0 **`PROG 5 AUT Test AUTOFF BOARD 2 GPIO 1 0 ACT LPO 3 VAL 1`**
Detail	**`PROG 5`** `=> programming board number 5` **`AUT Test AUTOFF BOARD 2 GPIO 1 VAL 0`** `=> when GPIO 1 of board 2 is set to 0. This action is not active in manual mode` **`ACT LPO 3 1`** `=> LPO 3 of board 5 is set to 1`

Program an automatic action on a track driven by a level change on a GPIO

Description	This command allows you to program an automatic change of track speed and direction when the level of a GPIO input has changed on a board. This action remains memorized when the power is off
Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status> BOARD <Board Number> GPIO <GPIO Number> <GPIO Level> ACT TRACK <Track Number> SPEED <Track Speed> <Track Dir> INERTIA <Inertia>`
Response	`No response`
Example	To quickly change speed and direction of track 2 of board 5 to speed 10 and travel forward when the GPIO 1 of board 2 changes to level 0 and reverse when the GPIO 1 of board 2 changes to level 1 **`PROG 5 AUT Test1 AUTOFF BOARD 2 GPIO 1 0 ACT TRACK 2 SPEED 10 FORW INERTIA 0`** **`PROG 5 AUT Test2 AUTOFF BOARD 2 GPIO 1 1 ACT TRACK 2 SPEED 10 BACK INERTIA 0`**
Detail	**`PROG 5`** `=> programming board number 5` **`AUT Test1 AUTOFF BOARD 2 GPIO 1 0`** `=> Automation Test1 when GPIO 1 of board 2 is set to 0. Not active in manual mode` **`AUT Test2 AUTOFF BOARD 2 GPIO 1 1`** `=> Automation Test2 when GPIO 1 of board 2 is set to 1. Not active in manual mode` **`ACT TRACK 2 SPEED 10 FORW INERTIA 0`** `=> set track 2 for board 5 at speed 10 forward, immediate change` **`ACT TRACK 2 SPEED 10 BACK INERTIA 0`** `=> set track 2 for board 5 at speed 10 backward, immediate change`

Program an automatic action on a loco (DCC) driven by a level change on a GPIO

Description	This command allows you to program an automatic change of loco setting when the level of a GPIO input has changed on a board. This action remains memorized when the power is off
Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status> BOARD <Board Number> GPIO <GPIO Number> <GPIO Level> ACT DCC <DCC Standard Command>`
Response	`No response`
Example	To speed up loco 3 at speed "0xF" forward on board 8 when the GPIO 1 of board 2 changes to level 0 **`PROG 8 AUT Test AUTOFF BOARD 2 GPIO 1 0 ACT DCC 0x03 0x6F`**
Detail	**`PROG 8`** `=> programming board number 8` **`AUT Test AUTOFF BOARD 2 GPIO 1 0`** `=> Automation Test when GPIO 1 of board 2 is set to 0. Not active in manual mode.` **`ACT DCC 0x03 0x6F`** `=> to speed up loco "0x3" at speed "0xF" forward`

Program on a timer driven by a level change on a GPIO

Description	This command allows you to program a timer when the level of a GPIO input has changed on a board. This action remains memorized when the power is off

Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status>`
	`BOARD <Board Number> GPIO <GPIO Number> <GPIO Level>`
	`ACT TIMER <TIMER Number> <TIMER delay>`

Response	`No response`
Example	To set TIMER 3 on board 5 with a delay of 100 when the GPIO 1 of board 2 changes to level 0 **`PROG 5 AUT Test1 AUTOFF BOARD 2 GPIO 1 0 ACT TIMER 3 VAL 100`**
Detail	**`PROG 5`** `=> programming board number 5` **`AUT Test1 AUTOFF BOARD 2 GPIO 1 0`** `=> Automation Test1 when GPIO 1 of board 2 is set to 0. Not active in manual mode.` **`ACT TIMER 3 100`** `=> TIMER 3 is set up at value 100`

Turn on or off an automation by a level change on a GPIO

Description	This command allows you to turn on or off an automation when the level of a GPIO input has changed on a board. This action remains memorized when the power is off

Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status> BOARD <Board Number> GPIO <GPIO Number> <GPIO Level> ACT AUTOFF <Identifier>`
	
	`PROG <Board Number> AUT <Identifier> <Manual Status> BOARD <Board Number> GPIO <GPIO Number> <GPIO Level> ACT AUTON <Identifier>`

Response	`No response`
Example	To turn off Test2 when the GPIO 1 of board 2 changes to level 0 **`PROG 5 AUT Test1 AUTOFF BOARD 2 GPIO 1 0 ACT AUTOFF Test2`**
Detail	**`PROG 5`** `=> programming board number 5` **`AUT Test1 AUTOFF BOARD 2 GPIO 1 0`** `=> Automation Test1 when GPIO 1 of board 2 is set to 0. Not active in manual mode` **`ACT AUTOFF Test2`** `=> turn off Test2`

Program an automatic action on a GPIO driven by a change of vehicle presence on a track

Description	This command allows you to program an automatic change on GPIO output when the presence of a vehicle has changed on a track. This action remains memorized when the power is off
Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status> BOARD <Board Number> TRACK <Track Number> STA <Vehicle Status> ACT GPIO <GPIO Number> <GPIO Level>`
Response	`No response`
Example	To switch the level of GPIO 3 of board 5 to 1 when a vehicle is on track 1 of board 2 and to 0 when a vehicle is no more on track 1 of board 2 **`PROG 5 AUT Test1 AUTOFF BOARD 2 TRACK 1 STA ONTRACK ACT GPIO 3 1`** **`PROG 5 AUT Test2 AUTOFF BOARD 2 TRACK 1 STA OFFTRACK ACT GPIO 3 0`**
Detail	**`PROG 5`** `=> programming board number 5` **`AUT Test1 AUTOFF BOARD 2 TRACK 1 STA ONTRACK`** `=> Automation Test1 when a vehicle is on track 1 of board 2. Not active in manual mode` **`AUT Test2 AUTOFF BOARD 2 TRACK 1 STA OFFTRACK`** `=> Automation Test2 when a vehicle is no more on track 1 of board 2. Not active in manual mode` **`ACT GPIO 3 1`** `=> GPIO 3 of board 5 is set to 1`

Program an automatic action on a LPO driven by a change of vehicle presence on a track

Description	This command allows you to program an automatic change on LPO output when the presence of a vehicle has changed on a track. This action remains memorized when the power is off. (LPO value 2 is used for automatic flashing).
Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status> BOARD <Board Number> TRACK <Track Number> STA <Vehicle Status> ACT LPO <LPO Number> <LPO Level>`
Response	`No response`
Example	To switch the level of LPO 3 of board 5 to 1 when a vehicle is on track 1 of board 2 and to 0 when a vehicle is no more on track 1 of board 2 **`PROG 5 Test1 AUTOFF AUT BOARD 2 TRACK 1 STA ONTRACK ACT LPO 3 1`** **`PROG 5 Test2 AUTOFF AUT BOARD 2 TRACK 1 STA OFFTRACK ACT LPO 3 0`**
Detail	**`PROG 5`** `=> programming board number 5` **`AUT Test1 AUTOFF BOARD 2 TRACK 1 STA ONTRACK`** `=> Automation Test1 when a vehicle is on track 1 of board 2. Not active in manual mode` **`AUT Test2 AUTOFF BOARD 2 TRACK 1 STA OFFTRACK`** `=> Automation Test2 when a vehicle is no more on track 1 of board 2. Not active in manual mode` **`ACT LPO 3 1`** `=> LPO 3 of board 5 is set to 1` **`ACT LPO 3 0`** `=> LPO 3 of board 5 is set to 0`

Program an automatic action on a track driven by a change of vehicle presence on a track

Description	This command allows you to program an automatic change of track, speed and direction when the presence of a vehicle has changed on a track. This action remains memorized when the power is off
Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status> BOARD <Board Number> TRACK <Track Number> STA <Vehicle Status> ACT TRACK <Track Number> SPEED <Track Speed> <Track Direction> INERTIA <Inertia>`
Response	`No response`
Example	To change the speed and direction of track 3 to 10 and forward when a vehicle is on track 1 of board 2 **`PROG 5 AUT Test AUTOFF BOARD 2 TRACK 1 STA ONTRACK ACT TRACK 3 SPEED 10 DIR FORW INERTIA 5`**
Detail	**`PROG 5`** `=> programming board number 5` **`AUT Test AUTOFF BOARD 2 TRACK 1 STA ONTRACK`** `=> Automation Test when a vehicle is on track 1 of board 2. Not active in manual mode` **`ACT TRACK 3 SPEED 10 FORW INERTIA 5=>`** `set speed of track 3 to 10 forward, 5 steps inertia to change speed`
How	**Speed and/or Inertia value(s) could be replaced by KNOB0 or KNOB1, in this case the value of the knob is read when the action is performed**

Program an automatic action on a loco (DCC) by a change of vehicle presence on a track

Description	This command allows you to program an automatic change of loco setting when the presence of a vehicle has changed on a track. This action remains memorized when the power is off
Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status> BOARD <Board Number> TRACK <Track Number> STA <Vehicle Status> ACT DCC <DCC Standard Command>`
Response	`No response`
Example	To speed up loco 3 at speed "0xF" forward on board 8 when a vehicle is on track 1 of board 2 **`PROG 8 AUT Test AUTOFF BOARD 2 TRACK 1 STA ONTRACK ACT DCC 0x03 0x6F`**
Detail	**`PROG 8`** `=> programming board number 8` **`AUT Test AUTOFF BOARD 2 TRACK 1 STA ONTRACK`** `=> Automation Test when a vehicle is on track 1 of board 2. Not active in manual mode` **`ACT DCC 0x03 0x6F`** `=> to speed up loco "0x3" at speed "0xF" forward`

Program on a timer driven by a change of vehicle presence on a track

Description	This command allows you to program a timer when the presence of a vehicle has changed on a track. This action remains memorized when the power is off
Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status> BOARD <Board Number> TRACK <Track Number> STA <Vehicle Status> ACT TIMER <TIMER Number> <TIMER delay>`
Response	`No response`
Example	To set TIMER 3 on board 5 with a delay of 100 when a vehicle is on track 1 of board 2 **`PROG 8 AUT Test AUTOFF BOARD 2 TRACK 1 STA ONTRACK ACT TIMER 3 100`**
Detail	**`PROG 8`** `=> programming board number 8` **`AUT Test AUTOFF BOARD 2 TRACK 1 STA ONTRACK`** `=> Automation Test when a vehicle is on track 1 of board 2. Not active in manual mode` **`ACT TIMER 3 100`** `=> TIMER 3 is set up at value 100`

Turn on or off an automation by a change of vehicle presence on a track

Description	This command allows you to turn on or off an automation when the presence of a vehicle has changed on a track. This action remains memorized when the power is off

Syntax	`PROG <Board Number> AUT <Identifier> <Manual Status> BOARD <Board Number> TRACK <Track Number> STA <Vehicle Status> ACT AUTOFF <Identifier>`
	
	`PROG <Board Number> AUT <Identifier> <Manual Status> BOARD <Board Number> TRACK <Track Number> STA <Vehicle Status> ACT AUTON <Identifier>`

Response	`No response`
Example	To turn off Test1 when a vehicle is on track 1 of board 2 **`PROG 8 AUT Test AUTOFF BOARD 2 TRACK 1 STA ONTRACK ACT AUTOFF Test1`**
Detail	**`PROG 8`** `=> programming board number 8` **`AUT Test AUTOFF BOARD 2 TRACK 1 STA ONTRACK`** `=> Automation Test when a vehicle is on track 1 of board 2. Not active in manual mode` **`ACT AUTOFF Test1`** `=> Turn off Test1`

Delete an automation on a board

Description	This command allows you to remove an automation on a board. This action remains memorized when the power is off
Syntax	`PROG <Board Number> DEL <Number>`
Response	`No response`
Example	To remove automation 3 on board 5 (List of automation could be retrieve using the Automation list command COM <Board Number> AUTLIST) **`PROG 5 LED 3`**
Warning	Deleting an automation changes the number of the other automations, so you need to perform once again an AUTLIST command to know the new numbers

Force the value on a GPIO on a board

Description	This command allows you to set the level of a GPIO. This action remains valid until the board is stopped or an automatic action is triggered. When GPIO is in input mode, this command reset the counter when set to 0
Syntax	`COM <Board Number> GPIO <GPIO Number> <GPIO Level>`
Response if GPIO in output mode	`No response`
Example	To set GPIO 3 at level 1 on board 4 **`COM 4 GPIO 3 1`**

Create a timer on a board

Description	This command allows you to set a timer. This action remains valid until the board is stopped or the timer is triggered
Syntax	`COM <Board Number> TIMER <Timer Number> <Timer delay>`
Response if GPIO in output mode	`No response`
Example	To set TIMER 3 with a delay of 10 on board 4 **`COM 4 TIMER 3 10`**

Force the value on a LPO on a board

Description	This command allows you to set the level of a LPO. This action remains valid until the board is stopped or an automatic action is triggered. (LPO value 2 is used for automatic flashing).
Syntax	`COM <Board Number> LPO <LPO Number> <LPO Level>`
Response	`No response`
Example	To set LPO 3 at level 1 on board 4 **`COM 4 LPO 3 1`**

Turn on an automation on a board

Description	This command allows you to turn on an automation. This action remains valid until the board is stopped or an automatic action is triggered
Syntax	`COM <Board Number> AUTON <Identifier>`
Response	`No response`
Example	To turn on automation Test On board 4 **`COM 4 AUTON Test`**

Turn off an automation on a board

Description	This command allows you to turn off an automation. This action remains valid until the board is stopped or an automatic action is triggered
Syntax	`COM <Board Number> AUTOFF <Identifier>`
Response	`No response`
Example	To turn off automation Test On board 4 **`COM 4 AUTOFF Test`**

Force speed and direction on a track

Description	This command allows you to set the speed and direction on a track. This action remains valid until the board is stopped or an automatic action is triggered
Syntax	`COM <Board Number> TRACK <Track Number> SPEED <Track Speed> <Dir> INERTIA <Inertia>`
Response	`No response`
Response if board in DCC mode	`Board <Board Number> Bad mode`
Example	To set speed at 4 and direction forward on track 2 board 8 **`COM 8 TRACK 2 SPEED 4 FORW INERTIA 2`**
How	**Speed and/or Inertia value(s) could be replaced by KNOB0 or KNOB1, in this case the value of the knob is read when the action is performed**

Send a DCC command to a board

Description	This command allows you to send a DCC NMRA command to a board (all tracks). This action remains valid until the board is stopped or an automatic action is triggered
Syntax	`COM <Board Number> DCC <DCC Stand Command>`
Response	`No response`
Response if board in ANA mode	`Board <Board Number> Bad mode`
Example	To send a command to speed up loco 3 at speed "0xF" forward on board 8 **`COM 8 DCC 0x03 0x6F`** To send a command to speed up loco 3 at speed "0xF" backward on board 8 **`COM 8 DCC 0x03 0x4F`**

Request the status of all the GPIOs on a board

Description	This command allows you to get the direction and the level of all the GPIOs on a board
Syntax	`COM <Board Number> GSTAT`

Response	`Board <Board Number> GPIO 0 <GPIO Dir> VAL <GPIO Level> COUNT <GPIO Counter>`
	`Board <Board Number> GPIO 1 <GPIO Dir> VAL <GPIO Level> COUNT <GPIO Counter>`
	`Board <Board Number> GPIO 2 <GPIO Dir> VAL <GPIO Level> COUNT <GPIO Counter>`
	`Board <Board Number> GPIO 3 <GPIO Dir> VAL <GPIO Level> COUNT <GPIO Counter>`
	`Board <Board Number> KNOB0 VAL <knob value>`
	`Board <Board Number> KNOB1 VAL <knob value>`

Example	To get the status of all the GPIO of board 5 **`COM 5 GSTAT`**

Request the status of all the LPOs on a board

Description	This command allows you to get the direction and the level of all the LPOs on a board
Syntax	`COM <Board Number> LSTAT`

Response	`Board <Board Number> LPO 0 VAL <LPO Level>`
	`Board <Board Number> LPO 1 VAL <LPO Level>`
	`Board <Board Number> LPO 2 VAL <LPO Level>`
	`Board <Board Number> LPO 3 VAL <LPO Level>`
	`Board <Board Number> LPO 4 VAL <LPO Level>`
	`Board <Board Number> LPO 5 VAL <LPO Level>`

Example	To get the status of all the LPO of board 5 **`COM 5 LSTAT`**

Request track status on a board

Description	This command allows you to get the direction and the speed and direction of all the tracks on a board if the board is in ANA mode
Syntax	`COM <Board Number> TSTAT`

Response	`Board <Board Number> TRACK 0 SPEED <Track Speed> <Track Dir> <vehicle status>`
	`Board <Board Number> TRACK 1 SPEED <Track Speed> <Track Dir> <vehicle status>`
	`Board <Board Number> TRACK 2 SPEED <Track Speed> <Track Dir> <vehicle status>`
	`Board <Board Number> TRACK 3 SPEED <Track Speed> <Track Dir> <vehicle status>`

Example	To get the status of all the tracks of board 5 **`COM 5 TSTAT`**

Request board status

Description	This command allows you to get the status of a board to know if the running mode is DCC or ANA
Syntax	`COM <Board Number> BSTAT`
Response if DCC mode	`Board <Board Number> DCC`
Response if ANA mode	`Board <Board Number> ANA`
Example	To get the status of the board 5 **`COM 5 BSTAT`**

Request the list of actions programmed on a board

Description	This command allows you to get all the Automations stored in a board
Syntax	`COM <Board Number> AUTLIST`
Response if no data	`No response`
Response for each automation	`Board <Board Number> <Number> <Identifier> ON` `Board <Board Number> <Number> <Identifier> OFF`
Example	To get the list of actions programmed on board 5 **`COM 5 AUTLIST`**

Request a dump of memory on a board

Description	This command allows you to get the content of Automations stored in a board in binary mode (reserved for PC application)
Syntax	`COM <Board Number> DUMP`
Response if no data	`No response`
Response for each automation	**`Board <Board Number> <Number> (<Number>,<Index>,<Data>)...`**
Example	To get the dump of board 5 **`COM 5 DUMP`**

Request a calibration of knobs

Description	This command is used to save the min and max values of knobs 0 and 1. To do this, turn both knobs to the min value and then to the max value and execute this command.
Syntax	`CALIB <Board Number>`
Response	`No response`
Example	To calibrate knobs of board 5 **`CALIB 5`**

6.1 Global command

Stop all

Description	This command will stop the activity on all the boards of the network
Syntax	`STOP`
Response	`No response`
Example	To stop all **`STOP`**

Run all

Description	This command will start the activity on all the boards of the network
Syntax	`RUNALL`
Response	`No response`
Example	To run all **`RUNALL`**

Run a specific board

Description	This command will start the activity on a specific board
Syntax	`RUN <Board Number>`
Response	`No response`
Example	To run board 5 **`RUN 5`**

Reset all automation of a specific board

Description	This command will delete all the automation on a specific board. **Warning this action is performed immediately without any notification!**

Syntax	`RESET <Board Number>`
Response	`No response`
Example	To reset board 5 **`RESET 5`**

Inconsistent programming

If an impossible automatic action is triggered, such as setting the speed and direction on the track of a board in DCC mode, or configuring a locomotive on a map in ANA mode, the action is simply ignored and a message is sent to the serial console:

- Unknown token
- Number missing
- Incomplete request
- Bad number
- Mode is missing
- Bad GPIO number
- Bad TIMER number
- Bad TIMER value
- Bad LPO number
- Bad Automation status
- Bad Automation name
- Automation name missing
- Bad GPIO direction
- Bad GPIO level
- Bad Low Power Output level
- Bad track speed
- Bad track direction
- Bad track number
- Bad fashion
- Wrong board number
- No more automation available
- Space missing
- Identifier is too long
- Automation already defined
- Wrong automation number
- Bad inertia value
- Bad user mode

7 Barraux's railway network (31/07/2025)

General System Structure

- The system uses several boards (from board 0 to board 3, plus a special board "MASTER 31").
- Each board controls:
  - Up to 4 tracks (tracks A0 to D3 for board 31, E0 to H3 for board 0, etc.)
  - 6 low-power outputs (LPO): for LEDs, lights, switch controls, etc.
  - 4 GPIO inputs: for buttons, optical sensors, contacts, etc.
  - 4 "Track Status" inputs: for 7-segment displays, speed and inertia potentiometers.

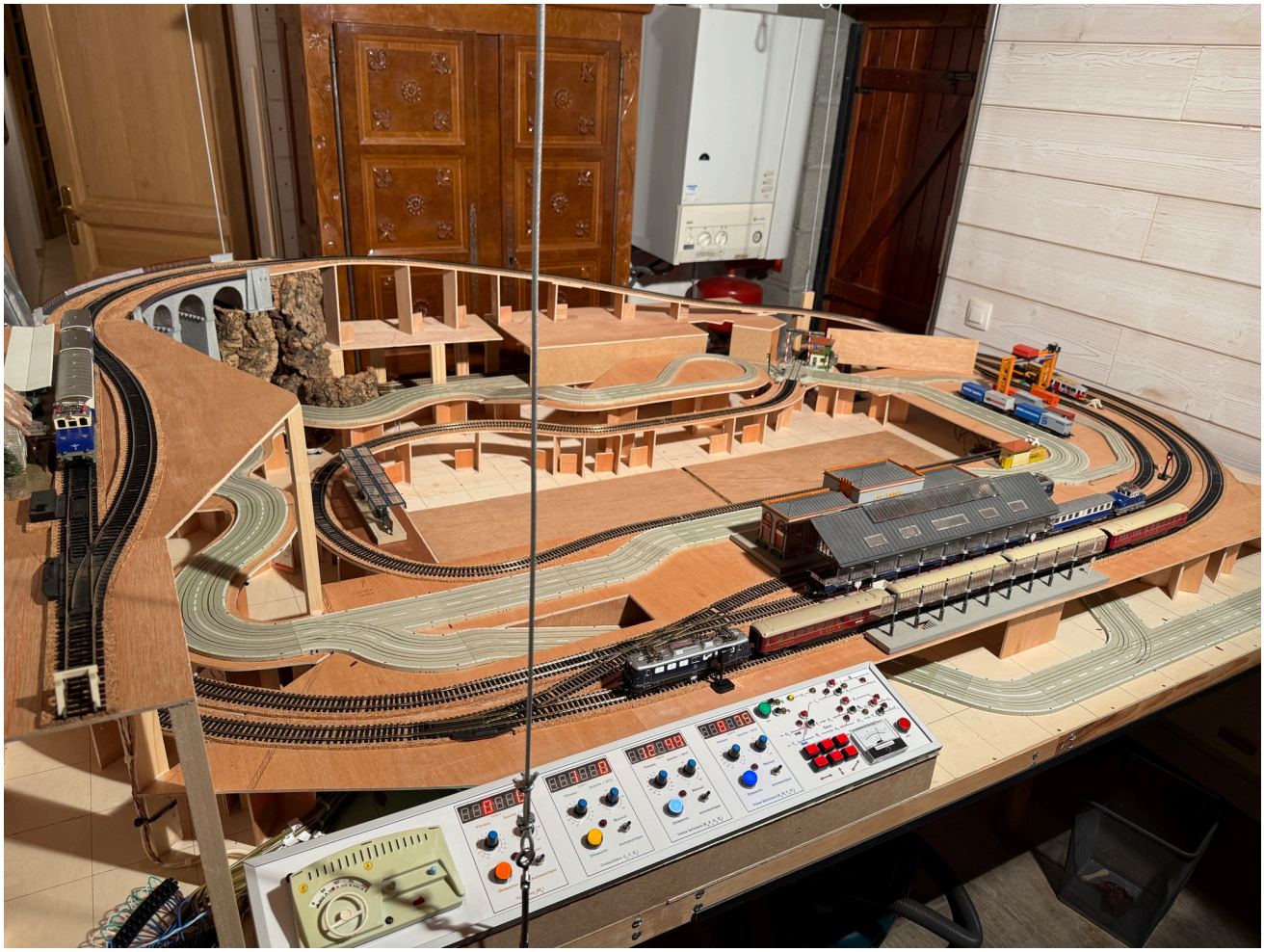


Figure 19 Railway network (31/07/2025)

Operating Modes

Automatic Mode

Commands like:

```
PROG <board> AUT <ID> AUTON/AUTOFF BOARD <n°> ... ACT ...
```

- Activates complex sequences with timing (TIMER), train detection (STA ONTRACK), rail actions (TRACK ... SPEED ...), LEDs (LPO ...), and I/O.
- Examples:
  - Launching a train based on a timer.
  - Controlling traffic lights (green/red) as trains pass.
  - Specific sequences for uncoupling wagons, level crossings, reversing locomotives, etc.

⏸️ Manual Mode

Commands like:

```
PROG <board> AUT M<ID> AUTON BOARD <n°> ... ACT ...
```

- GPIO resets.
- Stops tracks (speed 0, inertia 0).
- Turns indicator LEDs on/off (e.g., start button LED, manual mode indicator).

🧠 Automation per Board

🟩 MASTER 31 Board

- Controls tracks A0 to D3.
- Coordinates all other boards.
- Includes an optical control panel (TCO).
- Automations: loop sequences with station wait times, signal light control, train restart, synchronized timers (TIMER 1, TIMER 2, etc.).

🟦 Board 0

- Controls tracks E0 to H3.
- Includes logic for uncoupling and coordination with board 1 (rack train).
- Advanced management of two trains coexisting on internal tracks.

🟨 Board 1

- Controls tracks I0 to L3.
- Detailed sequence for fully automatic locomotive reversal.
- Uses six timers (TIMER 1 to TIMER 6).
- Tightly coupled with boards 0 and 2.

🟧 Board 2

- Controls one tramway track (M0) and two temporary Faller AMS tracks.

- Extensive commands for level crossing lights, optical sensors, and regular back-and-forth sequences.
- Manages TCO LEDs to display tramway position.

🟥 Board 3

- Dedicated board for manual/automatic switch control.
- Examples: toggling between main and passing track, dual-press manual commands.

📌 TCO (Optical Control Panel) Commands

- Enable manual or automatic control of events like:
  - Triggering red lights.
  - Switches when a train arrives or departs.
  - Activating or stopping uncoupling actions.
- Prefixes:
  - `M<i>`: manual mode.
  - `A<i>`: automatic.
  - `T<i>`: TCO command.
  - `D<i>`: deactivatable (can be temporarily removed from execution logic).

📌 Programing the boards

A Qt application is available to send the programming code to the different boards via board 31. This application recognizes the boards' programming language, can check the syntax before sending the instructions, and includes a graphical visualization of the interconnections between the commands.

The Qt code is available in **RailManagerUI,** a customized version of the open-source project **Scribe**, described as:

A simple, user-friendly text editor.

GitHub Stars: https://github.com/AleksandrHovhannisyan/Scribe-Text-Editor/stargazers
Latest Release: https://github.com/AleksandrHovhannisyan/Scribe-Text-Editor/releases

This application also incorporates the **QtNodes** library, which has been modified to meet the specific needs of the project.

As of **July 31, 2025**, the application is still under development but already functional. It is configured for compilation on both **macOS** and **Windows** platforms.

Source Code and Project Access

The code, along with the entire development of this railway network, is available on the public GitHub repository:

https://github.com/PhilippeCavenel/RAIL-DRIVER

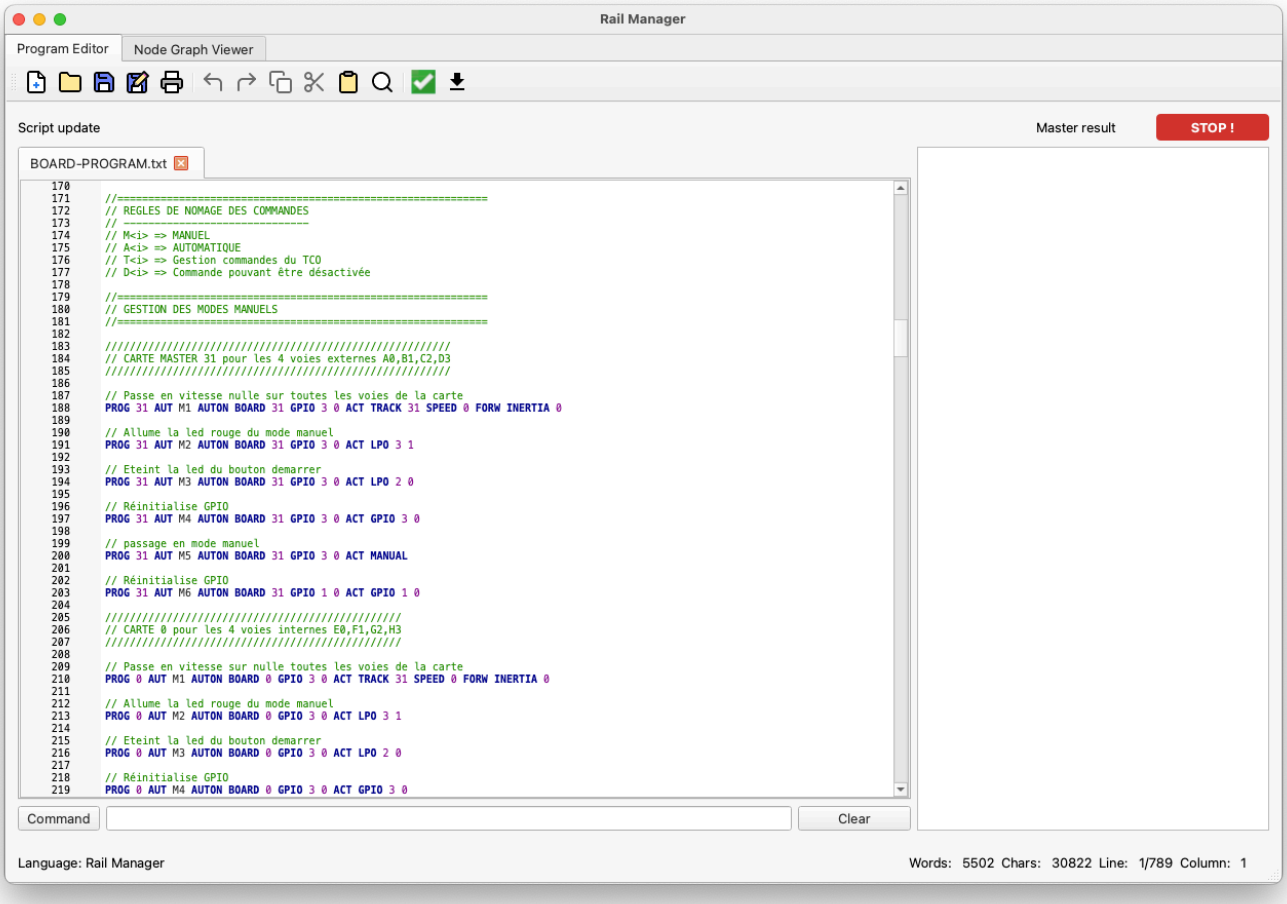


Figure 20 Rail Manager UI, sribe interface

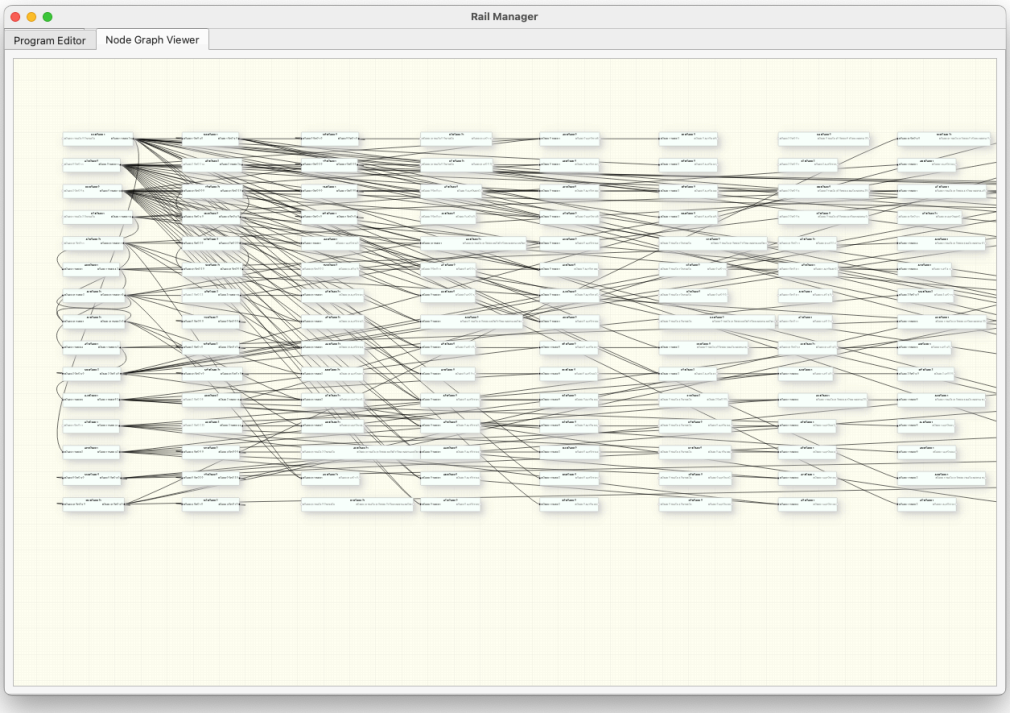


Figure 21 Rail Manager UI, node interface

By clicking on a node, you can easily view the interconnections.

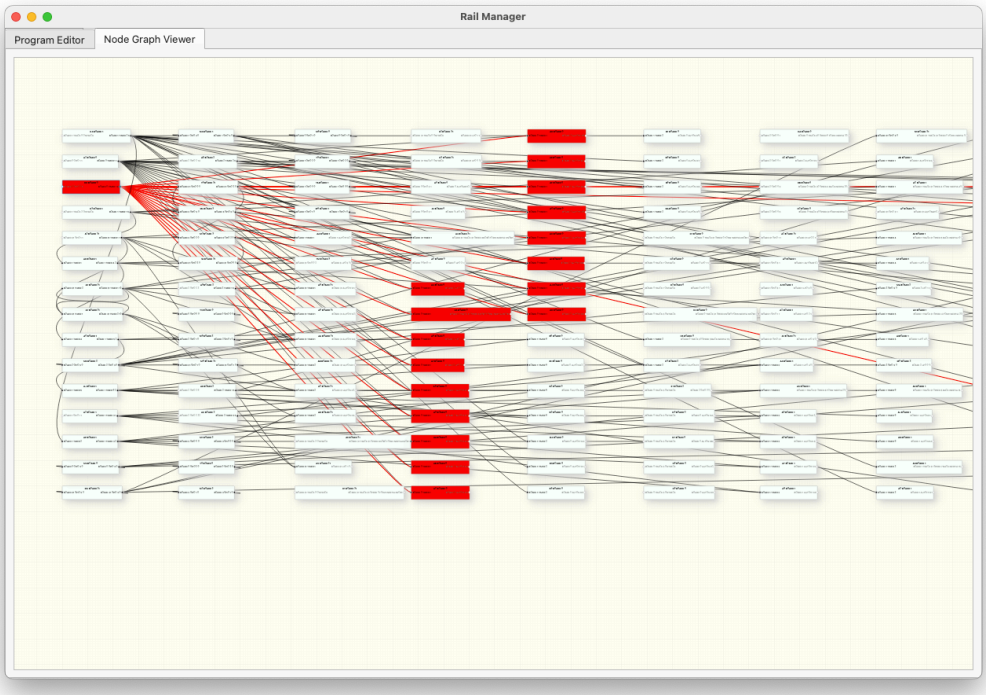


Figure 22 Rail Manager UI, node highlighted

Board Code (27/08/2025)

```
//=============================================
// VERSION DU 21 AOUT 2025
//=============================================
// Gestion des numeros de piste
// Entre 0 et 3, numéro de piste unique
// Entre 17 et 31 :
// 17 = 0
// 18 = 1
// 19 = 1 0
// 20 = 2
// 21 = 2 0
// 22 = 2 1
// 23 = 2 1 0
// 24 = 3
// 25 = 3 0
// 26 = 3 1
// 27 = 3 1 0
// 28 = 3 2
// 29 = 3 2 0
// 30 = 3 2 1
// 31 = 3 2 1 0
//=============================================

//=============================================
// GESTION DES AUTOMATISMES DE LA CARTE MASTER 31
//=============================================

// Connect Rail 0Voie A0
// Connect Rail 1Voie B1
// Connect Rail 2Voie C2
// Connect Rail 3Voie D3

// LPO 0         Feu vert B1
// LPO 1         Feu rouge B1
// LPO 2         Led bouton bleu
// LPO 3         Led mode manuel
// LPO 4         décrochage wagon circuit bas
// LPO 5         non connecté

// GPIO input 0          Bouton aiguillage CROSS rouge
// GPIO input 1          Bouton bleu démarrer
// GPIO input 2          Bouton aiguillage CROSS noir
// GPIO input 3          Contact mode manuel

// Track Status  0       Afficheur 7 segments DIO
// Track Status  1       Afficheur 7 segments CLK
// Track Status  2       Potentiomètre inertie
// Track Status  3       Potentiomètre vitesse

//=============================================
// GESTION DES AUTOMATISMES DE LA CARTE 0
//=============================================

// Connect Rail 0Voie E0
// Connect Rail 1Voie F1
// Connect Rail 2Voie G2
// Connect Rail 3Voie H3

// LPO 0         Feu vert E0
// LPO 1         Feu rouge E0
// LPO 2         Led bouton blanc
// LPO 3         Led mode manuel
// LPO 4         décrochage gare crémaillère haut
// LPO 5         voie CROSS sur voie F (0) / voie CROSS non connecté

// GPIO input 0          Bouton aiguillage D3 / H3 et C2/ G2 rouge
// GPIO input 1          Bouton blanc démarrer
// GPIO input 2          Bouton aiguillage D3 / H3 et C2/ G2 noir
// GPIO input 3          Contact mode manuel

// Track Status  0       Afficheur 7 segments DIO
// Track Status  1       Afficheur 7 segments CLK
// Track Status  2       Potentiomètre inertie
// Track Status  3       Potentiomètre vitesse

//=============================================
// GESTION DES AUTOMATISMES DE LA CARTE 1
//=============================================

// Connect Rail 0Voie I0
// Connect Rail 1Voie J1
// Connect Rail 2Voie K2
```

```
// Connect Rail 3Voie L3
// LPO 0         aiguillage E0 / (E0) H1 et E0 / G2
// LPO 1         non connecté (relié à un relai)
// LPO 2         Led bouton jaune
// LPO 3         Led mode manuel
// LPO 4         aiguillage I0 / J1 , K2
// LPO 5         non connecté (relié à un relai)

// GPIO input 0          aiguillage E0 / (E0) H1 et E0 / G2
// GPIO input 1          Bouton Jaune
// GPIO input 2          aiguillage E0 / (E0) H1 et E0 / G2
// GPIO input 3          Contact mode manuel

// Track Status  0       Afficheur 7 segments DIO
// Track Status  1       Afficheur 7 segments CLK
// Track Status  2       Potentiomètre inertie
// Track Status  3       Potentiomètre vitesse


//================================================
// GESTION DES AUTOMATISMES DE LA CARTE 2
//================================================

// Connect Rail 0Voie M0
// Connect Rail 1Voie voiture 1 - temporaire en attendant d'avoir d'autres cartes
// Connect Rail 2Voie voiture 2 - temporaire en attendant d'avoir d'autres cartes
// Connect Rail 3Voie non connectée

// LPO 0         Ouverture barrière passage à niveau
// LPO 1         Fermeture barrière passage à niveau
// LPO 2         Led bouton orange
// LPO 3         Led mode manuel
// LPO 4         Led tramway devant grue
// LPO 5         Led tramway devant gare terminale

// GPIO input 0          Détecteur optique
// GPIO input 1          Bouton orange
// GPIO input 2          Détecton passage tramway devant capteur magnétique
// GPIO input 3          Contact mode manuel

// Track Status  0       Afficheur 7 segments DIO
// Track Status  1       Afficheur 7 segments CLK
// Track Status  2       Potentiomètre inertie
// Track Status  3       Potentiomètre vitesse

//================================================
// GESTION DES AUTOMATISMES DE LA CARTE 3
//================================================

// Connect Rail 0Voie non connectée
// Connect Rail 1Voie non connectée
// Connect Rail 2Voie non connectée
// Connect Rail 3Voie non connectée

// LPO 0         aiguillage CROSS noir
// LPO 1         aiguillage CROSS rouge
// LPO 2         aiguillage B1 / CROSS (noir cmd manuel double appui)
// LPO 3         aiguillage B1 / CROSS (rouge cmd manuel double appui)
// LPO 4         non connecté (relié à un relai)
// LPO 5         aiguillage D3 / H3 et C2 / G2

// GPIO input 0          aiguillage I0 / J1 , K2
// GPIO input 1          aiguillage I0 / J1 , K2
// GPIO input 2          Commande décrochage wagon gare crémaillère haut
// GPIO input 3          Commande décrochage wagon circuit bas

// Track Status  0       non connecté
// Track Status  1       non connecté
// Track Status  2       non connecté
// Track Status  3       non connecté


//============================================================
// PRINCIPES GENERAUX DE FONCTIONNEMENT POUR TOUTES LES CARTES
//============================================================

// Passage en mode manuel
//       - passe en vitesse nulle toutes les voies de la carte jusqu'à modification d'une valeur de vitesse ou
d'internie
//       - allume la led rouge du mode manuel
//       - Eteint la led du bouton demarrer si elle était allumée
//       - Réinitialise tous les TIMERs
//       - Passage en mode manuel et désactivation de toutes les commandes de type AUTOFF

// Passage en mode automatique
//        - Eteint la led rouge du bouton mode manuel
```

```
//        - Passage en mode automatique
//        - L'appui sur le bouton démarrer
//                - Allume la led du bouton démarrer en mode clignotement
//                - Lance la séquence automatique programmée

//============================================================
// REGLES DE NOMAGE DES COMMANDES
// ------------------------------
// M<i> => MANUEL
// A<i> => AUTOMATIQUE
// T<i> => Gestion commandes du TCO
// D<i> => Commande pouvant être désactivée (via AUTOFF spécifique non lié au mode AUTOMATIC ou MANUAL)

// Les commandes en mode AUTON sont toujours actives en mode MANUAL et AUTOMATIC sauf si elles ont été désactivées
par une commande spécifique AUTOFF
// Les commandes en mode AUTOFF sont désactivées en mode MANUAL
// Au démarrage des cartes, toutes les commandes AUTOFF sont désactivées, il faut donc spécifier au moins une fois
un passage en mode AUTOMATIC

//============================================================
// ARRET COMPLET DES AUTOMATISMES
//============================================================

STOPALL

//============================================================
// GESTION DES MODES MANUELS
//============================================================

////////////////////////////////////////////////////
// CARTE MASTER 31 pour les 4 voies externes A0,B1,C2,D3
////////////////////////////////////////////////////

// Passe en vitesse nulle sur toutes les voies de la carte
PROG 31 AUT M1 AUTON BOARD 31 GPIO 3 0 ACT TRACK 31 SPEED 0 FORW INERTIA 0

// Allume la led rouge du mode manuel
PROG 31 AUT M2 AUTON BOARD 31 GPIO 3 0 ACT LPO 3 1

// Eteint la led du bouton demarrer
PROG 31 AUT M3 AUTON BOARD 31 GPIO 3 0 ACT LPO 2 0

// Réinitialise GPIO
PROG 31 AUT M4 AUTON BOARD 31 GPIO 3 0 ACT GPIO 3 0

// passage en mode manuel
PROG 31 AUT M5 AUTON BOARD 31 GPIO 3 0 ACT MANUAL

// Réinitialise GPIO
PROG 31 AUT M6 AUTON BOARD 31 GPIO 1 0 ACT GPIO 1 0

////////////////////////////////////////////////
// CARTE 0 pour les 4 voies internes E0,F1,G2,H3
////////////////////////////////////////////////

// Passe en vitesse nulle toutes les voies de la carte
PROG 0 AUT M1 AUTON BOARD 0 GPIO 3 0 ACT TRACK 31 SPEED 0 FORW INERTIA 0

// Allume la led rouge du mode manuel
PROG 0 AUT M2 AUTON BOARD 0 GPIO 3 0 ACT LPO 3 1

// Eteint la led du bouton demarrer
PROG 0 AUT M3 AUTON BOARD 0 GPIO 3 0 ACT LPO 2 0

// Réinitialise GPIO
PROG 0 AUT M4 AUTON BOARD 0 GPIO 3 0 ACT GPIO 3 0

// passage en mode manuel
PROG 0 AUT M5 AUTON BOARD 0 GPIO 3 0 ACT MANUAL

// Réinitialise GPIO
PROG 0 AUT M6 AUTON BOARD 0 GPIO 1 0 ACT GPIO 1 0

////////////////////////////////////////////////////////
// CARTE 1 pour les 4 voies de la crémaillère I0,J1,K2,L3
////////////////////////////////////////////////////////

// Passe en vitesse nulle toutes les voies de la carte
PROG 1 AUT M1 AUTON BOARD 1 GPIO 3 0 ACT TRACK 31 SPEED 0 FORW INERTIA 0

// Allume la led rouge du mode manuel
PROG 1 AUT M2 AUTON BOARD 1 GPIO 3 0 ACT LPO 3 1

// Eteint la led du bouton demarrer
PROG 1 AUT M3 AUTON BOARD 1 GPIO 3 0 ACT LPO 2 0
```

```
// Réinitialise GPIO
PROG 1 AUT M4 AUTON BOARD 1 GPIO 3 0 ACT GPIO 3 0

// Passage en mode manuel
PROG 1 AUT M5 AUTON BOARD 1 GPIO 3 0 ACT MANUAL

// Réinitialise GPIO
PROG 1 AUT M6 AUTON BOARD 1 GPIO 1 0 ACT GPIO 1 0

////////////////////////////////////////////////////////////////////
// CARTE 2 pour la voie du tramway et les 2 voies temporaire Faller AMS
////////////////////////////////////////////////////////////////////

// Passe en vitesse nulle toutes les voies de la carte
PROG 2 AUT M1 AUTON BOARD 2 GPIO 3 0 ACT TRACK 31 SPEED 0 FORW INERTIA 0

// Allume la led rouge du mode manuel
PROG 2 AUT M2 AUTON BOARD 2 GPIO 3 0 ACT LPO 3 1

// Eteint la led du bouton demarrer
PROG 2 AUT M3 AUTON BOARD 2 GPIO 3 0 ACT LPO 2 0

// Réinitialise GPIO
PROG 2 AUT M4 AUTON BOARD 2 GPIO 3 0 ACT GPIO 3 0

// Passage en mode manuel (voie 0)
PROG 2 AUT M5 AUTON BOARD 2 GPIO 3 0 ACT MANUAL0

// Réinitialise GPIO
PROG 2 AUT M6 AUTON BOARD 2 GPIO 1 0 ACT GPIO 1 0

// Eteint affichage des LEDs du TCO (position du Tramway sur la voie)
PROG 2 AUT M7 AUTON BOARD 2 GPIO 3 0 ACT LPO 4 0
PROG 2 AUT M8 AUTON BOARD 2 GPIO 3 0 ACT LPO 5 0

// Ferme les barrières
PROG 2 AUT M9 AUTON BOARD 2 GPIO 3 0 ACT LPO 0 0
PROG 2 AUT MA AUTON BOARD 2 GPIO 3 0 ACT LPO 1 1

//=========================================================
// GESTION DES MODES AUTOMATIQUES
//=========================================================

////////////////////////////////////////////////////////

// CARTE MASTER 31 pour les 4 voies externes A0,B1,C2,D3
// TIMER 1 => Temps d'attente en gare
// TIMER 2 => Temps de circulation sur les voies
////////////////////////////////////////////////////////

// Passage en mode automatique
PROG 31 AUT A0 AUTON BOARD 31 GPIO 3 1 ACT AUTOMATIC

// Eteint la led rouge du bouton mode manuel
PROG 31 AUT A1 AUTON BOARD 31 GPIO 3 1 ACT LPO 3 0

// Arme TIMER 1 pour lancer le cycle de démarrage sur les voies A0, B1, C2 et D3 par appui bouton
PROG 31 AUT A2 AUTOFF BOARD 31 GPIO 1 1 ACT TIMER 1 1

// Lance la séquence automatique programmée

// DESCRIPTION DE LA SEQUENCE
// --------------------------
// Sur appui bouton démarrer, le train va effectuer des tours pendant 10 unités de temps TIMER (environ 1mn20),
// puis s'arrêter en gare sur la même période avant de recommencer

// Allume la led du bouton démarrer (clignotement)
PROG 31 AUT A3 AUTOFF BOARD 31  GPIO 1 1  ACT LPO 2 2

// Active le démarrage du train voie A0, B1, C2 et D3 sur appui bouton démarrer
PROG 31 AUT A4 AUTOFF BOARD 31 GPIO 1 1 ACT AUTON D4

// Relance du train après passage du train à crémaillère
PROG 31 AUT A5 AUTOFF BOARD 31 GPIO 3 1 ACT AUTOFF D5
PROG 31 AUT A6 AUTOFF BOARD 31 GPIO 1 1 ACT AUTON D5

// Passage du feu au vert
PROG 31 AUT A7 AUTOFF BOARD 31 TIMER 1 ACT LPO 1 0
PROG 31 AUT A8 AUTOFF BOARD 31 TIMER 1 ACT LPO 0 1

// Arme TIMER 2 pour faire tourner le train pendant 10 unités
PROG 31 AUT A9 AUTOFF BOARD 31 TIMER 1 ACT TIMER 2 10

// Désactive l'arret en gare sur détection de train voie A0
PROG 31 AUT AA AUTOFF BOARD 31 TIMER 1 ACT AUTOFF D1
```

```
//Désactive Le passage du feu au rouge sur détection de train voie A0
PROG 31 AUT AB AUTOFF BOARD 31 TIMER 1 ACT AUTOFF D2
PROG 31 AUT AC AUTOFF BOARD 31 TIMER 1 ACT AUTOFF D3

// Réactive l'arret en gare sur déclenchement du TIMER 2
PROG 31 AUT AD AUTOFF BOARD 31 TIMER 2 ACT AUTON D1

// Réactive le passage du feu au rouge sur détection de train voie A0
PROG 31 AUT AE AUTOFF BOARD 31 TIMER 2 ACT AUTON D2
PROG 31 AUT AF AUTOFF BOARD 31 TIMER 2 ACT AUTON D3

// Réarme le TIMER 1 pour une attente en gare de 10 unités
PROG 31 AUT AG AUTOFF BOARD 31 TIMER 2 ACT TIMER 1 10

// Commande pouvant être désactivée
// --------------------------------

// Arret en gare sur détection voie A0, les voies A0, B1, C2 et D3 sont progressivement arrêtées
PROG 31 AUT D1 AUTOFF BOARD 31 TRACK 0 STA ONTRACK ACT TRACK 31 SPEED 0 FORW INERTIA 48

// Passage du feu au rouge sur détection de train voie A0
PROG 31 AUT D2 AUTOFF BOARD 31 TRACK 0 STA ONTRACK ACT LPO 1 1
PROG 31 AUT D3 AUTOFF BOARD 31 TRACK 0 STA ONTRACK ACT LPO 0 0

// Démarre le train voie A0, B1, C2 et D3 sur déclenchement TIMER 1 selon valeurs de consigne du TCO, cette commande
peut être désactivée pour le retournement du train à crémaillère
PROG 31 AUT D4 AUTOFF BOARD 31 TIMER 1 ACT TRACK 31 SPEED 9 FORW INERTIA 30

// On relance le train si le démarrage a été activé quand le train à crémaillère arrive en haut
PROG 31 AUT D5 AUTOFF BOARD 1 TRACK 2 STA ONTRACK ACT TIMER 1 1

//////////////////////////////////////////////////////////////////////
// CARTE 0 pour les 4 voies internes E0,F1,G2,H3
//TIMER 1 => mode normal de la voie
//TIMER 2 => Retour de la locomotive pour récupérer les wagons détachés
//////////////////////////////////////////////////////////////////////

// Passage en mode automatique
PROG 0 AUT A0 AUTON BOARD 0 GPIO 3 1 ACT AUTOMATIC

// Eteint la led rouge du bouton mode manuel
PROG 0 AUT A1 AUTON BOARD 0 GPIO 3 1 ACT LPO 3 0

// Mode normal de la carte par appui bouton démarrer ou action automatique du train à crémaillère
PROG 0 AUT A2 AUTOFF BOARD 0 GPIO 1 1 ACT TIMER 1 1

// Allume la led du bouton démarrer (clignotement)
PROG 0 AUT A3 AUTOFF BOARD 0 GPIO 1 1 ACT LPO 2 2

// Lance la séquence automatique programmée

// DESCRIPTION DE LA SEQUENCE
// --------------------------
// Sur appui bouton démarrer, le train va effectuer des tours jusqu'à ce que le train à crémaillère quitte la gare
du haut, afin de permettre à ce dernier de faire un retournement de la locomotive sur les voies internes. Le
redémarrage du train s'effectuera par déclenchement du Timer 1, activé quand le wagon du train à crémaillère
déclenche le capteur optique pour retour vers la gare centrale.

// Démarre le train voie E0, F1, G2 et H3
PROG 0 AUT A5 AUTOFF BOARD 0 TIMER 1 ACT TRACK 31 SPEED 11 FORW INERTIA 56

// Passage du feu au vert
PROG 0 AUT A6 AUTOFF BOARD 0 TIMER 1 ACT LPO 1 0
PROG 0 AUT A7 AUTOFF BOARD 0 TIMER 1 ACT LPO 0 1

// Désactive l'arret en gare sur détection de train voie F1
PROG 0 AUT A8 AUTOFF BOARD 0 TIMER 1 ACT AUTOFF D1

//Désactive Le passage du feu au rouge sur détection de train voie F1
PROG 0 AUT A9 AUTOFF BOARD 0 TIMER 1 ACT AUTOFF D2
PROG 0 AUT AA AUTOFF BOARD 0 TIMER 1 ACT AUTOFF D3

// Désactive la relance du train sur détection voie H3
PROG 0 AUT AB AUTOFF BOARD 0 TIMER 1 ACT AUTOFF D4

// Désactivation des commandes pour le train à crémaillère
PROG 0 AUT AC AUTOFF BOARD 0 TIMER 1 ACT AUTOFF D5
PROG 0 AUT AD AUTOFF BOARD 0 TIMER 1 ACT AUTOFF D6
PROG 0 AUT AE AUTOFF BOARD 0 TIMER 1 ACT AUTOFF D7
PROG 0 AUT AF AUTOFF BOARD 0 TIMER 1 ACT AUTOFF D8
PROG 0 AUT AG AUTOFF BOARD 0 TIMER 1 ACT AUTOFF D9
PROG 0 AUT AH AUTOFF BOARD 0 TIMER 1 ACT AUTOFF DA
PROG 0 AUT AI AUTOFF BOARD 0 TIMER 1 ACT AUTOFF DB
PROG 0 AUT AJ AUTOFF BOARD 0 TIMER 1 ACT AUTOFF DC
PROG 0 AUT AK AUTOFF BOARD 0 TIMER 1 ACT AUTOFF DD
```

```
// En mode automatique, même si le démarrage n'est pas lancé, la commande DD ne doit pas être active avant le retour du train à crémaillère
PROG 0 AUT AL AUTOFF BOARD 0 GPIO 3 1 ACT AUTOFF DD

////////////////////////////////////////////////////////////////////////////////////////////////////
// Ces différentes commandes sont pilotées depuis la carte 1 Crémaillère au travers du TIMER 6 (lorsque le train à crémaillère repart de la gare du haut)
// Active l'arret en gare sur détection de train voie F1
PROG 0 AUT B0 AUTOFF BOARD 1 TIMER 6 ACT AUTON D1

// Active le passage du feu au rouge sur détection de train voie F1
PROG 0 AUT B1 AUTOFF BOARD 1 TIMER 6 ACT AUTON D2
PROG 0 AUT B2 AUTOFF BOARD 1 TIMER 6 ACT AUTON D3

// Active la relance du train sur détection voie H3
PROG 0 AUT B3 AUTOFF BOARD 1 TIMER 6 ACT AUTON D4

// Désactive la relance du train sur détection voie H3, car le train à crémaillère passe par là
PROG 0 AUT B4 AUTOFF BOARD 1 TIMER 2 ACT AUTOFF D4

// Désactive l'arret du train en gare principale sur détection du train en voie F1, car le train à crémaillère passe par là
PROG 0 AUT B5 AUTOFF BOARD 1 TIMER 2 ACT AUTOFF D1

// Activation des commandes pour le train à crémaillère dès activation du TIMER 2 de la carte 1
PROG 0 AUT B6 AUTOFF BOARD 1 TIMER 2 ACT AUTON D5
PROG 0 AUT B7 AUTOFF BOARD 1 TIMER 2 ACT AUTON D6
PROG 0 AUT B8 AUTOFF BOARD 1 TIMER 2 ACT AUTON D7
PROG 0 AUT B9 AUTOFF BOARD 1 TIMER 2 ACT AUTON D8
PROG 0 AUT BA AUTOFF BOARD 1 TIMER 2 ACT AUTON D9

////////////////////////////////////////////////////////////////////////////////////////////////////

// Commande pouvant être désactivée
// --------------------------------

// => Gestion automatique décrochage circuit caché et aiguillage E0 / (E0) H1 et E0 / G2, voir les commandes automatiques du TCO

////////////////////////////////////////////////////////////////////////////////////////////////////
// Gestion normale de la carte pour arret du train en gare principale
// Arret en gare, les voies E0, F1, G2 et H3 sont progressivement arrêtées sur détection de train voie F1
PROG 0 AUT D1 AUTOFF BOARD 0 TRACK 1 STA ONTRACK ACT TRACK 31 SPEED 0 FORW INERTIA 60

// Passage du feu au rouge sur détection de train voie F1
PROG 0 AUT D2 AUTOFF BOARD 0 TRACK 1 STA ONTRACK ACT LPO 1 1
PROG 0 AUT D3 AUTOFF BOARD 0 TRACK 1 STA ONTRACK ACT LPO 0 0

// Si le train est détecté sur la voie H3 car il a reçu trop tard la commande d'arrêt, il faut le remettre en route pour un tour
PROG 0 AUT D4 AUTOFF BOARD 0 TRACK 3 STA ONTRACK ACT TRACK 31 SPEED 11 FORW INERTIA 56
////////////////////////////////////////////////////////////////////////////////////////////////////

////////////////////////////////////////////////////////////////////////////////////////////////////
// Gestion de la carte pour le train à crémaillère

// Démarrage train à crémaillère vers décrochage partie cachée
PROG 0 AUT D5 AUTOFF BOARD 1 TIMER 2 ACT TRACK 30 SPEED 7  BACK INERTIA 15

// Pas de passage de train sur la voie externe quand on retourne la locomotive, donc on bloque le train en gare
PROG 31 AUT AH AUTOFF BOARD 1 TIMER 2 ACT AUTON D1
PROG 31 AUT AI AUTOFF BOARD 1 TIMER 2 ACT AUTON D2
PROG 31 AUT AJ AUTOFF BOARD 1 TIMER 2 ACT AUTON D3
PROG 31 AUT AK AUTOFF BOARD 1 TIMER 2 ACT AUTOFF D4

// ON désactive le blocage du train en gare quand le train à crémaillère arrive en haut
PROG 31 AUT AL AUTOFF BOARD 1 TRACK 2 STA ONTRACK ACT AUTON D4

// Activation de la fonction de réinitialisation du capteur optique lorsque la locomotive arrive sur la piste H3
PROG 2 AUT E0 AUTON BOARD 1 TIMER 2 ACT AUTON E1

// Lorsque le train à crémaillère arrive sur la piste 3 on active le décrochage (voir TCO), on remet à zero le détecteur optique, on active les fonctions liées aux capteurs optiques
PROG 2 AUT E1 AUTON BOARD 0 TRACK 3 STA ONTRACK ACT GPIO 0 0

PROG 0 AUT D6 AUTOFF BOARD 0 TRACK 3 STA ONTRACK ACT AUTON DA
PROG 0 AUT D7 AUTOFF BOARD 0 TRACK 3 STA ONTRACK ACT AUTON DB
PROG 0 AUT D8 AUTOFF BOARD 0 TRACK 3 STA ONTRACK ACT AUTON DC
PROG 0 AUT D9 AUTOFF BOARD 0 TRACK 3 STA ONTRACK ACT AUTON DD

// Lorsque la locomotive passe sur le capteur optique après le système de dételage, on inverse le sens de la locomotive et des aiguillages et on stoppe le dételage (voir TCO)
PROG 0 AUT DA AUTOFF BOARD 2 GPIO 0 1 ACT TRACK 30 SPEED 6 FORW INERTIA 27

// On désactive la fonction de remise à zero du capteur optique
PROG 2 AUT E2 AUTON BOARD 2 GPIO 0 1 ACT AUTOFF E1
```

```
// Le deuxième passage devant le capteur optique stoppe le dételage (voir TCO)

// Lorsque la locomotive passe sur l'autre capteur optique, on remet les aiguillages correctement (voir TCO) et on
inverse le sens de la locomotive
PROG 0 AUT DB AUTOFF BOARD 2 GPIO 0 3 ACT TRACK 30 SPEED 6 BACK INERTIA 27

// Dès que le premier wagon passe devant le capteur optique après le dételage on retourne vers la gare centrale
PROG 0 AUT DC AUTOFF BOARD 2 GPIO 0 5 ACT TRACK 30 SPEED 8 FORW INERTIA 27

// ACtivation du TIMER 1 pour retour vers le mode normal de la carte
PROG 0 AUT DD AUTOFF  BOARD 2 GPIO 0 5 ACT TIMER 1 8

////////////////////////////////////////////////////////
// CARTE 1 pour les 4 voies de la crémaillère I0,J1,K2,L3
// TIMER 1 => Attente gare du haut avant décrochage locomotive
// TIMER 2 => démarrage du train à crémaillère vers les voies centrales pour retournement
// TIMER 3 => mode normal de la voie
// TIMER 4 => démarrage retour arrière locomotive
// TIMER 5 => démarrage retour avant pour accrochage locomotive
// TIMER 6 => Retour vers la gare centrale
////////////////////////////////////////////////////////

// Passage en mode automatique
PROG 1 AUT A0 AUTON BOARD 1 GPIO 3 1 ACT AUTOMATIC

// Eteint la led rouge du bouton mode manuel
PROG 1 AUT A1 AUTON BOARD 1 GPIO 3 1 ACT LPO 3 0

// Allume la led du bouton démarrer (Clignotement)
PROG 1 AUT A2 AUTOFF BOARD 1 GPIO 1 1 ACT LPO 2 2

// Lance la séquence automatique programmée

// DESCRIPTION DE LA SEQUENCE
// -------------------------
// Sur appui bouton démarrer, le train à crémaillère quitte la gare principale pour se rendre à la gare du haut.
Après un temps d'attente de 5 unités, la locomotive se decroche et se positionne à l'arrière du train pour revenir
vers la gare centrale. A l'arrivée à la gare centrale, après un temps d'attente de 5 unités, le train à crémaillère
va sur la voie interne pour un décrochage de la locomotive et positionnement à l'avant du train en voie cachée avant
de revenir en gare centrale puis remonter directement à la gare du haut

// Arme TIMER 3 pour déclenchement mode normal automatique
PROG 1 AUT A3 AUTOFF BOARD 1 GPIO 1 1 ACT TIMER 3 1

// Positionne correctement les aiguillages du haut
PROG 1 AUT A4 AUTOFF BOARD 1 TIMER 3 ACT LPO 4 0

// Demarre le train (qui doit être quelque part sur la voie I0 ou L3 (gare) )
PROG 1 AUT A5 AUTOFF BOARD 1 TIMER 3 ACT TRACK 31 SPEED 8 FORW INERTIA 27

// Active détection arrivée en gare du haut et activation TIMER
PROG 1 AUT A7 AUTOFF BOARD 1 TIMER 3 ACT AUTON D2
PROG 1 AUT A8 AUTOFF BOARD 1 TIMER 3 ACT AUTON D3

// Désactive arret du train en gare centrale
PROG 1 AUT A9 AUTOFF BOARD 1 TIMER 3 ACT AUTOFF D1

// Désactive armement TIMER 2 pour décrochage bas
PROG 1 AUT A9 AUTOFF BOARD 1 TIMER 3 ACT AUTOFF D4
PROG 1 AUT AA AUTOFF BOARD 1 TIMER 3 ACT AUTOFF D5

// Désactive Détection arrivée en gare du haut et activation TIMER
PROG 1 AUT AB AUTOFF BOARD 1 TIMER 1 ACT AUTOFF D2
PROG 1 AUT AC AUTOFF BOARD 1 TIMER 1 ACT AUTOFF D3

// Démarrage pour décrochage wagon
PROG 1 AUT AD AUTOFF BOARD 1 TIMER 1 ACT TRACK 31 SPEED 4 FORW INERTIA 40

// Arme TIMER 4 pour retour arrière locomotive
PROG 1 AUT AE AUTOFF BOARD 1 TIMER 1 ACT TIMER 4 2

// retour arrière locomotive sur déclenchement TIMER 4
PROG 1 AUT AF AUTOFF BOARD 1 TIMER 4 ACT TRACK 31 SPEED 5  BACK INERTIA 10

// changement aiguillage sur déclenchement TIMER 4
PROG 1 AUT AG AUTOFF BOARD 1 TIMER 4 ACT LPO 4 1

// Arme TIMER 5 pour retour avant locomotive
PROG 1 AUT AH AUTOFF BOARD 1 TIMER 4 ACT TIMER 5 2

//Retour avant de la locomotive au déclenchement TIMER 5
PROG 1 AUT AI AUTOFF BOARD 1 TIMER 5 ACT TRACK 31 SPEED 4 FORW INERTIA 27

// changement aiguillage sur déclenchement TIMER 5
PROG 1 AUT AJ AUTOFF BOARD 1 TIMER 5 ACT LPO 4 0
```

```
// Arme TIMER 6 pour retour final locomotive
PROG 1 AUT AK AUTOFF BOARD 1 TIMER 5 ACT TIMER 6 2

// Retour du train vers la gare centrale (voie I0, J1, K2 et L3)
PROG 1 AUT AL AUTOFF BOARD 1 TIMER 6 ACT TRACK 31 SPEED 5  BACK INERTIA 15

// Active arret du train en gare centrale
PROG 1 AUT AM AUTOFF BOARD 1 TIMER 6 ACT AUTON D1

// Active armement TIMER 2 pour décrochage bas
PROG 1 AUT AN AUTOFF BOARD 1 TIMER 6 ACT AUTON D4
PROG 1 AUT AO AUTOFF BOARD 1 TIMER 6 ACT AUTON D5

////////////////////////////////////////////////////////////////////////
// Démarre le train qui passe sur la CARTE 0
PROG 1 AUT AP AUTOFF BOARD 1 TIMER 2 ACT TRACK 31 SPEED 7  BACK INERTIA 15

// Désactive Arret du train lors de l'arrivée en gare centrale
PROG 1 AUT AQ AUTOFF BOARD 1 TIMER 2 ACT AUTOFF D1

// Arme TIMER 3 pour remise en mode normal de la voie
PROG 1 AUT AR AUTOFF BOARD 1 TIMER 2 ACT TIMER 3 2

////////////////////////////////////////////////////////////////////////

// Commande pouvant être désactivée
// --------------------------------

// Arret du train lors de l'arrivée en gare centrale (voie I0, J1, K2 et L3)
PROG 1 AUT D1 AUTOFF BOARD 1 TRACK 0 STA ONTRACK ACT TRACK 31 SPEED 0  BACK INERTIA 100

// Détection du train sur la voie K, arret des voies I0, J1, K2 et L3
PROG 1 AUT D2 AUTOFF BOARD 1 TRACK 2 STA ONTRACK ACT TRACK 31 SPEED 0 FORW INERTIA 27

// Arme TIMER 1 pour attente en gare du haut pendant 5 unités
PROG 1 AUT D3 AUTOFF BOARD 1 TRACK 2 STA ONTRACK ACT TIMER 1 5

// Arme TIMER 2 pour séquence de décrochage vers la partie cachée
PROG 1 AUT D4 AUTOFF BOARD 1 TRACK 0 STA ONTRACK ACT TIMER 2 5

// Désactive l'armement du TIMER 2
PROG 1 AUT D5 AUTOFF BOARD 1 TRACK 0 STA ONTRACK ACT AUTOFF D4


////////////////////////////////////////////////////////////////////////
// CARTE 2 pour la voie 0 du tramway et les 2 voies temporaire Faller AMS
// TIMER 1 => Temps d'attente devant les gares principales et intermédiaires à l'aller
// TIMER 2 => Temps d'attente à la gare devant la grue
// TIMER 3 => Temps d'attente à la gare intermédiaire au retour
////////////////////////////////////////////////////////////////////////

// Passage en mode automatique
PROG 2 AUT A0 AUTON BOARD 2 GPIO 3 1 ACT AUTOMATIC

// Eteint la led rouge du bouton mode manuel
PROG 2 AUT A1 AUTON BOARD 2 GPIO 3 1 ACT LPO 3 0

// Allume la led du bouton démarrer (clignotement)
PROG 2 AUT A2 AUTOFF BOARD 2 GPIO 1 1 ACT LPO 2 2

// Lance la séquence automatique programmée

// DESCRIPTION DE LA SEQUENCE
// --------------------------
// Sur appui bouton démarrer, et sous réserve que le Tramway ait bien été positionné devant la gare centrale, il va
venir se positionner devant la gare du tramway à droite de la gare centrale, puis suivre un cycle d'aller/retour
régulier avec un arret à l'aller et au retour à la gare intermédiaire. Chaque arret est programmé sur un TIMER de 5
unités

// Démarre le tramway pour le ramener sur la droite, le tramway doit être devant la gare de train
PROG 2 AUT A3 AUTOFF BOARD 2 GPIO 1 1 ACT TRACK 0 SPEED 5 BACK INERTIA 27

// Réinitialise GPIO 2 pour comptage position tramway
PROG 2 AUT A4 AUTOFF BOARD 2 GPIO 1 1 ACT GPIO 2 0

// Démarre les voies de voiture (temporaire)
PROG 2 AUT A5 AUTOFF BOARD 2 GPIO 1 1 ACT TRACK 1 SPEED 5 FORW INERTIA 1
PROG 2 AUT A6 AUTOFF BOARD 2 GPIO 1 1 ACT TRACK 2 SPEED 6 FORW INERTIA 1

// Allume led TCO droite M0
PROG 2 AUT A7 AUTOFF BOARD 2 GPIO 2 2 ACT LPO 5 1

// Stop le tramway
PROG 2 AUT A8 AUTOFF BOARD 2 GPIO 2 2 ACT TRACK 0 SPEED 0 BACK INERTIA 27
```

```
// Arme TIMER 1 d'attente en gare
PROG 2 AUT A9 AUTOFF BOARD 2 GPIO 2 2 ACT TIMER 1 5

// Redémarre le tramway sur déclenchement TIMER 1
PROG 2 AUT AA AUTOFF BOARD 2 TIMER 1 ACT TRACK 0 SPEED 5 FORW INERTIA 27

// Eteint led TCO droite M0
PROG 2 AUT AB AUTOFF BOARD 2 GPIO 2 4 ACT LPO 5 0

// Gare intermédiaire
PROG 2 AUT AC AUTOFF BOARD 2 GPIO 2 6 ACT TRACK 0 SPEED 0 FORW INERTIA 27

// Arme TIMER 1 d'attente en gare intermédiaire
PROG 2 AUT AD AUTOFF BOARD 2 GPIO 2 6 ACT TIMER 1 5

// ferme les barrieres
PROG 2 AUT AE AUTOFF BOARD 2 GPIO 2 8 ACT LPO 0 0
PROG 2 AUT AF AUTOFF BOARD 2 GPIO 2 8 ACT LPO 1 1

PROG 2 AUT AG AUTOFF BOARD 2 TIMER 2 ACT LPO 0 0
PROG 2 AUT AH AUTOFF BOARD 2 TIMER 2 ACT LPO 1 1

// ouvre les barrieres
PROG 2 AUT AI AUTOFF BOARD 2 GPIO 2 12 ACT LPO 0 1
PROG 2 AUT AJ AUTOFF BOARD 2 GPIO 2 12 ACT LPO 1 0

PROG 2 AUT AK AUTOFF BOARD 2 GPIO 2 20 ACT LPO 0 1
PROG 2 AUT AL AUTOFF BOARD 2 GPIO 2 20 ACT LPO 1 0

PROG 2 AUT AM AUTOFF BOARD 2 GPIO 2 2 ACT LPO 0 1
PROG 2 AUT AN AUTOFF BOARD 2 GPIO 2 2 ACT LPO 1 0

// Stop le tramway
PROG 2 AUT AO AUTOFF BOARD 2 GPIO 2 14 ACT TRACK 0 SPEED 0 FORW INERTIA 27

// Allume led TCO gauche M0
PROG 2 AUT AP AUTOFF BOARD 2 GPIO 2 14 ACT LPO 4 1

// Arme TIMER 2 d'attente en gare
PROG 2 AUT AQ AUTOFF BOARD 2 GPIO 2 14 ACT TIMER 2 5

// Redémarre le tramway sur déclenchement TIMER 2
PROG 2 AUT AR AUTOFF BOARD 2 TIMER 2 ACT TRACK 0 SPEED 5 BACK INERTIA 27

// Eteint led TCO gauche M0
PROG 2 AUT AS AUTOFF BOARD 2 GPIO 2 16 ACT LPO 4 0

// Gare intermédiaire retour
PROG 2 AUT AT AUTOFF BOARD 2 GPIO 2 22 ACT TRACK 0 SPEED 0 FORW INERTIA 27

// Arme TIMER 3 d'attente en gare intermédiaire
PROG 2 AUT AU AUTOFF BOARD 2 GPIO 2 22 ACT TIMER 3 5

// Redémarre le tramway sur déclenchement TIMER 3
PROG 2 AUT AV AUTOFF BOARD 2 TIMER 3 ACT TRACK 0 SPEED 5 BACK INERTIA 27

// Réinitialise GPIO 2 pour nouveau cycle aller-retour
PROG 2 AUT AW AUTOFF BOARD 2 GPIO 2 24 ACT GPIO 2 0

////////////////////////////////////////////////////////////////////////
// CARTE 3 AIGUILLAGES
////////////////////////////////////////////////////////////////////////

PROG 3 AUT M1 AUTON BOARD 3 GPIO 0 0 ACT GPIO 0 0
PROG 3 AUT M2 AUTON BOARD 3 GPIO 1 0 ACT GPIO 1 0
PROG 3 AUT M3 AUTON BOARD 3 GPIO 2 0 ACT GPIO 2 0
PROG 3 AUT M4 AUTON BOARD 3 GPIO 3 0 ACT GPIO 3 0

//============================================================
// GESTION DES COMMANDES DU TCO EN MODE AUTOMATIQUE
//============================================================

// Pilotage automatique décrochage circuit caché
PROG 31 AUT T0 AUTON BOARD 0 GPIO 3 0 ACT AUTOFF T2
PROG 31 AUT T1 AUTON BOARD 0 TIMER 1 ACT AUTOFF T2
PROG 31 AUT T2 AUTON BOARD 0 TRACK 3 STA ONTRACK ACT LPO 4 1
PROG 31 AUT T3 AUTON BOARD 1 TIMER 2 ACT AUTON T2
PROG 31 AUT T4 AUTON BOARD 2 GPIO 0 1 ACT AUTOFF T2
PROG 31 AUT T5 AUTON BOARD 2 GPIO 0 1 ACT LPO 4 0

// Commande automatique aiguillage E0 / (E0) H1 et E0 / G2
PROG 1 AUT T0 AUTON BOARD 0 GPIO 3 0 ACT AUTOFF T2
PROG 1 AUT T1 AUTON BOARD 0 TIMER 1 ACT AUTOFF T2
PROG 1 AUT T2 AUTON BOARD 2 GPIO 0 1 ACT LPO 0 1
PROG 1 AUT T3 AUTON BOARD 1 TIMER 2 ACT AUTON T2
PROG 1 AUT T4 AUTON BOARD 2 GPIO 0 4 ACT AUTOFF T2
```

```
PROG 1 AUT T5 AUTON BOARD 2 GPIO 0 3 ACT LPO 0 0

// Pilotage automatique décrochage gare crémaillère haut
PROG 0 AUT T1 AUTON BOARD 1 TIMER 1 ACT LPO 4 1
PROG 0 AUT T2 AUTON BOARD 1 TIMER 4 ACT LPO 4 0

PROG 3 AUT T1 AUTON BOARD 1 TIMER 2 ACT LPO 1 0
PROG 3 AUT T2 AUTON BOARD 1 TIMER 2 ACT LPO 0 1

// Commande automatique B1 / CROSS
PROG 3 AUT T3 AUTON BOARD 31 GPIO 1 1 ACT LPO 2 1
PROG 3 AUT T4 AUTON BOARD 31 GPIO 1 1 ACT LPO 3 0

// Commande automatique D3 / H3 et C2 / G2
PROG 3 AUT T5 AUTON BOARD 31 GPIO 1 1 ACT LPO 5 0
PROG 3 AUT T6 AUTON BOARD 0 GPIO 1 1 ACT LPO 5 0

// Commande automatique CROSS
PROG 3 AUT T7 AUTON BOARD 0 TIMER 1 ACT LPO 1 1
PROG 3 AUT T8 AUTON BOARD 0 TIMER 1 ACT LPO 0 0

PROG 3 AUT T9 AUTON BOARD 1 TIMER 2 ACT LPO 1 0
PROG 3 AUT TA AUTON BOARD 1 TIMER 2 ACT LPO 0 1

//============================================================
// GESTION DES COMMANDES DU TCO EN MODE MANUEL
//============================================================

// Pilotage manuel décrochage circuit caché
PROG 31 AUT T6 AUTON BOARD 3 GPIO 3 1 ACT LPO 4 1
PROG 31 AUT T7 AUTON BOARD 3 GPIO 3 0 ACT LPO 4 0

// Remise à zero GPIO
PROG 31 AUT T8 AUTON BOARD 31 GPIO 0 0 ACT GPIO 2 0
PROG 31 AUT T9 AUTON BOARD 31 GPIO 2 0 ACT GPIO 0 0

// Pilotage manuel décrochage gare crémaillère haut
PROG 0 AUT T3 AUTON BOARD 3 GPIO 2 1 ACT LPO 4 1
PROG 0 AUT T4 AUTON BOARD 3 GPIO 2 0 ACT LPO 4 0

// Commande manuel D3 / H3 et C2 / G2
PROG 0 AUT T5 AUTON BOARD 0 GPIO 0 0 ACT GPIO 0 0
PROG 0 AUT T6 AUTON BOARD 0 GPIO 2 0 ACT GPIO 2 0

// Commande manuel aiguillage E0 / (E0) H1 et E0 / G2
PROG 1 AUT T6 AUTON BOARD 1 GPIO 2 1 ACT LPO 0 1
PROG 1 AUT T7 AUTON BOARD 1 GPIO 0 1 ACT LPO 0 0
PROG 1 AUT T8 AUTON BOARD 1 GPIO 2 0 ACT GPIO 2 0
PROG 1 AUT T9 AUTON BOARD 1 GPIO 0 0 ACT GPIO 0 0

// Commande manuel aiguillage I0 / J1 , K2
PROG 1 AUT TA AUTON BOARD 3 GPIO 0 1 ACT LPO 4 1
PROG 1 AUT TB AUTON BOARD 3 GPIO 1 1 ACT LPO 4 0

// Commande manuel CROSS
PROG 3 AUT TB AUTON BOARD 31 GPIO 0 1 ACT LPO 1 1
PROG 3 AUT TC AUTON BOARD 31 GPIO 2 1 ACT LPO 0 1
PROG 3 AUT TD AUTON BOARD 31 GPIO 2 1 ACT LPO 1 0
PROG 3 AUT TE AUTON BOARD 31 GPIO 0 1 ACT LPO 0 0

// Commande manuel D3 / H3 et C2 / G2
PROG 3 AUT TF AUTON BOARD 0 GPIO 0 1 ACT LPO 5 1
PROG 3 AUT TG AUTON BOARD 0 GPIO 2 1 ACT LPO 5 0

// Commande manuel B1 / CROSS
PROG 3 AUT TH AUTON BOARD 31 GPIO 0 2 ACT LPO 2 1
PROG 3 AUT TI AUTON BOARD 31 GPIO 2 2 ACT LPO 3 1
PROG 3 AUT TJ AUTON BOARD 31 GPIO 2 2 ACT LPO 2 0
PROG 3 AUT TK AUTON BOARD 31 GPIO 0 2 ACT LPO 3 0

//============================================================
// REMISE EN ROUTE DES AUTOMATISMES
//============================================================

RUNALL
```

8 Plants & Hardware Systems

8.1 "Entry-level" ≲ 300 €

- **PIKO SmartControl light (55017)** — 2 A control unit + housing/handle, DCC, scalable via LocoNet (boosters). It can be found around ~€140–200 depending on the store. Coastal DCCConrad Electronic
- **Digitrax Zephyr Express (DCS52)** — ~3 A, DCC, USB, spikes/acc. and backroom control unit possible via Digitrax modules. Typical UK prices ~£190–£225. Geizhals.deRails of Sheffield
- **NCE Power Cab** — 2 Amp handle/center, simple and reliable, scalable to more powerful systems (SB5, etc.). Often ~£190–£210. Rails of Sheffielddcctrainautomation.co.uk
- **Märklin Mobile Station 2 (60657)** — Multi-protocol controller (mfx/motorola/DCC) for Märklin/Trix networks; requires track box. Price ~£80–£120 depending on kits. PriceRunnermaerklin.de
- **Roco/Fleischmann z21 start** — affordable DCC core (often in a box), Wi-Fi and app activable via upgrade; good gateway to the z21 ecosystem. In-Gauge Ltd Dapol Spares & Repairs

For whom? Small/medium networks, 1–3 simultaneous trains, first scanning.

8.2 "Mid-range" ~ €300–600

- **Roco z21 (black, 10820)** — LAN/Wi-Fi/app, RailCom, smartphone control, large ecosystem (WLAN-Maus, MultiMaus, apps, iTrain/TrainController, etc.). Typically ~£350–£460. piko-shop.deDCCconcepts
- **Gaugemaster / MRC Prodigy Advance² (DCC02)** — 3.5 A control panel, club thinking, extension by wired/wireless joysticks and modules. ~£400–£495 depending on offers. gaugemasterretail.comCoastal DCC
- **Lenz LZV200 (or Set101)** — XpressNet/RailCom reference, very rugged; LZV200 single UVP €344.50; Set101 UVP 399–589 € depending on the version (wired/wireless). Lenz Elektronik+1
- **ESU CabControl (50310)** — built-in 7 A control panel + wireless touch control, Wi-Fi/LAN, ECoSlink (ESU boosters). Typically positioned ~€500–600. esu.euovrtrains.com
- **Uhlenbrock Intellibox 2neo (65150)** — multi-bus (LocoNet, XpressNet, s88...), modern "all-in-one" approach. dcctrainautomation.co.uk

For whom? Scalable HO/OO networks (3–6 trains), need for Wi-Fi/app, feedback and convenient PC automation.

8.3 "High-end / club / large network" ≥ 600 €

- **ESU ECoS 2.1 / 2.5 (50210/50220)** — 7" touchscreen, 6–7 A, DCC+mfx/M4, RailCom, ECoSlink bus; popular in Europe. Price often ~£590–£700 (€≈680–800). Coastal DCCmodellbahnshop-lippe.com
- **Märklin Central Station 3 (CS3, 60226)** — multi-protocol (mfx/mfx+/DCC/Motorola) with display, TCO, built-in automation. Observed prices ~570 €+. Idealo
- **ZIMO MX10 / MX10EC** — extreme power (up to 20 A), CAN, advanced RailCom; clearly "pro/giant layout". Price ~£1,050 in the UK / ~US$1,559. Coastal DCCtrainli.com
- **Massoth DiMAX 1210Z (Garden/G)** — up to 12 A, designed for outdoor G scale. In UK stores ~£915. Garden Railway Specialists

For whom? Large networks, clubs, high intensity (many trains/lighting/sound), advanced integration.

8.4 PC-centric solutions & software (for automation)

- **iTrain (Lite → Pro)** — €119 / €209 / €269 / €349 depending on the edition; TCO, routes, retro, multi-interfaces. berros.eu
- **TrainController (Railroad & Co.) Bronze/Silver/Gold** — widely used in advanced automation; Gold sells for around ~€650 (varies by country/tax). freiwald.comRMweb
- **Rocrail & JMRI** — free/open-source, very capable (TCO, auto, programming). majorgeeks.comjmri.org
- **Hornby HM7000 (BLE)** — control of locos via Bluetooth decoders + app, without a "big" classic DCC control unit; interesting in OO/GB. uk.hornby.comGoogle Play
- **Hornby eLink + RailMaster** — historical PC interface (fluctuating availability these days). uk.hornby.comnewmodellersshop.co.uk

All these software programs communicate with the listed control panels via LAN/USB (z21, ECoS, Lenz, Digitrax, NCE, DCC-EX, etc.). Check the exact compatibility on the hardware driver side. jmri.org

---

8.5 DIY/open-source (very economical)

- **DCC-EX (EX-CommandStation / CSB1)** — open-source Arduino control panel (Wi-Fi possible), ready-to-use from ~€115–€120 in Europe; extremely scalable. dcc-ex.com+1

---

9 Market Estimate (units/year)

There are no precise public statistics by model or manufacturer for **control centres**. However, we can give a reasonable **order of magnitude** by triangulating:

1. **Size of the model railroad market** (all products combined) :p several sources of market research place global **turnover** around **$1.16–1.22 billion in 2024–2025**, with moderate growth. Market IntelligenceBusiness Research Insights
2. **Share of "control/electronics DCC"** in hobbyist expenditure: conservative assumption **8–12%** (the bulk of purchases remaining rolling stock, tracks, decoration). *Unpublished hypothesis* (I explain it for transparency).
3. **Average price of a central unit** (all ranges combined, weighted by sales): **~€400–500** (in view of the prices observed above). *(Based on detailed prices and in-store readings.)*

Range calculation

- Low bound: $1.16 billion × 8% ≈ $93 million; $93 million / $500 ≈ **~185,000 units/year**.
- High Bound: $1.79 billion × 12% ≈ $215 million; $215 million / $450 ≈ **~480,000 units/year**. Market Intelligence

👉 **Plausible order of magnitude: ~200,000 to 500,000 power plants sold/year worldwide.**
In addition, there are **tens of thousands** of paid software licenses (iTrain, TrainController) and a large volume of users of free solutions (JMRI/Rocrail), but the exact figures are not published. berros.eufreiwald.comjmri.org

Note : depending on the region (Germany/Austria/Switzerland, United Kingdom, United States, Japan), ecosystem preferences vary (Märklin/mfx on the 3-rail side, z21/ECoS/Lenz on the DCC 2-rail side), which influences the distribution of sales by brand. The above orders of magnitude remain global.

9.1 Quick advice to choose

- **You get started** : PIKO SmartControl light, NCE Power Cab, Digitrax Zephyr, z21 start.
- **Scalable HO network with smartphone & PC** : Roco z21 (black), Lenz LZV200 (with iTrain), ESU CabControl.
- **Club / large network / garden** : ESU ECoS, Märklin CS3 (3-rail), ZIMO MX10, Massoth 1210Z.

🔎 Key features of this DIY interface

- **PIC18F4585/4680 microcontroller** with **DCC NMRA and analog PWM management**
- **4 card-controlled channel sections** (DCC or analog, but not mixed on the same board)
- **Integrated vehicle presence detection** (diodes + ADC)
- **6 low-power** outputs (lighting, signals, small needle motors)
- **4 configurable GPIOs + 4 "track status" inputs** (buttons, sensors, potentiometers, 7-segment displays)
- **CAN bus (500 kBaud)** to link multiple boards (up to 31 slaves + master)
- **PC interface** via RS232/USB FTDI + ASCII text command language

- **Programmable automations** (timers, train conditions, GPIO, etc.), stored in compressed EEPROM
- **PC Qt (RailManager UI) application** under development to simplify graphical programming

In other words: it is a **distributed control system**, hybrid **rail + road (Faller AMS),** with **local automation + PC control**.

💰 Positioning in relation to the existing market

If we compare it to the products in the overview I gave you earlier:

- **Related features:**
  - Multi-point control such as **a booster/modular control unit** (e.g. Lenz LZV200 + modules, Roco z21 with extensions).
  - Standard NMRA DCC **management** (such as Digitrax, NCE, ESU, Lenz).
  - **Programmable local automation** (rare at the entry-level; closer to high/mid-range solutions).
  - **Built-in presence detection** → advantage over the competition (often sold as separate modules).
- **Differentiators:**
  - AMS Dual Purpose Rail + Road **Compatibility** → unique, niche yet innovative.
  - Reliable industrial CAN **bus** → close to pro systems (ZIMO, Märklin CS3, Massoth).
  - **Component prices (BOM page 23)** ≈ ~€50–80 of raw components → low production cost, margin possible.

📊 Likely range of commercialization

- **Entry-level (€<300**): often limited to a basic control panel, without advanced automation. → This system **offers more** (multi-board, detection, automation).
- **Mid-range (€300–600):** where you can find **z21 black (~€350–450), NCE Power Pro**, **Lenz LZV200 (~€400).** These systems manage PCs, feedback signals, and several trains. → **This interface aligns here** in terms of power and modularity.
- **High-end (€>600):** typically ESU ECoS, Märklin CS3, ZIMO MX10, with touch screens, proprietary buses, high intensity. → This product is **less "premium UI" oriented**, so not in this category.

👉 **So I would position this development in the *mid-range range (~300–500 € per master kit complete with 1 card + PC interface).***
Each additional slave card could sell for ~€100–150, which is still competitive with separate retro/detection modules (often €80–120 each from Lenz, Roco, Digitrax).

🚀 Market potential

- Niche "Faller AMS + HO mixed" (very differentiating, almost without competition).
- Wider market: model railways wanting **simple automation without a PC** → could compete with sets such as **z21 + detection modules**.
- Estimate: if we start on a global market of 200k–500k power plants/year (see my previous calculation), an **innovative niche** like this one could capture a few **% → 1,000–5,000 realistic units/year** if well distributed.

✅ In summary: This interface **is positioned between a black z21 and a Lenz LZV200**, so in the **mid-range (300–500 €)** with a **competitive advantage on the integrated detection and AMS support**. This is clearly above the entry-level but below the large premium power plants.

10🛠️ Product Range Simulation

10.1 "Starter Master" Pack

- **Contents** :
  - 1 Master card (PIC + USB/RS232 interface),
  - DCC/PWM management of 4 sections,
  - 6 auxiliary outputs,
  - 4 GPIO, built-in sensing,
  - RailManager UI (PC software).
- **Positioning** : entry into the ecosystem.
- **Recommended retail price (RRP): €349**

👉 Competitors: Roco z21 black (~€350–450), Lenz LZV200 (~€400).
👉 Advantage: Integrated detection, including simple automation.

10.2 "Slave Expansion" card

- **Contents** :
  - Identical slave card (4 sections + detection + outputs),
  - Connected via CAN bus, automatically addressable.
- **Recommended retail price** : **129 €**

👉 Comparison: Lenz/Roco retro modules = ~100–120 € for *just* the detection (without control).
👉 Here you offer **detection + control** in a single module → big perceived advantage.

10.3 Advanced Automation Pack

- **Contents** :
  - RailManager UI (full version) with graphical editor (timers, conditions, scenarios),
  - Firmware enabling advanced distributed automation features (compressed EEPROM, macros).
- **Recommended retail price** : **€99** (software license).

👉 Similar to iTrain "Lite" (119 €) or TrainController Bronze (≈150 €).
👉 Differentiator: *embedded* automation (not just PC).

10.4 "AMS Road Integration" package

- **Contents** :
  - Faller AMS specific road network adaptation kit (rails + dedicated sensors),
  - RailManager documentation/plug-ins.
- **Recommended retail price** : **79 €**

👉 Unique product **on the market** (no direct equivalent).
👉 Can appeal to AMS collectors and clubs that mix rail + road.

---

11 📊 Summary range

- **Master Starter** : €349
- **Slave Extension** : 129 €
- **Automation Advanced (software):** €99
- **AMS Road Kit** : 79 €

➡️ Example "club" configuration: 1 Master (349 €) + 3 Slaves (3×129 €) + Automation (99 €) = **~835 € TTC→ consistent with an ECoS ESU (~700–800 €) but with more modularity**.

---

12 🚀 Market strategy

- **Core target** : advanced modellers (clubs + demanding individuals).
- **Advantage** : Very competitive price per section compared to Lenz/Roco/ESU modular solutions.
- **Differentiating niche** : AMS Road Integration = unique element, attracts a small loyal community.

13 Mini Business Plan – Rail Interface/AMS

For those who want to go further….

13.1 Product

- **Master Starter** : Modular control unit with 4 sections, integrated sensing, basic automation, PC UI. (RRP: €349)
- **Slave Extension** : Extra 4-section module. (RRP: €129)
- **Automation Advanced** : Software license for scenarios and distributed automation. (RRP: 99 €)
- **AMS Road Kit** : Faller AMS road network integration. (RRP: €79)

⚡ Differentiators:

- Built-in detection by default (often modular option elsewhere).
- AMS rail + road compatible (unique).
- Distributed CAN (industrial reliability) architecture.

13.2 Market and potential

- Global DCC Plants Market = **200k–500k units/year**【Previous conversation】
- Niche penetration hypothesis: **0.5%** (≈ 1,500–2,500 sales/year) in the 1st year, gradually increasing if clubs/forums are adopted.
- Likely distribution:
  - 1 Master each customer
  - 1.5 slaves/average customer
  - 40% customers take Automation license
  - 20% take AMS Road Kit

13.3 Revenue model (per 1,000 customers)

Product	Price (€)	Customers (%)	Volume (u)	Turnover (€)
Master Starter	349	100 %	1 000	**349 000**
Slave Extension	129	150 %	1 500	**193 500**
Automation Advanced	99	40 %	400	**39 600**
AMS Road Kit	79	20 %	200	**15 800**
Total annual turnover				**€598,000**

13.4 Cost structure

- **BOM Master** ≈ 70 € (PCB + components + housing + mounting).
- **BOM Slave** ≈ 50 €
- AMS software & kit → margin of almost 90%.
- **Weighted average cost**≈ 45% of the public price → **gross margin 55%.**

👉 On €598k turnover, **gross margin ≈€329k**.

13.5 Fixed Expenses (Year 1)

- R&D finalization: 60 k€ (2 devs 6 months).
- Moulding of enclosures + CE certifications: 40 k€.
- Marketing (website, trade shows, forums, specialised pubs): 30 k€.
- After-sales service/support (freelance/part-time): 20 k€.
- **Fixed year 1 ≈ 150 k€**.

13.6 Estimated ROI

- **Annual gross margin** : €329k
- **Fixed year 1** : 150 k€
- **Operating income** : ≈ **+€179k**

➡️ ROI over 1 year: **>100%** (investment €150k, return ~€179k net from year 1 if 1,000 customers reached).
➡️ Break-even: **~450 customers** (0.1–0.2% of the overall market).

13.7 Competitive comparison (price per controlled section)

System	Central Prize	Detection included?	Price per section (4)	Extensions (4 sections)
This product	349 €	✅ Yes	87 €/section	129 € (32 €/section)
Roco z21 black	~399 €	❌ (detection in +)	100 €/section	+100 €/4 detections
Lenz LZV200	~400 €	❌	100 €/section	+110 €/8 detections
ESU ECoS 2.1	~700 €	❌	175 €/section	+ Modules

👉 Clearly competitive in €/section **with detection included**, especially once several slaves are added.

14🎯 Conclusion

- This product **is in the mid-range (€300–500)** but **offers more for the price** (sensing, modularity, AMS).
- **Differentiation strategy** : combine rail + road, reduce total cost per controlled section.
- **Viable business case**: ROI possible in the first year with only 450–500 customers, which is realistic given the size of the market.
